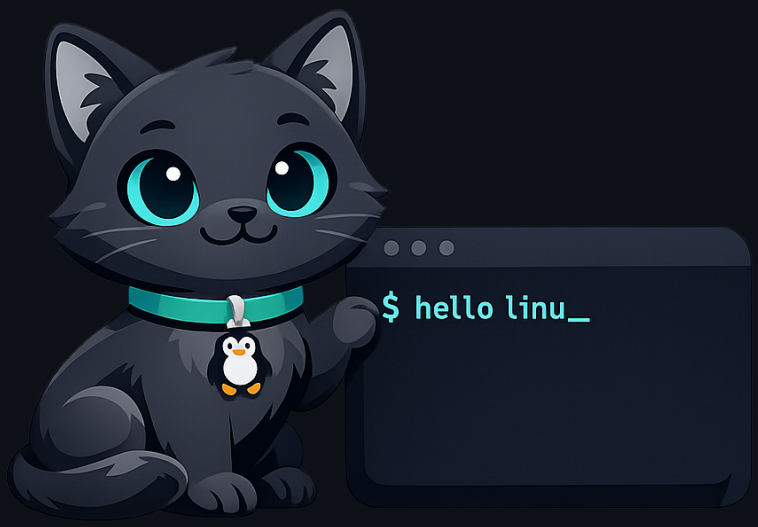
হাতেকলমে LINUX



লিনুকে পোষ মানানো

হাতেকলমে Linux — তোমার পোষা বিড়ালের সাথে

লেখক · ইফতে সামউল ওহী

Licensed to sakilorb@gmail.com · Do not redistribute

সূচিপত্র

পর্ব ১: লিনুকে বাসায় আনা ৬

পর্ব ২: লিনুর ঘরদোর চেনা (Filesystem) ২৪

পর্ব ৩: লিনুর খাওয়া-দাওয়া (Packages & Software) ৪৬

পর্ব ৪: লিনুকে হুকুম শেখানো (Power User) ৬৩

পর্ব ৫: লিনুকে প্রশিক্ষণ (Scripting & Automation) ৯৫

পর্ব ৬: লিনু বাইরের জগতে (Networking & Server) ১১৭

পরিশিষ্ট ১৫২

অধ্যায় ১: লিনুর সাথে পরিচয় ৭

অধ্যায় ২: লিনুকে বাসায় আনা ১২

অধ্যায় ৩: লিনুর ভাষা বোঝা ১৮

অধ্যায় ৪: লিনুর বাড়ি ঘোরা ২৫

অধ্যায় ৫: লিনুর জিনিসপত্র গোছানো ৩২

অধ্যায় ৬: লিনু হারিয়ে গেলে খোঁজা ৩৯

অধ্যায় ৭: লিনুর খাবার দেওয়া ৪৭

অধ্যায় ৮: ফাইল পড়া ও লেখা ৫৫

অধ্যায় ৯: হুকুমের চেইন — Pipe ও Redirection ৬৪

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১০: Permission — কে লিনুকে ছুঁতে পারবে ৭২

অধ্যায় ১১: লিনুর স্বাস্থ্য পরীক্ষা ৭৯

অধ্যায় ১২: Environment ও Shell Customization ৮৭

অধ্যায় ১৩: প্রথম Bash Script ৯৬

অধ্যায় ১৪: সময় ধরে কাজ — Cron ১০৭

অধ্যায় ১৫: লিনুর সাথে দূর থেকে কথা — SSH ১১৮

অধ্যায় ১৬: Network-এর খোঁজখবর ১২৪

অধ্যায় ১৭: 🏆 Final Project — লিনুকে চাকরিতে পাঠানো ১৩০

অধ্যায় ১৮: লিনুর নতুন নতুন কর্মস্থল — Cloud জুড়ে Deployment ১৪১

পরিশিষ্ট ক: লিনুর হুকুমনামা 📜 — জরুরি Command Cheat Sheet ১৫৩

পরিশিষ্ট খ: লিনু কথা না শুনলে — সাধারণ Error ও সমাধান ১৫৫

পরিশিষ্ট গ: এরপর কী — লিনুর সাথে সামনের রাস্তা ১৫৭

পরিশিষ্ট ঘ: Resources ও Community ১৫৯

Licensed to sakilorb@gmail.com · Do not redistribute

লেখক পরিচিতি

আমি ইফতে সামউল ওহী — পেশায় সফটওয়্যার ইঞ্জিনিয়ার।

Django, Python আর Linux নিয়ে কাজ করতে গিয়ে একটা জিনিস বারবার চোখে পড়েছে: আমাদের অনেকেই প্রোগ্রামিং শিখছে, কিন্তু Linux আর terminal দেখলেই ভয় পাচ্ছে। অথচ চাকরির বাজারে, server-এ, cloud-এ — সবখানে Linux। সেই ভয়টা ভাঙানোর জন্যই এই বই।

এর আগে লিখেছি **"Journey by Python"** — যেখানে ট্রেনে চড়ে আমরা Python শিখেছিলাম। এই বইয়ে ট্রেন নেই — আছে **লিনু**, একটা কাল্পনিক কালো বিড়াল, যে আসলে তোমার terminal-এরই প্রতীক। বিড়াল পোষা আর Linux শেখা অদ্ভুতভাবে একই জিনিস: দুটোই প্রথমে কথা শোনে না, কিন্তু সঠিকভাবে হুকুম দিলে পোষ মানে — আর একবার পোষ মানলে সারাজীবনের বন্ধু।

চলো, লিনুর সাথে পরিচয় করিয়ে দিই।

Licensed to sakilorb@gmail.com · Do not redistribute

এই বইটা কীভাবে পড়বে

এই বইটা গল্পের বই না, আবার শুধু কমান্ডের লিস্টও না — এটা একটা **হাতেকলমে করার বই**। তাই কিছু নিয়ম মেনে পড়লে সবচেয়ে বেশি লাভ পাবে:

১. পাশে টার্মিনাল খোলা রাখো। প্রতিটা কমান্ড নিজের হাতে টাইপ করবে। Copy-paste না — **টাইপ**। বিড়ালকে আদর করা যেমন ভিডিও দেখে শেখা যায় না, terminal-ও তেমনি না চালিয়ে শেখা যায় না।

২. ভুল করো, ভয় পেয়ো না। এই বইয়ের কোনো কমান্ডে তোমার কম্পিউটার নষ্ট হবে না। যেখানে সাবধান হতে হবে, সেখানে আমি বড় করে ⚠️ চিহ্ন দিয়ে দেবো। বাকি সব জায়গায় — খেলো, ভাঙো, আবার চালাও।

৩. চিহ্নগুলো চিনে রাখো:

- 🐾 **লিনুর সাথে দৃশ্য** — গল্পের অংশ, যেখানে লিনুকে দিয়ে concept বোঝাবো
- 💡 **টিপস** — কাজে লাগার মতো ছোট কৌশল
- ⚠️ **সাবধান** — এখানে ভুল করলে ঝামেলা হতে পারে
- 🛠️ **হাতেকলমে** — নিজে করার প্র্যাকটিস বা মিনি প্রজেক্ট
- 📜 **হুকুমনামা** — অধ্যায় শেষে সেই অধ্যায়ের কমান্ডের সারসংক্ষেপ

৪. পর্ব সাজানো আছে ধাপে ধাপে। পর্ব ১ না পড়ে পর্ব ৪-এ লাফ দিলে লিনু রাগ করবে। প্রথমবার সিরিয়ালি পড়ো, পরে reference হিসেবে যেখানে খুশি যাও।

তাহলে শুরু করা যাক। দরজায় কে যেন মিঁয়াও করছে… 🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পর্ব ১: লিনুকে বাসায় আনা

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১: লিনুর সাথে পরিচয়

> 🐾 **দৃশ্য ১: এক বৃষ্টির সন্ধ্যা**

ধরো, এক বৃষ্টির সন্ধ্যায় তুমি বাসায় ফিরছো। গেটের সামনে দেখলে একটা কুচকুচে কালো বিড়ালছানা বসে আছে — ভিজে একাকার, কিন্তু চোখদুটো উজ্জ্বল সবুজ, আর সেই চোখে কোনো ভয় নেই। তুমি এগিয়ে গেলে ও পালালো না, বরং মাথা উঁচু করে এমনভাবে তাকালো যেন বলছে — "দেরি করলে কেন? কখন থেকে বসে আছি।"

তুমি ওকে কোলে তুলে বাসায় নিয়ে এলে। নাম দিলে — **লিনু**।

প্রথম রাতেই বুঝলে, এ সাধারণ বিড়াল না। তুমি "আয় আয়" বললে আসে না, কিন্তু ঠিক নিয়ম মেনে নির্দিষ্টভাবে ডাকলে ঠিকই আসে। ভুলভাল ডাকলে শুধু একবার তাকায় — যেন বলছে, "এই ভাষা আমি বুঝি না।"

এই বইজুড়ে আমরা লিনুকে পোষ মানাবো। আর মজার ব্যাপার হলো — **লিনুকে পোষ মানানো শিখতে শিখতে তুমি আসলে Linux শিখে ফেলবে।** কারণ লিনু আর Linux-এর স্বভাব হুবহু এক:

লিনু (বিড়াল)	Linux (অপারেটিং সিস্টেম)
ভুলভাল ডাকলে সাড়া দেয় না	ভুল কমান্ড দিলে কাজ করে না
সঠিকভাবে ডাকলে সঙ্গে সঙ্গে আসে	সঠিক কমান্ড দিলে নিমেষে কাজ করে
নিজের মর্জির মালিক, কিন্তু বিশ্বস্ত	তোমাকে পূর্ণ নিয়ন্ত্রণ দেয়, কিন্তু দায়িত্বও তোমার
একবার পোষ মানলে সারাজীবনের বন্ধু	একবার শিখলে ক্যারিয়ারজুড়ে কাজে লাগে

Licensed to sakilorb@gmail.com · Do not redistribute

Linux আসলে কী?

সহজ কথায়: **Linux একটা অপারেটিং সিস্টেম (OS)** — যেমন Windows বা macOS। অপারেটিং সিস্টেম হলো সেই সফটওয়্যার যেটা তোমার কম্পিউটারের হার্ডওয়্যার (প্রসেসর, র‍্যাম, হার্ডডিস্ক) আর তোমার প্রোগ্রামগুলোর মাঝখানে দাঁড়িয়ে সব সামলায়।

একটু গভীরে গেলে: Linux আসলে একটা **kernel** — অপারেটিং সিস্টেমের হৃদপিণ্ড। ১৯৯১ সালে ফিনল্যান্ডের এক ছাত্র, **লিনুস টরভাল্ডস (Linus Torvalds)**, শখের বশে এটা বানানো শুরু করেন এবং পুরো কোড ইন্টারনেটে সবার জন্য খুলে দেন। সেই "শখের প্রজেক্ট" আজ পৃথিবীর সবচেয়ে বেশি ব্যবহৃত অপারেটিং সিস্টেম কার্নেল।

> 💡 খেয়াল করেছো? **Linus** ▯ **Linux** ▯ **লিনু**। আমাদের বিড়ালের নামের রহস্য এখানেই। আর Linux-এর অফিসিয়াল মাসকট একটা পেঙ্গুইন, নাম **Tux** — সেজন্যই লিনুর গলার ট্যাগে একটা ছোট্ট পেঙ্গুইন ঝোলে।

লিনুর জাত ভাইরা সবখানে আছে

তুমি হয়তো ভাবছো, "Linux তো কখনো ব্যবহার করিনি।" ভুল ভাবছো — তুমি **প্রতিদিন** Linux ব্যবহার করো, না জেনেই:

- **তোমার Android ফোন** — Android-এর ভেতরের কার্নেলটা Linux
- **Facebook, Google, YouTube** — যে সার্ভারগুলোতে এসব চলে, তার প্রায় সবই Linux
- **ইন্টারনেটের ৯০%-এর বেশি সার্ভার** — Linux-এ চলে
- **Cloud (AWS, Google Cloud, Azure)** — মেরুদণ্ড Linux
- **বাংলাদেশের ব্যাংক, টেলিকম, ISP-র সার্ভার** — বেশিরভাগই Linux

Licensed to sakilorb@gmail.com · Do not redistribute

- এমনকি **পৃথিবীর সবচেয়ে শক্তিশালী ৫০০টা সুপারকম্পিউটারের সবগুলো** — Linux

অর্থাৎ লিনু একা না — ওর ভাই-বেরাদাররা পুরো ইন্টারনেট চালাচ্ছে। তুমি যখন এই বইয়ের PDF ডাউনলোড করেছো, সেটাও সম্ভবত একটা Linux সার্ভার থেকেই এসেছে। 😄

তাহলে তুমি কেন শিখবে?

সোজাসাপ্টা তিনটা কারণ:

১. ক্যারিয়ার। তুমি যদি Backend Developer, DevOps Engineer, Cloud Engineer, Data Engineer, Cybersecurity — এই লাইনের যেকোনো একটাতে যেতে চাও, Linux ছাড়া গতি নেই। জব ইন্টারভিউতে "SSH করে সার্ভারে ঢুকে log দেখতে পারবেন?" — এই প্রশ্নের উত্তর "না" হলে অনেক দরজা বন্ধ।

২. ক্ষমতা। Terminal শিখলে যে কাজ মাউস দিয়ে করতে ১০ মিনিট লাগে, সেটা এক লাইনে ২ সেকেন্ডে হয়। ১০০টা ফাইলের নাম বদলানো, হাজার লাইনের log থেকে দরকারি লাইন খুঁজে বের করা — এসব terminal-এ পানির মতো সহজ।

৩. স্বাধীনতা। Linux ফ্রি। লাইসেন্সের ঝামেলা নেই, চুরি করা Windows-এর অপরাধবোধ নেই। পুরনো, স্লো ল্যাপটপেও Linux দিব্যি চলে — অনেক ছাত্রের জন্য এটাই সবচেয়ে বড় সুবিধা।

Distro মানে কী — লিনুর বিভিন্ন breed

> **🐾 দৃশ্য ২: বিড়ালের হাট**

Licensed to sakilorb@gmail.com · Do not redistribute

ধরো তুমি বিড়াল নিতে গেছো। দোকানি বললো — "কোনটা নেবেন? পার্সিয়ান আছে, দেশি আছে, সিয়ামিজ আছে..." সবগুলোই বিড়াল — চার পা, লেজ, মিঁয়াও সব এক — কিন্তু চেহারা, লোমের ধরন আর মেজাজে আলাদা।

Linux-এর জগতেও ঠিক তাই। **Kernel** (হৃদপিণ্ড) এক, কিন্তু তার ওপরে বিভিন্ন দল বিভিন্নভাবে সাজিয়ে বানিয়েছে আলাদা আলাদা প্যাকেজ — এগুলোকে বলে **Distribution**, সংক্ষেপে **Distro**। প্রত্যেক distro-তে kernel-এর সাথে থাকে আলাদা সফটওয়্যার সেট, আলাদা চেহারা (desktop environment), আর আলাদা package manager (এটা কী, পরে বিস্তারিত জানবে — আপাতত ভাবো "খাবারের দোকান")।

জনপ্রিয় কয়েকটা breed:

Ubuntu (উবুন্টু) — সবচেয়ে জনপ্রিয়, সবচেয়ে beginner-friendly। কোনো সমস্যায় পড়লে Google করলেই সমাধান পাবে, কারণ ব্যবহারকারী সবচেয়ে বেশি। সার্ভার জগতেও Ubuntu-র রাজত্ব। **এই বইয়ে আমরা Ubuntu ব্যবহার করবো।**

Debian (ডেবিয়ান) — Ubuntu-র বাবা। ভীষণ stable, একটু রক্ষণশীল। সার্ভারে খুব জনপ্রিয়।

Fedora (ফেডোরা) — সবসময় নতুন প্রযুক্তি আগে পায়। Red Hat কোম্পানির community সংস্করণ; কর্পোরেট জগতে Red Hat-এর অনেক দাম।

Arch (আর্চ) — জেদি, বুনো বিড়াল। সব নিজের হাতে সেট করতে হয়। Arch পোষ মানাতে পারলে কমিউনিটিতে আলাদা সম্মান আছে — কিন্তু প্রথম বিড়াল হিসেবে একদম না। 😅

Licensed to sakilorb@gmail.com · Do not redistribute

> 💡 কেউ যদি জিজ্ঞেস করে "ভাই কোন distro দিয়ে শুরু করবো?" — উত্তর একটাই: **Ubuntu LTS**। LTS মানে Long Term Support — এই সংস্করণগুলো ৫ বছর ধরে আপডেট পায়, তাই stable আর নির্ভরযোগ্য।

এই অধ্যায়ে যা শিখলে

- Linux একটা অপারেটিং সিস্টেম, যার হৃদপিণ্ড হলো kernel
- লিনুস টরভাল্ডস ১৯৯১ সালে এটা বানিয়ে ফ্রিতে বিলিয়ে দেন
- Android থেকে সুপারকম্পিউটার — Linux সবখানে
- Distro = একই kernel-এর ওপর বানানো বিভিন্ন সংস্করণ (breed)
- আমরা শুরু করবো Ubuntu দিয়ে

পরের অধ্যায়ে লিনুকে সত্যি সত্যি বাসায় আনবো — মানে তোমার কম্পিউটারে Linux চালু করবো। মিঁয়াও। 🐾

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ২: লিনুকে বাসায় আনা

> 🐾 **দৃশ্য ৩: বিড়াল আনার আগের প্রস্তুতি**

বিড়াল আনার আগে ভাবতে হয় — ও থাকবে কোথায়? কেউ বিড়ালকে পুরো বাড়ি ছেড়ে দেয়, কেউ একটা ঘর ঠিক করে দেয়, কেউ শুরুতে খাঁচায় রেখে অভ্যাস করায়।

Linux-ও তোমার কম্পিউটারে আনার তিনটা রাস্তা আছে — আর কোনটা তোমার জন্য, সেটা নির্ভর করে তোমার পরিস্থিতির ওপর।

রাস্তা ১: WSL2 — খাঁচায় রেখে পোষা (Windows ব্যবহারকারীদের জন্য সেরা)

তুমি যদি Windows 10 বা 11 চালাও, তাহলে সবচেয়ে সহজ রাস্তা হলো **WSL2 (Windows Subsystem for Linux)**। এটা Microsoft-এর নিজের বানানো ব্যবস্থা, যেখানে Windows-এর ভেতরেই একটা আস্ত Linux চলে — আলাদা কম্পিউটার লাগে না, Windows মুছতেও হয় না।

"খাঁচা" শব্দটা শুনে খারাপ ভেবো না — এই খাঁচা রাজপ্রাসাদের মতো। লিনু ওখানে দিব্যি থাকে, আর তুমি Windows-এ তোমার সব কাজও চালিয়ে যেতে পারো। বিশ্বের লাখো professional developer ঠিক এভাবেই কাজ করে।

ইনস্টল করা মাত্র দুই ধাপ:

প্রথমে Windows-এ **PowerShell** খোলো **Administrator হিসেবে** (Start মেনুতে PowerShell লিখে সার্চ করো □ ডান ক্লিক □ Run as administrator)। তারপর লেখো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
wsl --install
```

ব্যস। এই এক কমান্ডে Windows নিজেই WSL2 চালু করবে এবং Ubuntu নামিয়ে ইনস্টল করে দেবে। শেষ হলে কম্পিউটার restart দাও।

Restart-এর পর Start মেনুতে **Ubuntu** নামে একটা অ্যাপ পাবে। খুললে প্রথমবার তোমাকে দুটো জিনিস ঠিক করতে বলবে:

- **Username** — ছোট হাতের অক্ষরে, স্পেস ছাড়া (যেমন: `rahim`)
- **Password** — টাইপ করার সময় স্ক্রিনে **কিছুই দেখাবে না** — কোনো তারকা চিহ্নও না। এটা কোনো সমস্যা না, Linux-এ এটাই নিয়ম (নিরাপত্তার জন্য)। সাহস করে টাইপ করে Enter চাপো।

> ⚠️ এই password টা মনে রাখো — লিনুকে গুরুত্বপূর্ণ হুকুম দেওয়ার সময় (পরে শিখবে, `sudo`) বারবার লাগবে।

ব্যস — লিনু এখন তোমার বাসায়! কালো স্ক্রিনে সবুজ-সাদা লেখা দেখে ভয় পেয়ো না, ওটাই লিনুর মুখ। 🐱

রাস্তা ২: VirtualBox — আলাদা ঘরে রাখা

তোমার যদি Windows-এর পুরনো সংস্করণ থাকে, বা তুমি চাও পুরো Ubuntu Desktop (গ্রাফিক্যাল চেহারাসহ) দেখতে — তাহলে **VirtualBox** ব্যবহার করতে পারো। এটা একটা ফ্রি সফটওয়্যার যেটা তোমার কম্পিউটারের ভেতরে আরেকটা "ভার্চুয়াল কম্পিউটার" বানায়, আর সেখানে Ubuntu চলে।

Licensed to sakilorb@gmail.com · Do not redistribute

সংক্ষেপে ধাপগুলো:

1. **virtualbox.org** থেকে VirtualBox নামিয়ে ইনস্টল করো
2. **ubuntu.com/download/desktop** থেকে Ubuntu LTS-এর ISO ফাইল নামাও (প্রায় ৫-৬ GB)
3. VirtualBox খুলে **New** চাপো → নাম দাও → ISO ফাইলটা দেখিয়ে দাও
4. RAM দাও অন্তত **৪ GB**, ডিস্ক **২৫ GB**
5. Start চাপো, স্ক্রিনের নির্দেশনা মেনে Ubuntu ইনস্টল করো

> 💡 VirtualBox একটু ভারী — তোমার কম্পিউটারে অন্তত ৮ GB RAM থাকলে ভালো চলবে। কম RAM হলে WSL2-ই ভালো, ওটা অনেক হালকা।

রাস্তা ৩: Dual Boot — পুরো বাড়িই লিনুর

এখানে তুমি কম্পিউটারে Windows-এর **পাশাপাশি** সরাসরি Ubuntu ইনস্টল করো — কম্পিউটার চালু করার সময় জিজ্ঞেস করবে, "আজ কোনটা চালাবে?" এতে Linux পূর্ণ গতিতে চলে, কারণ মাঝখানে কোনো খাঁচা বা ভার্চুয়াল স্তর নেই।

কিন্তু ⚠️ **সাবধান:** dual boot-এ পার্টিশন নিয়ে কাজ করতে হয়। ভুল করলে Windows-এর ডেটা মুছে যেতে পারে। তাই এই রাস্তায় যাওয়ার আগে অবশ্যই গুরুত্বপূর্ণ ফাইল backup করবে। একদম নতুনদের আমি এটা প্রথমে করতে বলি না।

আর যদি তোমার একটা পুরনো, অব্যবহৃত ল্যাপটপ পড়ে থাকে — তাহলে সবচেয়ে মজার অপশন: ওটাতে সরাসরি শুধু Ubuntu ইনস্টল করে দাও। পুরনো ল্যাপটপ নতুন জীবন পাবে, আর তুমি পাবে খেলার মাঠ।

Licensed to sakilorb@gmail.com · Do not redistribute

কোনটা বেছে নেবে?

তোমার পরিস্থিতি	সেরা রাস্তা
Windows 10/11 আছে, ঝামেলা চাই না	**WSL2** ✅
Ubuntu-র পুরো desktop চেহারা দেখতে চাই	VirtualBox
পুরনো/ফালতু ল্যাপটপ পড়ে আছে	সরাসরি Ubuntu ইনস্টল
আত্মবিশ্বাসী, backup নেওয়া আছে	Dual Boot
Mac ব্যবহার করি	macOS-এর Terminal-ই চলবে*

* 💡 Mac ব্যবহারকারীদের জন্য সুখবর: macOS-এর ভেতরটা Linux-এর জ্ঞাতি ভাই (দুটোই Unix পরিবারের)। এই বইয়ের ৮০% কমান্ড Mac-এর Terminal-এ হুবহু চলবে। তবে server-এর অভিজ্ঞতার জন্য পরে একটা Ubuntu VM বানিয়ে নিলে ভালো।

এই বইয়ের বাকি অংশে আমি ধরে নেবো তুমি WSL2 বা Ubuntu — যেকোনো একটাতে বসে আছো। কমান্ড সব জায়গায় একই।

প্রথমবার টার্মিনাল খোলা — লিনুর চোখে চোখ রাখা

> **🐾 দৃশ্য ৪: প্রথম সাক্ষাৎ**

লিনু বাসায় এসেছে। ও এখন ঘরের কোণায় বসে আছে, আর তোমার দিকে একদৃষ্টে তাকিয়ে আছে। কালো পর্দার ওপর একটা ছোট্ট দাগ মিটমিট করছে — যেন লিনুর লেজ নড়ছে, অপেক্ষায়।

টার্মিনাল খোলো (WSL2 হলে Start মেনু থেকে Ubuntu, Ubuntu Desktop হলে `Ctrl+Alt+T` চাপো)। দেখবে এরকম একটা লাইন:

Licensed to sakilorb@gmail.com · Do not redistribute

```
rahim@laptop:~$
```

আর তার পাশে একটা মিটমিট করা cursor। **এটাই লিনুর চোখ।** ও তাকিয়ে আছে, অপেক্ষা করছে — তুমি কী হুকুম দাও।

অনেকে এই কালো স্ক্রিন দেখেই ঘাবড়ে যায়। মনে রেখো: **এই স্ক্রিন তোমার শত্রু না, এটা লিনুর মুখ।** ও কামড়াবে না। সবচেয়ে খারাপ যেটা হবে — তুমি ভুল কিছু লিখলে ও ভদ্রভাবে বলবে, `command not found`। ব্যস, এইটুকুই।

চলো প্রথম কথাটা বলি। টাইপ করো (হাতে টাইপ করো!):

```
echo "hello linu"
```

Enter চাপো। লিনু জবাব দেবে:

```
hello linu
```

🎉 **অভিনন্দন! তুমি এইমাত্র তোমার প্রথম Linux কমান্ড চালালে।** লিনু তোমার কথা শুনেছে এবং হুবহু ফেরত বলেছে। `echo` কমান্ডের কাজই এটা — যা বলবে, তা-ই প্রতিধ্বনি করা।

সামান্য মনে হচ্ছে? মোটেই না। এই মুহূর্ত থেকে তোমার আর লিনুর মধ্যে যোগাযোগ শুরু হলো। বাকি বইটা শুধু এই কথোপকথনটাকে সমৃদ্ধ করা।

🛠️ হাতেকলমে

1. তোমার পরিস্থিতি অনুযায়ী একটা রাস্তা বেছে Linux চালু করো (আটকে গেলে ঘাবড়িয়ো না — error message-টা হুবহু Google-এ সার্চ দাও, এটাই developer-দের আসল skill 😄)

Licensed to sakilorb@gmail.com · Do not redistribute

2. টার্মিনাল খুলে `echo "hello linu"` চালাও

3. এবার `echo` দিয়ে নিজের নাম বলাও: `echo "amar nam Rahim"`

4. দেখো তো এটা চালালে কী হয়: `echo $USER` — লিনু কি তোমাকে চিনে ফেলেছে?

📜 হুকুমনামা

কমান্ড	কাজ
`wsl --install`	Windows-এ WSL2 + Ubuntu ইনস্টল (PowerShell, Admin)
`echo "কিছু লেখা"`	লেখাটা স্ক্রিনে ফেরত দেখায়

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৩: লিনুর ভাষা বোঝা

> 🐾 **দৃশ্য ৫: মিঁয়াও-এর ব্যাকরণ**

কয়েকদিন লিনুর সাথে থাকার পর তুমি একটা জিনিস খেয়াল করলে — লিনুর "মিঁয়াও"-এর রকমফের আছে। খাবার চাইলে এক রকম, দরজা খুলতে বললে আরেক রকম, বিরক্ত হলে অন্য রকম। মানে ওর একটা **ভাষা** আছে, আর সেই ভাষার **ব্যাকরণ** আছে।

তুমি যদি সেই ব্যাকরণটা ধরে ফেলতে পারো, লিনুর সাথে কথা বলা পানির মতো সহজ হয়ে যাবে। এই অধ্যায়ে আমরা সেটাই করবো।

Terminal, Shell, Bash — তিনটা শব্দের জট খোলা

এই তিনটা শব্দ সবাই গুলিয়ে ফেলে। একবারে পরিষ্কার করে দিই — বিড়াল দিয়েই:

Terminal হলো **ঘরটা** — যে ঘরে বসে তুমি লিনুর সাথে কথা বলো। ওই কালো উইন্ডোটা, যেটায় তুমি টাইপ করো। এটা শুধু একটা জানালা — নিজে কিছু বোঝে না।

Shell হলো **দোভাষী** — যে তোমার কথা শুনে লিনুর ভাষায় (মানে অপারেটিং সিস্টেমের ভাষায়) অনুবাদ করে দেয়, আবার লিনুর জবাব তোমাকে বুঝিয়ে বলে। তুমি `ls` লিখলে shell-ই বুঝে নেয় তুমি কী চাইছো এবং সেটা করিয়ে আনে।

Bash হলো সেই দোভাষীর **নাম**। দোভাষী তো অনেকেই আছে — bash, zsh, fish — কিন্তু Linux জগতে সবচেয়ে প্রচলিত দোভাষীর নাম Bash (মজার ব্যাপার: এর পুরো নাম **B**ourne **A**gain **SH**ell — ইংরেজি শ্লেষ, "born again")।

Licensed to sakilorb@gmail.com · Do not redistribute

সংক্ষেপে: **তুমি Terminal-এ (ঘর) বসে Bash-এর (দোভাষী) মাধ্যমে লিনুর (সিস্টেম) সাথে কথা বলো।**

Prompt-এর অ্যানাটমি — লিনুর নেমপ্লেট

টার্মিনাল খুললে যে লাইনটা দেখো, সেটাকে বলে **prompt**:

```
rahim@laptop:~$
```

এটা আসলে একটা নেমপ্লেট + স্ট্যাটাস বোর্ড। ভেঙে দেখি:

- `rahim` — তুমি কে (username)। লিনু জানে কে ওকে হুকুম দিচ্ছে।
- `@laptop` — কোন কম্পিউটারে আছো (hostname)। এখন তুচ্ছ লাগছে, কিন্তু যেদিন তুমি একসাথে নিজের ল্যাপটপ আর দূরের একটা সার্ভারে কাজ করবে, সেদিন এই অংশটাই তোমাকে বাঁচাবে — "আমি এখন কোন মেশিনে আছি?"
- `~` — তুমি এখন কোন ফোল্ডারে দাঁড়িয়ে আছো। `~` (টিল্ডা) চিহ্নের মানে **home ফোল্ডার** — লিনুর নিজের ঘর। এটা নিয়ে পরের অধ্যায়ে বিস্তারিত।
- `$` — মানে তুমি সাধারণ ব্যবহারকারী। এটা যদি `#` হয়, তার মানে তুমি **root** — সর্বময় ক্ষমতার মালিক (সেটা কী জিনিস, পর্ব ৪-এ জানবে)।

> 💡 বইয়ে যখন আমি কমান্ড লিখবো, শুরুর `$` চিহ্নটা টাইপ করার দরকার নেই — ওটা শুধু বোঝায় "এটা prompt-এ লেখার কমান্ড।"

Licensed to sakilorb@gmail.com · Do not redistribute

কমান্ডের গঠন: হুকুম, ভঙ্গি, বস্তু

এবার আসল ব্যাকরণ। Linux-এর প্রায় প্রতিটা কমান্ডের গঠন এক:

```
কমান্ড   -অপশন  আর্গুমেন্ট
```

বিড়ালের ভাষায়: **হুকুম, ভঙ্গি, বস্তু।**

ধরো তুমি লিনুকে বলছো — "লিনু, নিয়ে আয় — তাড়াতাড়ি — বলটা।"

- **"নিয়ে আয়"** = কমান্ড (কী করতে হবে)
- **"তাড়াতাড়ি"** = অপশন (কীভাবে করতে হবে — কাজটা একই, ধরনটা বদলায়)
- **"বলটা"** = আর্গুমেন্ট (কীসের ওপর করতে হবে)

এবার সত্যিকারের উদাহরণ। `ls` কমান্ড ফোল্ডারের ভেতরের জিনিসপত্র দেখায় (list):

```
ls
```

শুধু হুকুম — লিনু বর্তমান ফোল্ডারের জিনিস দেখালো।

```
ls -l
```

হুকুম + ভঙ্গি। `-l` (long) অপশন বলছে — "বিস্তারিতভাবে দেখা": ফাইলের আকার, তারিখ, মালিক সব সহ।

```
ls -l /home
```

হুকুম + ভঙ্গি + বস্তু। এবার বলছি — `/home` ফোল্ডারের ভেতরটা, বিস্তারিতভাবে দেখাও।

Licensed to sakilorb@gmail.com · Do not redistribute

অপশন একসাথে জোড়াও যায়:

```
ls -la
```

`-l` (বিস্তারিত) + `-a` (all — লুকানো ফাইলসহ)। লিনু দুটো ভঙ্গি একসাথে বুঝে নেয়।

> ⚠️ **Linux ছোট-বড় হাতের অক্ষর আলাদা করে চেনে (case-sensitive)।** `ls` আর `LS` এক জিনিস না — দ্বিতীয়টা লিখলে লিনু বলবে `command not found`। আর কমান্ড, অপশন, আর্গুমেন্টের মাঝে **স্পেস** থাকা বাধ্যতামূলক। `ls-l` লিখলে লিনু চিনবে না — `ls -l` লিখতে হবে।

লিনু যখন বুঝতে পারে না

ভুল কমান্ড লিখলে কী হয়? চেষ্টা করে দেখো:

```
lss
```

লিনু জবাব দেবে:

```
lss: command not found
```

ব্যস — এইটুকুই। কোনো বিস্ফোরণ না, কম্পিউটার নষ্ট না। লিনু শুধু কান নাড়িয়ে বললো, "এই ডাক আমি চিনি না।" তুমি আবার ঠিক করে লিখবে, ও শুনবে।

এই জিনিসটা মাথায় গেঁথে নাও: **terminal-এ ভুল করা শেখারই অংশ।** যারা Linux ভালো পারে, তারা ভুল কম করে না — তারা ভুলের message পড়ে বুঝতে পারে, এই যা পার্থক্য।

Licensed to sakilorb@gmail.com · Do not redistribute

সাহায্য চাওয়া — লিনুর অভিধান

কোনো কমান্ডের ব্যবহার ভুলে গেলে লিনুর কাছেই জিজ্ঞেস করা যায়। দুটো উপায়:

```
ls --help
```

সংক্ষিপ্ত সাহায্য — কোন কোন অপশন আছে, এক পাতায়।

```
man ls
```

`man` মানে **manual** — পুরো অভিধান খুলে যাবে। ওপরে-নিচে যেতে তীর চিহ্ন (arrow key), আর **বের হতে `q` চাপো।**

> 💡 `man` থেকে বের হতে না পারা — নতুনদের সবচেয়ে কমন প্যানিক মুহূর্ত। 😄 মনে রাখো: **q মানে quit।** এই এক অক্ষর জানা থাকলে আর কখনো আটকাবে না।

🛠️ হাতেকলমে

1. `ls`, `ls -l`, `ls -a`, `ls -la` — চারটা চালিয়ে output-এর পার্থক্য নিজের চোখে দেখো
2. `date` কমান্ড চালাও — লিনু কি সময় জানে?
3. `whoami` চালাও — লিনুকে জিজ্ঞেস করো, "আমি কে?"
4. ইচ্ছা করে একটা ভুল কমান্ড লেখো (যেমন `meow`) — error message-টা পড়ো, ভয় ভাঙাও
5. `man date` খুলে দেখো, তারপর `q` চেপে বের হও — এটাই আজকের সবচেয়ে গুরুত্বপূর্ণ অনুশীলন 😄

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড	কাজ
`ls`	ফোল্ডারের জিনিসপত্র দেখায়
`ls -l`	বিস্তারিতসহ দেখায়
`ls -a`	লুকানো ফাইলসহ দেখায়
`date`	বর্তমান তারিখ-সময়
`whoami`	বর্তমান username
`কমান্ড --help`	সংক্ষিপ্ত সাহায্য
`man কমান্ড`	পূর্ণ ম্যানুয়াল (`q` দিয়ে বের হও)

🐾 **পর্ব ১ শেষ।** লিনু এখন তোমার বাসায়, তুমি ওর ভাষার ব্যাকরণ জানো, আর প্রথম কয়েকটা হুকুমও দিয়ে ফেলেছো। পরের পর্বে আমরা লিনুর পুরো বাড়িটা ঘুরে দেখবো — Linux-এর filesystem — কোন ঘরে কী থাকে, আর হারিয়ে না গিয়ে কীভাবে ঘোরাঘুরি করতে হয়।

লিনু লেজ নাড়ছে। ও তৈরি। তুমি? 🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পর্ব ২: লিনুর ঘরদোর চেনা (Filesystem)

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৪: লিনুর বাড়ি ঘোরা

> 🐾 **দৃশ্য ৬: নতুন বাসার ট্যুর**

লিনু তোমার বাসায় এসেছে ঠিকই, কিন্তু একটা মজার ব্যাপার খেয়াল করেছো? বিড়াল নতুন বাসায় এলে প্রথম কাজটা কী করে? **পুরো বাড়ি ঘুরে ঘুরে দেখে।** প্রতিটা ঘরে ঢোকে, প্রতিটা কোণা শোঁকে, খাটের নিচে ঢুকে যায়, আলমারির মাথায় উঠে বসে। যতক্ষণ না পুরো বাড়ির নকশা ওর মাথায় গেঁথে যায়, ততক্ষণ ওর শান্তি নেই।

আজ আমরা ঠিক সেই কাজটাই করবো — লিনুর সাথে ওর বাড়িটা ঘুরে দেখবো। এই বাড়ির নাম **filesystem**।

লিনুর বাড়ির নকশা — উল্টো গাছ

Windows-এ তুমি অভ্যস্ত `C:` ড্রাইভ, `D:` ড্রাইভ দেখতে। Linux-এ ওসব নেই। Linux-এর পুরো জগৎটা **একটাই গাছ** — আর সেই গাছটা উল্টো: শিকড় ওপরে, ডালপালা নিচে।

শিকড়টার নাম **root**, লেখা হয় একটা মাত্র চিহ্ন দিয়ে: `/`

সবকিছু — প্রতিটা ফাইল, প্রতিটা ফোল্ডার, এমনকি তোমার পেনড্রাইভও — এই `/` থেকে শুরু হওয়া গাছের কোনো না কোনো ডালে ঝুলে আছে:

Licensed to sakilorb@gmail.com · Do not redistribute

```
/
├── home          ← বাসিন্দাদের ঘর
│   └── rahim     ← তোমার নিজের ঘর
├── etc           ← বাড়ির নিয়মকানুনের খাতা
├── var           ← রোজনামচা আর জমা জিনিস
├── usr           ← সফটওয়্যারের গুদাম
├── tmp           ← আবর্জনার ঝুড়ি  (নিজেই খালি হয়)
├── bin           ← হুকুমের বাক্স
└── root          ← বাড়িওয়ালার নিজের ঘর
```

> 💡 Linux-এ "ফোল্ডার"-কে বলা হয় **directory**। দুটো একই জিনিস — বইয়ে আমি দুটোই ব্যবহার করবো, যাতে দুই শব্দেই অভ্যাস হয়।

কোন ঘরে কী থাকে

বিড়ালের মতো প্রতিটা ঘরে উঁকি দিয়ে আসি:

`/home` — **বাসিন্দাদের ঘর।** প্রতিটা ব্যবহারকারীর নিজের ঘর এখানে। তোমার username যদি `rahim` হয়, তোমার ঘর `/home/rahim`। তোমার নিজের ফাইল, ডাউনলোড, প্রজেক্ট — সব এখানে রাখবে। **এটাই সেই `~` (টিল্ডা)** — অধ্যায় ৩-এ prompt-এ যেটা দেখেছিলে। `~` হলো `/home/rahim`-এর ডাকনাম।

`/etc` — **নিয়মকানুনের খাতা।** পুরো সিস্টেমের configuration ফাইল এখানে থাকে। কোন সফটওয়্যার কীভাবে চলবে, নেটওয়ার্ক কীভাবে কাজ করবে — সব নিয়ম এই ঘরে লেখা। এখন শুধু চিনে রাখো; পরে যখন server চালাবে, এই ঘরে তোমার নিয়মিত যাতায়াত হবে।

Licensed to sakilorb@gmail.com · Do not redistribute

`/var` — **রোজনামচা।** var মানে variable — যা বাড়তে থাকে। সবচেয়ে গুরুত্বপূর্ণ বাসিন্দা: `/var/log` — সিস্টেমের **ডায়েরি**, যেখানে কে কখন কী করলো সব লেখা হয়। সার্ভারে সমস্যা হলে প্রথম প্রশ্নই হয়: "log-এ কী বলছে?"

`/usr` — **সফটওয়্যারের গুদাম।** তোমার ইনস্টল করা প্রোগ্রামগুলোর মূল অংশ এখানে থাকে।

`/bin` — **হুকুমের বাক্স।** `ls`, `echo`, `cat` — এই কমান্ডগুলো নিজেরাও তো প্রোগ্রাম, নাকি? সেই প্রোগ্রাম-ফাইলগুলো এখানে থাকে। মানে তুমি যখন `ls` লেখো, লিনু আসলে এই বাক্স থেকে `ls` নামের জিনিসটা বের করে চালায়।

`/tmp` — **আবর্জনার ঝুড়ি।** অস্থায়ী ফাইলের জায়গা। এখানে কিছু রাখলে ধরে নাও restart-এ মুছে যেতে পারে।

`/root` — **বাড়িওয়ালার ঘর।** সাবধান, এটা `/` (root directory) না — এটা **root user**-এর (সিস্টেমের সর্বময় কর্তার) ব্যক্তিগত ঘর। পর্ব ৪-এ এই কর্তার সাথে বিস্তারিত পরিচয় হবে।

> ⚠️ শুরুর দিকে তোমার কাজের ৯৯% হবে নিজের ঘরে — `/home`-এ। বাকি ঘরগুলো এখন শুধু **চিনে রাখো**, ঘাঁটাঘাঁটি পরে।

এখন আমি কোথায়? — `pwd`

> 🐾 **দৃশ্য ৭: হারিয়ে যাওয়া**

বিড়াল কখনো হারায় না, জানো কেন? কারণ ও সবসময় জানে ও **এখন কোথায় আছে**। বাড়ির যেকোনো জায়গা থেকে ও নিজের খাবারের বাটি পর্যন্ত পথ চেনে।

Licensed to sakilorb@gmail.com · Do not redistribute

Terminal-এ কাজ করার সময় এটাই এক নম্বর নিয়ম: **সবসময় জানতে হবে তুমি এখন কোন directory-তে দাঁড়িয়ে আছো।** কারণ তোমার সব কমান্ড সেই জায়গাটার ওপরেই কাজ করবে।

জানার কমান্ড:

```
pwd
```

Output:

```
/home/rahim
```

`pwd` মানে **p**rint **w**orking **d**irectory — "আমি এখন কোথায়?" যখনই দিশেহারা লাগবে, `pwd` চালাও। এটা তোমার GPS।

ঘরে ঘরে হাঁটা — `cd`

জায়গা বদলানোর কমান্ড `cd` (**c**hange **d**irectory):

```
cd /tmp
pwd
```

Output: `/tmp` — তুমি এখন আবর্জনার ঝুড়ির ঘরে। 😄 এবার নিজের ঘরে ফেরা যাক।

তিনভাবেই ফেরা যায়:

```
cd /home/rahim    # পুরো ঠিকানা দিয়ে
cd ~              # ডাকনাম দিয়ে
cd                # খালি cd মানেই "বাসায় চলো"
```


Licensed to sakilorb@gmail.com · Do not redistribute

> 💡 `cd` একা লিখলেই home-এ ফেরত — বিড়াল যেমন যেখানেই থাকুক, "খাবার!" শুনলে সোজা বাসায়। এটা মনে রাখলে কখনো হারাবে না।

আরো দুটো জরুরি চাল:

```
cd ..     # এক ধাপ ওপরে (parent directory-তে)
cd -      # আগের জায়গায় ফেরত (undo-র মতো)
```

`..` মানে "যে ঘরে আছি তার বাইরের ঘর"। আর `cd -` হলো জাদু — তুমি `/var/log`-এ ছিলে, `cd ~` দিয়ে বাসায় এলে, আবার `cd -` দিলে সোজা `/var/log`-এ ফেরত। দুই জায়গার মধ্যে লাফালাফির জন্য দারুণ।

ঠিকানা বলার দুই কায়দা — Absolute vs Relative Path

> 🐾 **দৃশ্য ৮: ঠিকানা জিজ্ঞেস**

কেউ তোমার বাসার ঠিকানা জিজ্ঞেস করলে দুইভাবে বলতে পারো:

কায়দা ১: "বাংলাদেশ, ঢাকা, মিরপুর, রোড ১২, বাসা ৭" — পৃথিবীর যেকোনো জায়গা থেকে এই ঠিকানায় আসা যাবে। এটা **absolute path** — সবসময় `/` (শিকড়) থেকে শুরু।

কায়দা ২: "এই তো, সামনের গলিতে ঢুকে দুই বাসা পরেই" — এটা শুধু তখনই কাজ করে যখন শোনার মানুষটা কাছাকাছি দাঁড়িয়ে। এটা **relative path** — তুমি **এখন যেখানে আছো** সেখান থেকে হিসাব।

চেনার উপায় খুবই সহজ: **`/` দিয়ে শুরু হলে absolute, না হলে relative!**

Licensed to sakilorb@gmail.com · Do not redistribute

ধরো তুমি এখন `/home/rahim`-এ, আর সেখানে `projects` নামে একটা ফোল্ডার আছে:

```
cd /home/rahim/projects   # absolute — যেকোনো জায়গা থেকে কাজ করবে
cd projects               # relative — শুধু /home/rahim-এ দাঁড়িয়ে থাকলে কাজ করবে
```

দুটোই একই জায়গায় নেয় — কিন্তু দ্বিতীয়টা লিখতে কম লাগে, তাই দৈনন্দিন কাজে relative-ই বেশি চলে।

> 💡 **Tab completion — লিনুর সবচেয়ে বড় উপহার।** পথের নাম পুরোটা টাইপ করতে হয় না। `cd pro` লিখে **Tab** চাপো — লিনু নিজেই `projects` বানিয়ে দেবে। একাধিক মিল থাকলে দুইবার Tab চাপো, সব অপশন দেখাবে। **আজ থেকে Tab তোমার সবচেয়ে ঘনিষ্ঠ বন্ধু** — টাইপিং অর্ধেক কমে যাবে, বানান ভুলও।

🛠️ হাতেকলমে

1. `pwd` দিয়ে দেখো তুমি কোথায়
2. `cd /` দিয়ে শিকড়ে যাও, `ls` চালিয়ে গাছের প্রধান ডালগুলো নিজের চোখে দেখো
3. `cd /var/log` → `pwd` → `ls` — সিস্টেমের ডায়েরির ঘরটা দেখে এসো (শুধু দেখবে, কিছু ছোঁবে না 😄)
4. এক লাফে বাসায় ফেরো: `cd`
5. `cd -` চাপো — কোথায় গেলে? আবার `cd -` — এবার?
6. `cd /ho` লিখে Tab চাপো — জাদুটা দেখো

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড	কাজ
`pwd`	আমি এখন কোথায়
`cd ঠিকানা`	ওই directory-তে যাও
`cd` বা `cd ~`	home-এ ফেরো
`cd ..`	এক ধাপ ওপরে
`cd -`	আগের জায়গায় ফেরত
`Tab`	নাম auto-complete (কমান্ড না, কিন্তু সমান জরুরি!)

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৫: লিনুর জিনিসপত্র গোছানো

> 🐾 **দৃশ্য ৯: খেলনার ঝুড়ি**

লিনুর জিনিস দিন দিন বাড়ছে — বল, দড়ি, ইঁদুর-খেলনা, আঁচড়ানোর বোর্ড। সব ছড়িয়ে-ছিটিয়ে থাকলে চলবে না। আজ আমরা শিখবো জিনিস **বানানো, সাজানো, সরানো আর ফেলা** — অর্থাৎ ফাইল ম্যানেজমেন্ট। মাউস ছাড়া, শুধু হুকুম দিয়ে।

প্রথমে একটা খেলার মাঠ বানিয়ে নিই, যাতে ভুল হলেও কোনো ক্ষতি নেই:

```
cd ~
mkdir khelaghor
cd khelaghor
```

ঘর বানানো — `mkdir`

`mkdir` (**m**a**k**e **dir**ectory) নতুন directory বানায়:

```
mkdir khelna
mkdir khabar ghum     # একসাথে দুটোও বানানো যায়
ls
```

Output:

```
ghum  khabar  khelna
```

ভেতরে ভেতরে ঘর বানাতে চাইলে (যেমন `khelna`-র ভেতরে `bol`-এর ভেতরে `lal`):

Licensed to sakilorb@gmail.com · Do not redistribute

```
mkdir -p khelna/bol/lal
```

`-p` (parents) অপশন বলছে — "পথে যা যা ঘর দরকার, সব বানিয়ে নাও।" এটা ছাড়া `mkdir khelna/bol/lal` তখনই কাজ করবে যখন `khelna/bol` আগে থেকেই আছে।

খালি ফাইল বানানো — `touch`

```
touch note.txt
ls
```

`touch` একটা খালি ফাইল বানায়। বিড়াল যেমন থাবা দিয়ে ছুঁয়ে জিনিসের অস্তিত্ব ঘোষণা করে — `touch` ঠিক তাই। (ফাইলে লেখালেখি কীভাবে করবে, সেটা অধ্যায় ৮-এ।)

কপি করা — `cp`

`cp` (**c**o**p**y)-র গঠন সবসময়: `cp উৎস গন্তব্য` — **কোনটা, কোথায়।**

```
cp note.txt note-backup.txt      # একই জায়গায় আরেক নামে কপি
cp note.txt khelna/              # khelna ফোল্ডারের ভেতরে কপি
```

কিন্তু পুরো directory কপি করতে গেলে:

```
cp khelna khelna2
```

লিনু আপত্তি করবে: `cp: -r not specified; omitting directory 'khelna'`।

কারণ directory-র ভেতরে আরো জিনিস থাকতে পারে — লিনু নিশ্চিত হতে চায় তুমি **সব ভেতরসুদ্ধ** কপি করতে চাও কিনা। সেটা বোঝাতে `-r` (**r**ecursive — "ভেতরের সবসহ"):

Licensed to sakilorb@gmail.com · Do not redistribute

```
cp -r khelna khelna2
```

এই `-r` জিনিসটা মনে রেখো — directory নিয়ে কারবার করা অনেক কমান্ডেই এটা লাগবে।

সরানো আর নাম বদলানো — `mv`

`mv` (**m**o**v**e) এক ঢিলে দুই পাখি মারে। **সরানো:**

```
mv note-backup.txt ghum/
```

আর **নাম বদলানো:**

```
mv note.txt diary.txt
```

দাঁড়াও — নাম বদলানো আবার "move" হলো কীভাবে? লিনুর চোখে দুটো একই জিনিস: `note.txt`-কে "সরিয়ে" `diary.txt` নামের জায়গায় নেওয়া হলো। জায়গা একই, নাম বদলে গেলো। দর্শনটা মজার, আর কাজে খুবই লাগে।

ফেলা — `rm`, এবং লিনুর ভয়ঙ্কর থাবা

`rm` (re**m**ove) ফাইল মুছে দেয়:

```
rm diary.txt
```

Directory মুছতে সেই চেনা `-r`:

```
rm -r khelna2
```


Licensed to sakilorb@gmail.com · Do not redistribute

এবার সবচেয়ে গুরুত্বপূর্ণ সতর্কতা।

> ⚠️ `rm` **কোনো Recycle Bin-এ ফেলে না। মুছলো মানে চিরতরে গেলো।** Windows-এ ভুল করে delete করলে Recycle Bin থেকে ফেরত আনা যায়। Linux-এর `rm`-এ সেই দ্বিতীয় সুযোগ **নেই**। Enter চাপার আগে এক সেকেন্ড থেমে ভাবো — সঠিক ফাইল তো?

আর এবার সেই কুখ্যাত কমান্ড, যেটা নিয়ে ইন্টারনেটে হাজারটা meme আছে:

```
rm -rf ফোল্ডার
```

`-r` = ভেতরের সবসহ, `-f` (**f**orce) = "কোনো প্রশ্ন করবে না, চুপচাপ মুছবে।" দুটো মিলে: **নিঃশব্দে, প্রশ্নহীনভাবে, সব শেষ।**

🐾 এটা হলো লিনুর রেগে যাওয়া রূপ। শান্ত বিড়ালও রেগে গেলে এক থাবায় ফুলদানি শেষ করে দিতে পারে — `rm -rf` সেই থাবা। দরকারি কমান্ড, প্রতিদিন ব্যবহারও হয় — কিন্তু চালানোর আগে **দুইবার পড়ো কী লিখেছো।** বিশেষ করে path-এ typo আছে কিনা। ইন্টারনেটে যত "ভাই আমার সব ফাইল উধাও" কান্নার পোস্ট আছে, তার বেশিরভাগের পেছনে এই কমান্ড।

লুকানো ফাইল — লিনুর গোপন খেলনা

> 🐾 **দৃশ্য ১০: সোফার নিচের রহস্য**

একদিন সোফা সরাতে গিয়ে দেখলে — নিচে লিনুর গোপন ভাণ্ডার! হারানো বল, শুকনো মাছের টুকরা, তোমার সেই কবে হারিয়ে যাওয়া মোজা। বিড়াল মাত্রই কিছু জিনিস লুকিয়ে রাখে।

Licensed to sakilorb@gmail.com · Do not redistribute

Linux-ও রাখো। যে ফাইলের নাম `.` **(ডট) দিয়ে শুরু**, সেটা লুকানো (hidden) — সাধারণ `ls` -এ দেখা যায় না:

```
cd ~
ls          # সাধারণ জিনিসপত্র
ls -a       # -a (all) — সোফা সরিয়ে দেখা 😄
```

`ls -a` চালালে দেখবে `.bashrc` , `.profile` — এরকম ডট-ওয়ালা ফাইল। এগুলো লুকানো কেন? **তোমার থেকে লুকানোর জন্য না** — এগুলো configuration ফাইল, রোজ দরকার হয় না, তাই চোখের সামনে ভিড় না করে সরে থাকে। (`.bashrc` -র সাথে অধ্যায় ১২-তে ভালো করে পরিচয় হবে — ওটা দিয়ে লিনুর অভ্যাস বদলানো যায়।)

Wildcards — একসাথে অনেক জিনিস ধরা

ধরো তোমার কাছে ১০০টা ছবি (`.jpg`) আর ৫০টা নোট (`.txt`) একই ফোল্ডারে। শুধু ছবিগুলো `chobi` ফোল্ডারে সরাতে চাও। একটা একটা করে `mv` ? জীবনেও শেষ হবে না।

এখানেই wildcard — লিনুর জাল:

`*` **(তারা)** — "যেকোনো কিছু, যত অক্ষরই হোক":

```
mv *.jpg chobi/        # .jpg দিয়ে শেষ হওয়া সব ফাইল
ls note*               # note দিয়ে শুরু হওয়া সব ফাইল
```

`?` **(প্রশ্নবোধক)** — "ঠিক একটা অক্ষর, যেকোনোটা":

```
ls note?.txt           # note1.txt, note2.txt মিলবে — note10.txt
মিলবে না
```


Licensed to sakilorb@gmail.com · Do not redistribute

> ⚠️ Wildcard আর `rm` একসাথে ব্যবহারের আগে একটা পাকা অভ্যাস বানাও: **আগে `ls` দিয়ে দেখো জালে কী কী ধরা পড়ছে।** যেমন `rm *.txt` চালানোর আগে `ls *.txt` চালাও — যা যা দেখাবে, ঠিক সেগুলোই মুছবে। কোনো চমক নেই, কোনো কান্না নেই।

🛠️ হাতেকলমে: লিনুর সংসার গোছাও

একটা এলোমেলো সংসার বানিয়ে সেটা গোছাও:

1. `~`-তে `songsar` নামে ফোল্ডার বানাও, ভেতরে ঢোকো
2. এক কমান্ডে বানাও:

   ```
   touch bol1.txt bol2.txt bol3.txt mach1.jpg mach2.jpg dori.txt
   ```

3. `khelna` আর `chobi` নামে দুটো ফোল্ডার বানাও
4. এক কমান্ডে সব `.jpg` পাঠাও `chobi`-তে (আগে `ls` দিয়ে check!)
5. `bol` দিয়ে শুরু সব ফাইল পাঠাও `khelna`-তে
6. `dori.txt`-র নাম বদলে করো `rossi.txt`
7. সব শেষে পুরো `songsar` ফোল্ডারটাই মুছে ফেলো — কোন কমান্ড লাগবে ভেবে বের করো (উত্তর এই অধ্যায়েই আছে 😉)

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড	কাজ
`mkdir নাম`	directory বানাও (`-p` = পথের সবসহ)
`touch নাম`	খালি ফাইল বানাও
`cp উৎস গন্তব্য`	কপি (directory হলে `-r`)
`mv উৎস গন্তব্য`	সরাও / নাম বদলাও
`rm ফাইল`	মুছে ফেলো — ⚠️ ফেরত আসে না
`rm -rf`	⚠️⚠️ প্রশ্নহীন গণমোছা — দুইবার পড়ে তারপর Enter
`ls -a`	লুকানো ফাইলসহ দেখো
`*` / `?`	wildcard — অনেক/এক অক্ষরের জাল

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৬: লিনু হারিয়ে গেলে খোঁজা

> 🐾 **দৃশ্য ১১: লিনু কোথায়?**

সন্ধ্যাবেলা। লিনুর খাবারের সময় হয়ে গেছে, কিন্তু লিনুর দেখা নেই। ডাকলে সাড়া নেই। এখন?

তুমি জানো ও বাসাতেই আছে — খুঁজতে হবে পদ্ধতি মেনে: ঘরে ঘরে যাও, খাটের নিচে দেখো, আলমারির মাথায় দেখো, পর্দার পেছনে দেখো।

কম্পিউটারেও একই সমস্যা রোজ হয়: "ফাইলটা যেন কোথায় রাখলাম?" — আর তোমার ফাইল যত বাড়বে, এই সমস্যাও তত বাড়বে। আজ শিখবো Linux-এর খোঁজাখুঁজির অস্ত্রগুলো — আর এগুলো এতই শক্তিশালী যে একবার আয়ত্তে এলে মাউস দিয়ে ফোল্ডারে ফোল্ডারে ক্লিক করে খোঁজাকে পাথরযুগ মনে হবে।

নাম ধরে খোঁজা — `find`

`find` হলো তন্নতন্ন অভিযান — তুমি যেখান থেকে বলবে, সেখান থেকে শুরু করে ভেতরের **প্রতিটা** ঘরে ঢুকে খুঁজবে। গঠন:

```
find কোথায়-খুঁজবে -কী-দিয়ে-খুঁজবে মান
```

আগে খোঁজার মতো কিছু জিনিস বানিয়ে নিই:

Licensed to sakilorb@gmail.com · Do not redistribute

```
cd ~
mkdir -p bari/ghor1 bari/ghor2/khat-er-niche
touch bari/ghor1/linu-r-bol.txt
touch bari/ghor2/khat-er-niche/linu.txt
touch bari/mach.jpg
```

এবার খোঁজো:

```
find bari -name "linu.txt"
```

Output:

```
bari/ghor2/khat-er-niche/linu.txt
```

পেয়ে গেছি — খাটের নিচে! 😄 `find` শুধু বলে না ফাইলটা **আছে** কিনা, বলে **ঠিক কোথায়** আছে।

পুরো নাম মনে নেই? Wildcard তো শিখেছোই:

```
find bari -name "linu*"       # linu দিয়ে শুরু যা কিছু
find bari -name "*.jpg"       # সব ছবি
```

আরো কয়েকটা কাজের চাল:

```
find bari -type d             # শুধু directory খোঁজো
find bari -type f             # শুধু ফাইল খোঁজো
find ~ -name "*.pdf"          # পুরো home জুড়ে সব PDF
find bari -iname "LINU*"      # -i মানে case ভুলে যাও (Linu, LINU,
linu সব মিলবে)
```


Licensed to sakilorb@gmail.com · Do not redistribute

> 💡 `find / -name "কিছু"` চালালে পুরো সিস্টেম জুড়ে খুঁজবে — কিন্তু অনেক জায়গায় ঢোকার অনুমতি না থাকায় স্ক্রিনভর্তি `Permission denied` দেখাবে। ঘাবড়িয়ো না — ওগুলো error, তোমার ভুল না। Permission জিনিসটা কী, পর্ব ৪-এ পরিষ্কার হবে।

ফাইলের ভেতরে খোঁজা — `grep`

> 🐾 **দৃশ্য ১২: ডায়েরির পাতা ওল্টানো**

`find` দিয়ে লিনুকে খুঁজে পাওয়া যায়। কিন্তু ধরো অন্য সমস্যা: তুমি ডায়েরিতে কবে যেন লিখেছিলে লিনুর টিকার তারিখ — কিন্তু **কোন পাতায়?** এখন তো ফাইল খুঁজলে হবে না, খুঁজতে হবে **ফাইলের ভেতরে, লেখার মধ্যে।**

এই কাজের ওস্তাদ — `grep`। এটা Linux জগতের সবচেয়ে বেশি ব্যবহৃত কমান্ডগুলোর একটা, আর server-এ কাজ করা মানুষদের রুটি-রুজি।

```
cd ~
echo "linu ke tika dite hobe 15 tarikh" > diary.txt
echo "aj linu saradin ghumiyeche" >> diary.txt
echo "linu r khabar sesh, kinte hobe" >> diary.txt
```

(এই `>` আর `>>` চিহ্নের রহস্য অধ্যায় ৯-এ ফাঁস করবো — আপাতত জেনে রাখো, এতে লেখাগুলো ফাইলে জমা হলো।)

এবার খোঁজো:

```
grep "tika" diary.txt
```


Licensed to sakilorb@gmail.com · Do not redistribute

Output:

```
linu ke tika dite hobe 15 tarikh
```

`grep` পুরো ফাইল পড়ে **শুধু সেই লাইনগুলো** দেখালো যেখানে "tika" শব্দটা আছে। এক ফাইলে তিন লাইন তো চোখেই পড়া যায় — কিন্তু ভাবো সেই ফাইলে যদি ৫০,০০০ লাইন থাকে? `grep`-এর কাছে দুটোই সমান।

কাজের অপশনগুলো:

```
grep -i "LINU" diary.txt      # -i: ছোট-বড় হাতের পার্থক্য ভুলে যাও
grep -n "linu" diary.txt      # -n: কত নম্বর লাইনে আছে সেটাও বলো
grep -r "tika" ~/bari         # -r: পুরো ফোল্ডারের সব ফাইলের ভেতরে খোঁজো
grep -c "linu" diary.txt      # -c: কয়বার আছে, শুধু সংখ্যা বলো
```

এই `-r`-টা বিশেষ শক্তিশালী — ধরো তোমার একটা কোড প্রজেক্টে ২০০টা ফাইল, আর তুমি খুঁজছো `password` শব্দটা কোন কোন ফাইলে লেখা আছে:

```
grep -rn "password" project/
```

এক নিমেষে সব ফাইল, সব লাইন নম্বরসহ হাজির। এই একটা কমান্ড জানার কারণে ইন্টারভিউতে মানুষ চাকরি পেয়েছে — আমি মজা করছি না। 😄

বাকি তিন গোয়েন্দা — `which`, `whereis`, `locate`

এই তিনটা ছোট কিন্তু কাজের:

`which` — কোনো কমান্ড আসলে কোথায় থাকে:

Licensed to sakilorb@gmail.com · Do not redistribute

```
which ls
```

Output: `/usr/bin/ls` — মনে আছে অধ্যায় ৪-এ বলেছিলাম কমান্ডগুলোও ফাইল, হুকুমের বাক্সে থাকে? `which` সেই বাক্সের ঠিকানা বলে দেয়।

`whereis` — `which`-এর বড় ভাই; প্রোগ্রামের সাথে তার manual কোথায় সেটাও বলে।

`locate` — বিদ্যুৎগতির খোঁজ। `find` প্রতিবার সত্যি সত্যি সব ঘরে ঢুকে খোঁজে (ধীর কিন্তু নির্ভুল), আর `locate` একটা আগে-বানানো তালিকা (index) থেকে দেখে বলে দেয় (দ্রুত, কিন্তু তালিকা পুরনো হতে পারে)। অনেক সিস্টেমে এটা আগে থেকে ইনস্টল থাকে না:

```
sudo apt install plocate     # ইনস্টল (sudo কী, শিগগিরই জানবে)
locate diary.txt
```

> 💡 কোনটা কখন? **নাম দিয়ে ফাইল খুঁজলে** → `find`। **ফাইলের ভেতরের লেখা খুঁজলে** → `grep`। **কমান্ড কোথায় থাকে** → `which`। এই তিনটা মনে রাখলেই ৯৫% কাজ চলে যাবে।

🛠️ হাতেকলমে: হারানো জিনিসের অভিযান

এবার আসল পরীক্ষা। প্রথমে এই এলোমেলো স্তূপটা বানাও:

Licensed to sakilorb@gmail.com · Do not redistribute

```
cd ~
mkdir -p golokdhadha/ghor{1,2,3}/tak
touch golokdhadha/ghor1/purono-note.txt
touch golokdhadha/ghor2/tak/dorkari-file.txt
touch golokdhadha/ghor3/chobi.jpg
touch golokdhadha/ghor3/tak/aro-chobi.jpg
echo "linu r janmodin 12 December" > golokdhadha/ghor2/tak/
dorkari-file.txt
```

(💡 লক্ষ্য করেছো `ghor{1,2,3}` লিখলাম? এটা brace expansion — এক কথায় তিনটা ঘর! লিনুর আরেকটা গোপন কৌশল, বোনাস হিসেবে দিলাম।)

এবার অভিযান:

1. `find` দিয়ে বের করো `dorkari-file.txt` কোথায় লুকিয়ে আছে
2. এক কমান্ডে `golokdhadha`-র সব `.jpg` ফাইল খুঁজে বের করো
3. `golokdhadha`-র ভেতরে কয়টা directory আছে, `find` দিয়ে বের করো
4. `grep -r` দিয়ে বের করো "janmodin" শব্দটা কোন ফাইলে লেখা
5. `which` দিয়ে দেখো `grep` নিজে কোথায় থাকে — গোয়েন্দার বাড়ি কোথায়? 😄

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড	কাজ
`find জায়গা -name "নাম"`	নাম দিয়ে ফাইল/directory খোঁজো
`find জায়গা -type f` / `-type d`	শুধু ফাইল / শুধু directory
`grep "শব্দ" ফাইল`	ফাইলের ভেতরে শব্দ খোঁজো
`grep -rn "শব্দ" ফোল্ডার`	পুরো ফোল্ডারে, লাইন নম্বরসহ
`grep -i`	ছোট-বড় হাত উপেক্ষা করো
`which কমান্ড`	কমান্ডটা কোথায় থাকে
`locate নাম`	index থেকে বিদ্যুৎগতির খোঁজ

🐾 **পর্ব ২ শেষ।** লিনুর পুরো বাড়ি এখন তোমার চেনা — কোন ঘরে কী থাকে জানো, ঘরে ঘরে হাঁটতে পারো, জিনিস বানাতে-সরাতে-ফেলতে পারো, আর কিছু হারালে খুঁজে বের করতে পারো। সত্যি বলতে, এইটুকু দিয়েই তুমি টার্মিনালে দৈনন্দিন কাজ চালিয়ে দিতে পারবে।

পরের পর্বে লিনুর খাওয়া-দাওয়া — মানে সফটওয়্যার ইনস্টল করা শিখবো (`apt`-এর জগৎ), আর শিখবো ফাইল পড়া-লেখা — সেই বিখ্যাত vim থেকে বের হওয়ার কৌশলসহ। 😄

লিনু খাবারের বাটির পাশে বসে আছে। মিঁয়াও। 🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পর্ব ৩: লিনুর খাওয়া-দাওয়া (Packages & Software)

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৭: লিনুর খাবার দেওয়া

> 🐾 **দৃশ্য ১৩: খাবারের দোকান**

লিনু বাসায় settle হয়ে গেছে। এবার সবচেয়ে গুরুত্বপূর্ণ দায়িত্ব — **খাওয়ানো।**

বিড়ালের খাবার তুমি কোথা থেকে আনো? নিশ্চয়ই রাস্তা থেকে কুড়িয়ে না — আনো **বিশ্বস্ত দোকান** থেকে, যেখানে খাবারের মান যাচাই করা, মেয়াদ লেখা, আর নকল হওয়ার ভয় নেই।

Linux-এ সফটওয়্যার ইনস্টল করাও ঠিক তাই। Windows-এ আমাদের অভ্যাস কী? Google-এ সার্চ কোনো এক ওয়েবসাইট `setup.exe` ডাউনলোড Next Next Next সাথে ফাউ হিসেবে তিনটা toolbar আর একটা ভাইরাস। 😅

Linux এই ব্যবস্থাটাই পাল্টে দিয়েছে। এখানে আছে **package manager** — লিনুর নিজস্ব, বিশ্বস্ত খাবারের দোকান।

Package আর Package Manager

Package = একটা সফটওয়্যার, সুন্দর করে মোড়কে বাঁধা — ভেতরে প্রোগ্রামটা, তার নিয়মকানুন, আর একটা তালিকা: "আমাকে চালাতে আর কী কী লাগবে" (এগুলোকে বলে **dependency**)।

Package Manager = সেই দোকানদার, যে এক হুকুমে খুঁজে আনে, dependency-সহ ইনস্টল করে, আপডেট রাখে, আর দরকারে ফেরতও নেয়।

Licensed to sakilorb@gmail.com · Do not redistribute

Ubuntu/Debian-এর দোকানদারের নাম **APT** (**A**dvanced **P**ackage **T**ool)। অন্য distro-র দোকানদার অন্য — Fedora-তে `dnf`, Arch-এ `pacman` — কিন্তু ধারণা হুবহু এক। একটা শিখলে বাকিগুলো শুধু নাম বদল।

আর দোকানের গুদামগুলোকে বলে **repository** (সংক্ষেপে repo) — ইন্টারনেটে থাকা বিশাল সার্ভার, যেখানে হাজার হাজার যাচাই করা package সাজানো আছে।

দোকানদারের সাথে প্রথম কথা — `sudo`

APT-কে হুকুম দেওয়ার আগে একটা জিনিস জানতে হবে। সফটওয়্যার ইনস্টল করা মানে **পুরো বাড়ির জিনিসপত্র বদলানো** — এটা যেকোনো বাসিন্দার কাজ না, এর জন্য **মালিকের ক্ষমতা** লাগে।

সেই ক্ষমতা চাওয়ার শব্দটা হলো `sudo` — কমান্ডের সামনে বসিয়ে দিলে তুমি সাময়িকভাবে মালিকের ক্ষমতা নিয়ে কাজটা করো। প্রথমবার তোমার password চাইবে (সেই ইনস্টলের সময়েরটা — আর মনে আছে তো, টাইপ করলে স্ক্রিনে কিছু দেখায় না!)।

> ⚠️ `sudo` মানে "আমি জানি আমি কী করছি।" এখন আমরা শুধু সফটওয়্যার ইনস্টলে ব্যবহার করবো — এর পুরো দর্শন আর দায়িত্বের গল্প অধ্যায় ১০-এ।

`apt update` — মেনু দেখা

দোকানে ঢুকে প্রথম কাজ — **আজকের মেনু দেখা:**

```
sudo apt update
```


Licensed to sakilorb@gmail.com · Do not redistribute

এই কমান্ডটা নিয়ে সবচেয়ে বড় ভুল ধারণাটা এখনই ভেঙে দিই: `apt update` **কোনো সফটওয়্যার আপডেট করে না!** এটা শুধু দোকানের **সর্বশেষ মেনু-তালিকাটা** নামিয়ে আনে — কোন package-এর কোন সংস্করণ এখন পাওয়া যাচ্ছে। জিনিস কেনা হয় না, শুধু ক্যাটালগ হাতে আসে।

Output-এর শেষে এরকম দেখতে পারো:

```
Reading package lists... Done
Building dependency tree... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see
them.
```

মানে: মেনু হালনাগাদ হলো, আর দেখা গেলো তোমার ঘরের ১২টা জিনিসের নতুন সংস্করণ দোকানে এসেছে।

`apt upgrade` — সত্যিকারের খাওয়ানো

মেনু তো দেখা হলো — এবার পুরনো জিনিসগুলো নতুন করে আনা:

```
sudo apt upgrade
```

এটা তোমার ইনস্টল করা সব package-কে মেনুর সর্বশেষ সংস্করণে তুলে দেয়। চালালে দেখাবে কী কী আপগ্রেড হবে, কত জায়গা লাগবে, তারপর জিজ্ঞেস করবে:

```
Do you want to continue? [Y/n]
```

`Y` চেপে (বা শুধু Enter — বড় হাতের `Y` মানে ওটাই default) এগিয়ে যাও।

Licensed to sakilorb@gmail.com · Do not redistribute

> 💡 জোড়াটা মনে রাখার সবচেয়ে সহজ সূত্র: `update` = **মেনু দেখা,** `upgrade` = **খাওয়ানো।** আর বাস্তবে দুটো প্রায় সবসময় জোড়ায় চলে:

```
sudo apt update && sudo apt upgrade
```

(`&&` মানে — প্রথমটা সফল হলে তারপর দ্বিতীয়টা চালাও। এক ঢিলে দুই হুকুম!)

সপ্তাহে অন্তত একবার এই জোড়া চালানোর অভ্যাস করো — লিনু সুস্থ থাকবে, নিরাপত্তার ফুটোগুলোও বন্ধ থাকবে।

নতুন খাবার আনা — `apt install`

এবার আসল মজা। চলো লিনুর জন্য নতুন কিছু আনি:

```
sudo apt install htop
```

`htop` একটা দারুণ জিনিস — সিস্টেমের স্বাস্থ্য দেখার রঙিন ড্যাশবোর্ড(অধ্যায় ১১-তে এটা দিয়ে লিনুর health checkup করবো)। ইনস্টল শেষে চালিয়ে দেখো:

```
htop
```

রঙিন গ্রাফ, চলন্ত সংখ্যা — তোমার কম্পিউটারের হৃদস্পন্দন! `q` চেপে বের হও (মনে আছে তো — **q মানে quit**)।

একসাথে কয়েকটাও আনা যায়:

```
sudo apt install tree curl wget
```


Licensed to sakilorb@gmail.com · Do not redistribute

`tree`-টা এখনই চালিয়ে দেখো:

```
cd ~
tree bari
```

মনে আছে পর্ব ২-এর `bari` ফোল্ডারটা? `tree` সেটাকে সত্যিকারের গাছের মতো ছবি এঁকে দেখাবে — `ls`-এর চেয়ে অনেক সুন্দর করে।

খোঁজা আর জানা — `apt search`, `apt show`

দোকানে কী কী আছে জানতে:

```
apt search screenshot        # নাম/বর্ণনায় "screenshot" আছে এমন package
apt show htop                # htop-এর বিস্তারিত: সংস্করণ, আকার, বর্ণনা
apt list --installed         # আমার ঘরে এখন কী কী আছে
```

> 💡 খেয়াল করো — এগুলোতে `sudo` লাগেনি! কারণ এগুলো শুধু **দেখা**, কিছু **বদলানো** না।
>
> নিয়মটা সুন্দর: পড়তে sudo লাগে না, বদলাতে লাগে।

ফেরত দেওয়া — `apt remove`

কোনো জিনিস আর ভালো লাগছে না?

```
sudo apt remove tree
```


Licensed to sakilorb@gmail.com · Do not redistribute

package চলে যাবে, কিন্তু তার configuration ফাইলগুলো থেকে যাবে (যদি পরে আবার ইনস্টল করো, সেটিংস যাতে বেঁচে থাকে)। একদম শিকড়সুদ্ধ মুছতে চাইলে:

```
sudo apt purge tree
```

আর মাঝে মাঝে ঘর ঝাড়ু দিতে:

```
sudo apt autoremove
```

এটা সেই dependency-গুলো ফেলে দেয় যেগুলো কোনো package-এর সাথে এসেছিলো, কিন্তু সেই package চলে যাওয়ার পর এখন অকারণে পড়ে আছে — সোফার নিচে জমে থাকা পুরনো খেলনার মতো। 😄

(ও হ্যাঁ — `tree` মুছে দেখালাম, কিন্তু জিনিসটা কাজের। আবার ইনস্টল করে নাও: `sudo apt install tree`)

বাইরের খাবার — `.deb`, PPA, Snap

> **🐾 দৃশ্য ১৪: রাস্তার খাবারের লোভ**

একদিন বাইরে থেকে এসে দেখলে — প্রতিবেশী লিনুকে কী যেন খাওয়াচ্ছে। জিনিসটা দেখতে লোভনীয়, লিনুও মজা করে খাচ্ছে। কিন্তু তোমার মনে খচখচ — **এটা কি নিরাপদ?**

APT-র দোকানে সব পাওয়া যায় না। কিছু সফটওয়্যার (যেমন Chrome, VS Code) নিজেদের ওয়েবসাইট থেকে আনতে হয়। তখন তিনটা রাস্তা:

Licensed to sakilorb@gmail.com · Do not redistribute

১. `.deb` ফাইল — Windows-এর `setup.exe`-র Linux ভাই। ওয়েবসাইট থেকে `.deb` নামিয়ে ইনস্টল:

```
sudo apt install ./google-chrome-stable_current_amd64.deb
```

(শুরুর `./` টা জরুরি — এটা APT-কে বলে "দোকান থেকে না, **এই ফাইলটা** থেকে ইনস্টল করো।")

২. PPA (Personal Package Archive) — কারো ব্যক্তিগত দোকান। মূল দোকানে যে জিনিস নেই বা পুরনো সংস্করণ আছে, সেটার টাটকা সংস্করণ অনেকে নিজের PPA-তে রাখে। যোগ করা হয় `sudo add-apt-repository ppa:নাম` দিয়ে।

৩. Snap — Ubuntu-র আধুনিক ব্যবস্থা, যেখানে প্রতিটা সফটওয়্যার নিজের সব dependency-সহ একটা বাক্সে মোড়া থাকে: `sudo snap install code --classic`

> ⚠️ **রাস্তার খাবারের নিয়ম:** `.deb` বা PPA যোগ করা মানে সেই উৎসকে তোমার সিস্টেমে ঢোকার চাবি দেওয়া। শুধু **অফিসিয়াল ওয়েবসাইট** (google.com, code.visualstudio.com) বা সুপরিচিত উৎস থেকে নাও। "Free download full version" মার্কা সাইট থেকে `.deb` নামানো আর রাস্তার ময়লা থেকে খাবার তুলে লিনুকে দেওয়া — একই জিনিস।

> 💡 সোনালি নিয়ম: **আগে APT-র দোকানে খোঁজো (`apt search`)। সেখানে থাকলে সেখান থেকেই নাও।** না থাকলে তবেই অফিসিয়াল সাইটে যাও।

Licensed to sakilorb@gmail.com · Do not redistribute

🛠️ হাতেকলমে

1. `sudo apt update && sudo apt upgrade` চালিয়ে লিনুকে টাটকা করে নাও
2. `apt search cowsay` দিয়ে খোঁজো — কী পেলে?
3. `cowsay` ইনস্টল করো, তারপর চালাও: `cowsay "miyaw"` — 🐄 কথা বলবে!
4. আরো মজা: `sudo apt install fortune` তারপর `fortune | cowsay` (এই `|` চিহ্নটাই পরের পর্বের নায়ক 😉)
5. `apt show htop` পড়ে দেখো — package-টার আকার কত?
6. `apt list --installed | wc -l` — তোমার ঘরে মোট কয়টা package আছে?

📜 হুকুমনামা

কমান্ড	কাজ
`sudo apt update`	দোকানের মেনু হালনাগাদ (কিছু ইনস্টল হয় না!)
`sudo apt upgrade`	সব package নতুন সংস্করণে
`sudo apt install নাম`	ইনস্টল
`sudo apt remove নাম`	মুছে ফেলা (config থাকে)
`sudo apt purge নাম`	শিকড়সুদ্ধ মোছা
`sudo apt autoremove`	অকেজো dependency পরিষ্কার
`apt search শব্দ`	দোকানে খোঁজা
`apt show নাম`	package-এর বিস্তারিত

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৮: ফাইল পড়া ও লেখা

> 🐾 **দৃশ্য ১৫: লিনুর ডায়েরি**

লিনুকে নিয়ে তুমি একটা ডায়েরি রাখা শুরু করেছো — কবে টিকা, কবে কী খেলো, কবে কার সাথে ঝগড়া করলো। ডায়েরি তো লেখা হচ্ছে, কিন্তু **পড়বে কীভাবে?** আর টার্মিনালে বসে **লিখবেই** বা কীভাবে?

মজার ব্যাপার — Linux-এ ফাইল পড়ার একটা না, **চারটা** আলাদা কায়দা আছে। শুনে বাড়াবাড়ি মনে হচ্ছে? দাঁড়াও, প্রতিটার আলাদা কারণ আছে — যেমন বিড়ালের ঘুমেরও তো রকমফের আছে: ভাতঘুম, রোদঘুম, সোফাঘুম। 😄

আগে একটা ডায়েরি বানিয়ে নিই খেলার জন্য:

```
cd ~
echo "Din 1: linu bashay elo" > linu-diary.txt
echo "Din 2: linu sofa achorailo" >> linu-diary.txt
echo "Din 3: linu tika nilo, kanna korlo" >> linu-diary.txt
echo "Din 4: linu machh khelo" >> linu-diary.txt
echo "Din 5: linu sarakkhon ghumailo" >> linu-diary.txt
```

কায়দা ১: `cat` — এক নিঃশ্বাসে পুরোটা

```
cat linu-diary.txt
```


Licensed to sakilorb@gmail.com · Do not redistribute

`cat` পুরো ফাইলটা একবারে স্ক্রিনে ঢেলে দেয়। ছোট ফাইলের জন্য perfect — ঝটপট দেখো, কাজ শেষ।

(নামটা বিড়ালের সম্মানে না, দুঃখিত 😅 — এসেছে con**cat**enate থেকে, মানে জোড়া লাগানো: `cat file1 file2` লিখলে দুটো ফাইল পরপর জুড়ে দেখায়।)

কিন্তু সমস্যাটা ভাবো — ফাইলে যদি ১০,০০০ লাইন থাকে? `cat` সব ঢেলে দেবে, স্ক্রিন দিয়ে ঝড়ের মতো লেখা ছুটবে, আর তুমি দেখবে শুধু শেষ কয়েক লাইন। তখন দরকার —

কায়দা ২: `less` — ধীরেসুস্থে পাতা উল্টে

```
less linu-diary.txt
```

`less` ফাইলটা খুলে দেয় **পড়ার ঘরে** — এক স্ক্রিনে যতটুকু ধরে ততটুকু দেখায়, বাকিটা তুমি নিজে ওল্টাবে:

- **তীর চিহ্ন / PageUp / PageDown** — ওপরে-নিচে
- `/শব্দ` তারপর Enter — ফাইলের ভেতরে খোঁজা (`n` চাপলে পরের মিল)
- `g` — একদম শুরুতে, `G` — একদম শেষে
- `q` — বের হও (আমাদের পুরনো বন্ধু!)

চেনা চেনা লাগছে? অধ্যায় ৩-এর `man` আসলে ভেতরে ভেতরে `less` দিয়েই পাতা দেখায় — তাই নিয়মগুলো হুবহু এক। এক শেখায় দুই জায়গায় কাজ!

> 💡 বড় ফাইল = `less`, ছোট ফাইল = `cat`। ব্যস।

Licensed to sakilorb@gmail.com · Do not redistribute

কায়দা ৩ ও ৪: `head` আর `tail` — মাথা আর লেজ

মাঝে মাঝে পুরো ফাইল দরকারই নেই — শুধু শুরুটা বা শেষটা দেখলেই চলে:

```
head linu-diary.txt        # প্রথম ১০ লাইন
head -n 2 linu-diary.txt   # প্রথম ২ লাইন
tail linu-diary.txt        # শেষ ১০ লাইন
tail -n 2 linu-diary.txt   # শেষ ২ লাইন
```

বিড়ালের **মাথা** (head) আর **লেজ** (tail) — এর চেয়ে সহজে মনে রাখার উপায় আর হয় না। 🐾

"শেষ কয়েক লাইন দেখা" জিনিসটা তুচ্ছ মনে হচ্ছে? সার্ভারের জগতে `tail` হলো রাজা — কারণ log ফাইলে **সবচেয়ে নতুন ঘটনা সবসময় শেষে** লেখা হয়। "সাইট এইমাত্র ক্র্যাশ করলো, কী হয়েছে?" — উত্তর সবসময় log-এর লেজে।

`tail -f` — লিনুর ডায়েরি live পড়া

এবার এই অধ্যায়ের সবচেয়ে দারুণ কৌশল:

```
tail -f linu-diary.txt
```

`-f` মানে **follow** — কমান্ডটা শেষ হয়ে যায় না, **বসে থাকে**, আর ফাইলে নতুন লাইন যোগ হওয়া মাত্র সাথে সাথে দেখায়। যেন ডায়েরিটা তোমার চোখের সামনে **নিজে নিজে লেখা হচ্ছে।**

নিজে দেখো জাদুটা — **দুটো টার্মিনাল খোলো** (WSL-এ আরেকটা Ubuntu উইন্ডো, বা Ubuntu Desktop-এ `Ctrl+Alt+T`)। প্রথমটায়:

```
tail -f ~/linu-diary.txt
```


Licensed to sakilorb@gmail.com · Do not redistribute

দ্বিতীয়টায়:

```
echo "Din 6: linu ekhon LIVE!" >> ~/linu-diary.txt
```

প্রথম টার্মিনালের দিকে তাকাও — লাইনটা **নিজে নিজে হাজির!** ✨ যতবার খুশি নতুন লাইন পাঠাও, ততবার হাজির হবে। দেখা শেষে `Ctrl+C` চেপে বের হও (`Ctrl+C` = "থামো" — চলন্ত যেকোনো কমান্ড থামানোর সর্বজনীন হুকুম, এটাও আজই মুখস্থ করে ফেলো)।

সার্ভারে এটাই হয় `tail -f /var/log/nginx/access.log` — কে কখন তোমার সাইটে ঢুকছে, **live** দেখা। সিনেমার হ্যাকারদের স্ক্রিনে যে ঝরঝর করে লেখা নামতে দেখো — অর্ধেক সময় ওটা আসলে এই `tail -f` । 😎

এবার লেখা — `nano` , ভদ্র সম্পাদক

পড়া তো হলো — এবার লেখা। টার্মিনালের ভেতরেই আস্ত text editor আছে। সবচেয়ে বন্ধুসুলভটার নাম **nano:**

```
nano linu-diary.txt
```

খুলে গেলো এডিটর — এবার একদম সাধারণভাবে লেখো, তীর চিহ্নে ঘোরো, Backspace-এ মোছো। কোনো চালাকি নেই।

নিচের দিকে তাকাও — সাহায্য-বার দেখাচ্ছে: `^O Write Out` , `^X Exit` … এই `^` মানে **Ctrl**। যা যা লাগবে:

- **সেভ:** `Ctrl+O` ফাইলের নাম দেখাবে Enter
- **বের হওয়া:** `Ctrl+X` (সেভ না করা থাকলে জিজ্ঞেস করবে — `Y` মানে সেভ করে বের হও)

Licensed to sakilorb@gmail.com · Do not redistribute

ডায়েরিতে "Din 7" যোগ করে সেভ করে বের হও, তারপর `cat` দিয়ে দেখো লেখাটা আছে কিনা। এই তো — তুমি এখন টার্মিনালে বসেই ফাইল সম্পাদনা করতে পারো!

vim — জেদি বিড়ালের ঘর

> **🐾 দৃশ্য ১৬: পাশের বাড়ির হুলো**

পাশের বাড়িতে একটা হুলো বিড়াল আছে — বিশাল, রাজকীয়, আর ভয়ানক জেদি। যারা ওকে পোষ মানিয়েছে তারা বলে, "এর মতো বিড়াল হয় না!" কিন্তু নতুন কেউ ওর ঘরে ঢুকলে একটা মজার সমস্যা হয় — **ঢুকতে পারে, বের হতে পারে না।** 😹

এই হুলোর নাম **vim**। প্রোগ্রামারদের জগতে কিংবদন্তি এক এডিটর — যারা আয়ত্ত করেছে তারা বলে চিন্তার গতিতে লেখা যায়। কিন্তু এর নিয়মকানুন একদম আলাদা, আর ইন্টারনেটের অন্যতম বিখ্যাত রসিকতাই হলো: "How do I exit vim?" (এই প্রশ্নটা Stack Overflow-তে ২০ লাখের বেশি মানুষ দেখেছে — আমি বানিয়ে বলছি না!)

তোমাকে এখন vim শিখতে হবে না। কিন্তু **একটা পরিস্থিতির জন্য তৈরি থাকতে হবে:** কোনো একদিন, কোনো এক সার্ভারে, কোনো কমান্ড হুট করে তোমাকে vim-এর ভেতরে ফেলে দেবে (git এটা প্রায়ই করে)। তখন যেন প্যানিক না হয়, তাই আজই মহড়া দিয়ে রাখি:

```
vim test.txt
```

ঢুকে পড়েছো হুলোর ঘরে। এখন **মন্ত্রটা** শেখো:

বের হওয়ার মন্ত্র: `Esc` চাপো, তারপর লেখো `:q!` তারপর Enter!

Licensed to sakilorb@gmail.com · Do not redistribute

মানে: `Esc` (যেখানেই থাকো, শান্ত হও) → `:q!` (quit, জোর করে, কিছু সেভ না করেই) → Enter। ব্যস, মুক্তি!

আর যদি কখনো ভেতরে লিখতেই হয়, ন্যূনতম যা জানা দরকার: vim-এর দুটো জগৎ আছে — **হুকুমের জগৎ** (Normal mode, যেখানে অক্ষর চাপলে লেখা হয় না, কমান্ড হয়) আর **লেখার জগৎ** (Insert mode)। `i` চাপলে লেখার জগতে ঢোকো, `Esc` চাপলে হুকুমের জগতে ফেরো, আর `:wq` লিখে Enter দিলে **সেভ করে** বের হও।

মন্ত্র	কাজ
`i`	লেখা শুরু (insert)
`Esc`	লেখা থামাও, হুকুমের জগতে ফেরো
`:q!`	সেভ ছাড়া পালাও
`:wq`	সেভ করে বের হও

এই চারটাই যথেষ্ট। হুলোকে পুরো পোষ মানানো — সে অন্য কোনো বইয়ের গল্প। আপাতত আমাদের রোজকার সঙ্গী nano। 😄

🛠️ হাতেকলমে

1. `nano` দিয়ে `amar-lokkho.txt` নামে একটা ফাইল বানাও, তাতে লেখো এই বই শেষ করে তুমি কী বানাতে চাও — সেভ করে বের হও
2. `cat` দিয়ে পড়ো, তারপর `head -n 1` দিয়ে শুধু প্রথম লাইন দেখো
3. দুই টার্মিনালের `tail -f` জাদুটা নিজে করে দেখো (না করে থাকলে — এটাই এই অধ্যায়ের সেরা মজা, বাদ দিও না!)

Licensed to sakilorb@gmail.com · Do not redistribute

4. `less /etc/passwd` খোলো (এটা সিস্টেমের ব্যবহারকারীদের তালিকা) — `/root` লিখে খোঁজো, `G` চেপে শেষে যাও, `q` চেপে বের হও
5. সাহস পরীক্ষা: `vim test.txt` → `i` চেপে দুই লাইন লেখো → `Esc` → `:wq` → `cat test.txt` দিয়ে প্রমাণ দেখো। তুমি এইমাত্র vim থেকে **লিখে, সেভ করে** বের হলে — যেটা লাখো মানুষ পারে না 😎

📜 হুকুমনামা

কমান্ড	কাজ
`cat ফাইল`	পুরোটা একবারে দেখাও
`less ফাইল`	পাতা উল্টে পড়ো (`/খোঁজা`, `q` বের)
`head -n X ফাইল`	প্রথম X লাইন
`tail -n X ফাইল`	শেষ X লাইন
`tail -f ফাইল`	live দেখো (`Ctrl+C` থামাও)
`nano ফাইল`	সম্পাদনা (`Ctrl+O` সেভ, `Ctrl+X` বের)
`vim` থেকে পালানো	`Esc` তারপর `:q!` তারপর Enter
`Ctrl+C`	চলন্ত কমান্ড থামাও

🐾 **পর্ব ৩ শেষ।** লিনুর খাওয়া-দাওয়ার ব্যবস্থা এখন তোমার হাতে — বিশ্বস্ত দোকান থেকে সফটওয়্যার আনতে পারো, রাস্তার খাবার চিনতে পারো, আর ফাইল পড়া-লেখা দুটোই পারো। এমনকি vim-এর ঘর থেকে জ্যান্ত বেরিয়ে আসার মন্ত্রও জানো — এটা কিন্তু কম কথা না। 😄

খেয়াল করেছো, যাত্রাটা কেমন বদলে যাচ্ছে? প্রথম পর্বে তুমি লিনুকে ভয় পেতে — এখন ওকে খাওয়াচ্ছো, ওর ডায়েরি পড়ছো। পরের পর্বে সম্পর্কটা আরেক ধাপ গভীর হবে: লিনুকে শেখাবো

Licensed to sakilorb@gmail.com · Do not redistribute

হুকুমের চেইন — একটা কমান্ডের ফলাফল আরেকটায় জুড়ে দেওয়া (সেই রহস্যময় `|`, `>`, `>>` চিহ্নের পুরো গল্প), শিখবো কে লিনুকে ছুঁতে পারবে (permission আর `sudo`-র আসল দর্শন), আর করবো লিনুর পুরো health checkup।

এই পর্ব থেকেই তুমি সাধারণ ব্যবহারকারী থেকে **power user** হওয়ার পথে পা রাখবে। লিনু লেজ উঁচিয়ে দাঁড়িয়ে আছে। 🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পর্ব ৪: লিনুকে হুকুম শেখানো (Power User)

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৯: হুকুমের চেইন — Pipe ও Redirection

> 🐾 **দৃশ্য ১৭: সার্কাসের প্রশিক্ষণ**

লিনু এখন একটা একটা হুকুম ভালোই শোনে — "বল আন", "বসো", "খাও"। কিন্তু আজ তুমি নতুন কিছু চেষ্টা করলে: "লিনু — বলটা আন, ঝুড়িতে রাখো, তারপর ঝুড়িটা ঠেলে আমার কাছে নিয়ে এসো।"

তিনটা কাজ, একটার ফলাফল পরেরটার কাঁচামাল — একটা **চেইন**। লিনু মাথা কাত করে তাকালো... তারপর করে দেখালো! 🐱

আজ আমরা Linux-এর সবচেয়ে সুন্দর ধারণাটা শিখবো। যে জিনিসটার কারণে টার্মিনাল মাউসের চেয়ে হাজার গুণ শক্তিশালী — **ছোট ছোট হুকুম জুড়ে বিশাল কাজ করানো।**

গোপন কথাটা আগে: প্রতিটা কমান্ডের মুখ আর কান আছে

এতদিন যা শিখেছো, সবগুলো কমান্ডের একটা মিল খেয়াল করেছো? প্রতিটা কমান্ড কিছু একটা **বলে** — `ls` বলে ফাইলের তালিকা, `cat` বলে ফাইলের ভেতরটা, `grep` বলে খুঁজে পাওয়া লাইন। এই বলাটার একটা নাম আছে: **output** (আনুষ্ঠানিক নাম stdout)।

আর অনেক কমান্ড **শুনতেও** পারে — বাইরে থেকে আসা জিনিস নিয়ে কাজ করতে পারে। এটার নাম **input**।

এতদিন সব কমান্ডের মুখ ছিলো স্ক্রিনের দিকে — যা বলে, স্ক্রিনে দেখায়। আজকের জাদু: **সেই মুখ ঘুরিয়ে অন্যদিকে তাক করা।**

Licensed to sakilorb@gmail.com · Do not redistribute

`>` — ফলাফল ফাইলে ঢালা

মনে আছে পর্ব ২ থেকে এই চিহ্নটা ব্যবহার করছি, আর আমি বলেছিলাম "রহস্য পরে ফাঁস করবো"? সময় এসেছে। 😄

```
ls -l > talika.txt
```

চালাও — স্ক্রিনে **কিছুই আসবে না।** তাহলে গেলো কোথায়? দেখো:

```
cat talika.txt
```

ওই যে! `ls -l`-এর পুরো ফলাফল ফাইলের ভেতরে। `>` চিহ্নটা কমান্ডের মুখ স্ক্রিন থেকে ঘুরিয়ে **ফাইলের দিকে** তাক করে দিয়েছে। এটাকে বলে **redirection**।

> ⚠️ **সাবধান:** `>` ফাইলে আগে যা ছিলো সব **মুছে** নতুনটা লেখে। `talika.txt`-তে তোমার জীবনের গল্প লেখা থাকলেও `ls > talika.txt` চালানো মাত্র সব সাফ — শুধু নতুন তালিকা। কোনো সতর্কবার্তা ছাড়াই!

`>>` — মোছা নয়, জোড়া

আগেরটার ভদ্র ভাই। `>>` ফাইলের **শেষে যোগ করে**, আগেরটা মোছে না:

```
echo "Din 8: linu notun khela shikhlo" >> linu-diary.txt
```

এবার পর্ব ২-৩ এর সব রহস্য পরিষ্কার: ডায়েরি বানানোর সময় প্রথম লাইন `>` দিয়ে (নতুন ফাইল), বাকি লাইনগুলো `>>` দিয়ে (যোগ) লিখেছিলাম।

Licensed to sakilorb@gmail.com · Do not redistribute

> 💡 মনে রাখার সূত্র: **এক তীর = নতুন করে লেখো, দুই তীর = লেজে জোড়ো।** সন্দেহ হলে `>>` ব্যবহার করো — জোড়া নিরাপদ, মোছা বিপজ্জনক।

`|` — এই অধ্যায়ের নায়ক: Pipe

Redirection ফলাফল পাঠায় **ফাইলে**। আর pipe (`|` — কীবোর্ডে Shift + Backslash) ফলাফল পাঠায় **আরেকটা কমান্ডের কানে:**

```
কমান্ড১ | কমান্ড২
```

মানে: কমান্ড১ যা **বলবে**, কমান্ড২ সেটা **শুনে** তার ওপর কাজ করবে। একজনের মুখ আরেকজনের কানে লাগানো পাইপ — নাম এজন্যই pipe।

সবচেয়ে কাজের উদাহরণ দিয়ে শুরু:

```
ls /usr/bin | less
```

`/usr/bin`-এ হাজারখানেক কমান্ড আছে — `ls` একা চালালে স্ক্রিন দিয়ে ঝড় বয়ে যেতো। Pipe দিয়ে সেই ঝড় পাঠিয়ে দিলাম `less`-এর পড়ার ঘরে — এখন আরামে পাতা উল্টে দেখো। (`q` — বলতে হবে না নিশ্চয়ই 😄)

আরেকটা, আমাদের পুরনো বন্ধু `grep`-এর সাথে:

```
ls /usr/bin | grep zip
```


Licensed to sakilorb@gmail.com · Do not redistribute

`ls` বললো হাজারটা নাম, `grep` শুনে ছেঁকে দিলো শুধু যেগুলোয় "zip" আছে। **এই জুটিটা — `কিছু | grep কিছু` — Linux জগতের সবচেয়ে বেশি ব্যবহৃত চাল।** Server-এ মানুষ দিনে পঞ্চাশবার এটা চালায়।

আর মনে আছে গত অধ্যায়ের `fortune | cowsay`? এখন বোঝো — `fortune` একটা বাণী **বললো**, `cowsay` সেটা **শুনে** গরুর মুখে বসিয়ে দিলো। 🐄

চেইনের কারিগররা: `wc`, `sort`, `uniq`, `cut`

Pipe-এর আসল মজা পেতে কিছু ছোট, এক-কাজের-ওস্তাদ কমান্ড চেনা দরকার। এরা একা তেমন কিছু না — কিন্তু চেইনে বসলে জাদুকর।

`wc` **(word count)** — গোনে: লাইন, শব্দ, অক্ষর। সবচেয়ে বেশি লাগে `-l` (লাইন গোনা):

```
ls /usr/bin | wc -l
```

এক লাইনে জেনে গেলে তোমার সিস্টেমে কয়টা কমান্ড আছে! (আগের অধ্যায়ের হাতেকলমে-র শেষ প্রশ্নটাও এই চালেই করেছিলে — এখন বুঝলে কীভাবে কাজ করেছিলো।)

`sort` — সাজায়। `uniq` — পাশাপাশি একই লাইন থাকলে একটা রাখে। `cut` — প্রতি লাইন থেকে নির্দিষ্ট অংশ কেটে নেয়।

এগুলো কথায় না বুঝিয়ে সরাসরি যুদ্ধক্ষেত্রে নামাই — এই অধ্যায়ের mini project।

🛠️ Mini Project: Log থেকে গোয়েন্দাগিরি — এক লাইনে

> **🐾 দৃশ্য ১৮: কে কে এসেছিলো বাড়িতে?**

Licensed to sakilorb@gmail.com · Do not redistribute

তুমি বাইরে ছিলে। ফিরে দেখো দরজার সামনে লিনুর "অতিথি খাতা" — ও নাকি সব দর্শনার্থীর হিসাব রাখে। খাতায় শত শত লাইন। প্রশ্ন: **কোন অতিথি সবচেয়ে বেশিবার এসেছে?**

এটা কোনো খেলনা সমস্যা না — একটা ওয়েবসাইটের access log থেকে "কোন IP সবচেয়ে বেশি request করেছে" বের করা মানে হ্যাকিং-চেষ্টা ধরা, DDoS শনাক্ত করা। Server-জীবনের একদম আসল কাজ। চলো খাতাটা বানাই:

```
cd ~
cat > otithi-khata.txt << 'EOF'
10.0.0.5 GET /home
10.0.0.9 GET /about
10.0.0.5 GET /products
10.0.0.7 GET /home
10.0.0.5 GET /contact
10.0.0.9 GET /home
10.0.0.5 GET /products
10.0.0.7 GET /about
10.0.0.5 GET /home
10.0.0.9 GET /products
EOF
```

(এই `cat > ফাইল << 'EOF'` চালটা একটা বোনাস কৌশল — টার্মিনালে সরাসরি কয়েক লাইন লিখে ফাইল বানানো। `EOF` লেখা পর্যন্ত সব ফাইলে চলে যায়।)

এবার ধাপে ধাপে গোয়েন্দাগিরি — প্রতিটা ধাপে চেইনে একটা করে কড়া যোগ করবো, আর **প্রতিটা ধাপ চালিয়ে output নিজের চোখে দেখবে:**

ধাপ ১: প্রতি লাইনের শুধু IP-টা কাটি:

```
cut -d ' ' -f 1 otithi-khata.txt
```


Licensed to sakilorb@gmail.com · Do not redistribute

`-d ' '` = স্পেস দিয়ে ভাগ করো, `-f 1` = প্রথম টুকরাটা দাও। Output: শুধু IP-র তালিকা।

ধাপ ২: সাজাই, যাতে একই IP পাশাপাশি আসে:

```
cut -d ' ' -f 1 otithi-khata.txt | sort
```

ধাপ ৩: এবার `uniq -c` — জোড়া ভেঙে গুনে দাও (count):

```
cut -d ' ' -f 1 otithi-khata.txt | sort | uniq -c
```

Output:

```
   5 10.0.0.5
   2 10.0.0.7
   3 10.0.0.9
```

(💡 এজন্যই আগে `sort` — `uniq` শুধু **পাশাপাশি** থাকা জোড়া চেনে। sort না করলে ভুল গুনতো।)

ধাপ ৪: সংখ্যা অনুযায়ী সাজাই, বড়টা ওপরে:

```
cut -d ' ' -f 1 otithi-khata.txt | sort | uniq -c | sort -rn
```

`-n` = সংখ্যা হিসেবে সাজাও, `-r` = উল্টো করে (বড় আগে)।

ধাপ ৫: শুধু চ্যাম্পিয়নটা দাও:

```
cut -d ' ' -f 1 otithi-khata.txt | sort | uniq -c | sort -rn |
head -n 1
```


Licensed to sakilorb@gmail.com · Do not redistribute

Output:

```
   5 10.0.0.5
```

পেয়ে গেছি! `10.0.0.5` — পাঁচবার এসেছে। 🕵️

এক মুহূর্ত থামো আর দেখো কী করলে: **পাঁচটা সাধারণ কমান্ড — `cut`, `sort`, `uniq`, `sort`, `head` — পাইপে জুড়ে একটা আস্ত data analysis!** কোনোটাই একা এই কাজ পারতো না। এটাই Unix দর্শন: প্রতিটা কমান্ড একটা কাজ ভালো পারবে, আর pipe দিয়ে জুড়ে যেকোনো কাজ বানানো যাবে। লেগো-র টুকরার মতো — টুকরাগুলো সরল, বানানো জিনিসটার সীমা নেই।

`<` — উল্টো দিকের তীর

পরিবারের শেষ সদস্য, কম ব্যবহৃত কিন্তু চিনে রাখো: `<` ফাইল থেকে **input খাওয়ায়:**

```
wc -l < otithi-khata.txt
```

`wc` ফাইলটা খেলো ইনপুট হিসেবে। ফলাফল `wc -l otithi-khata.txt`-র মতোই — পার্থক্য সূক্ষ্ম (output-এ ফাইলের নাম দেখায় না), আর script লেখার সময় মাঝে মাঝে কাজে লাগে। আপাতত শুধু জেনে রাখা যথেষ্ট।

🛠️ হাতেকলমে

1. `history > amar-itihas.txt` — তোমার এ পর্যন্ত চালানো সব কমান্ডের তালিকা ফাইলে ঢালো, `less` দিয়ে পড়ে দেখো নিজের যাত্রাটা 😄
2. `history | wc -l` — এ পর্যন্ত মোট কয়টা কমান্ড চালিয়েছো?
3. `history | grep apt` — apt দিয়ে কী কী করেছিলে?

Licensed to sakilorb@gmail.com · Do not redistribute

4. `linu-diary.txt`-তে মোট কয়টা লাইনে "linu" শব্দটা আছে — `grep` আর `wc -l` পাইপে জুড়ে বের করো
5. Otithi-khata-য় এবার উল্টো প্রশ্ন: **কোন পাতা (path) সবচেয়ে জনপ্রিয়?** Hint: `-f 1`-এর জায়গায় কোন টুকরা লাগবে ভাবো
6. Challenge: `ls /usr/bin | grep "^z"` চালাও — কী পেলে? `^` চিহ্নটা কী করছে অনুমান করো (উত্তর: "লাইনের শুরু" — regex জগতের প্রথম উঁকি, এই গভীর জঙ্গলের গল্প অন্যদিন 😄)

📜 হুকুমনামা

চিহ্ন/কমান্ড	কাজ
`>`	output ফাইলে (⚠️ আগেরটা মোছে)
`>>`	output ফাইলের শেষে জোড়ে
`|`	এক কমান্ডের output আরেকটার input-এ
`<`	ফাইল থেকে input
`wc -l`	লাইন গোনো
`sort` / `sort -rn`	সাজাও / সংখ্যায় উল্টো সাজাও
`uniq -c`	পাশাপাশি জোড়া গুনে ভাঙো
`cut -d ' ' -f 1`	স্পেসে ভেঙে প্রথম টুকরা

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১০: Permission — কে লিনুকে ছুঁতে পারবে

> 🐾 **দৃশ্য ১৯: লিনুর নিয়মকানুন**

তোমার বাসায় লিনুকে নিয়ে অলিখিত কিছু নিয়ম দাঁড়িয়ে গেছে:

- **তুমি** — লিনুকে কোলে নিতে পারো, খাওয়াতে পারো, ওষুধও খাওয়াতে পারো। তুমি **মালিক**।
- **পরিবারের সদস্যরা** — আদর করতে পারে, খেলতে পারে, কিন্তু ওষুধ খাওয়ানো নৈব নৈব চ।
- **বাইরের মানুষ** — দূর থেকে দেখতে পারে। ছোঁয়া? লিনুই দেবে না। 😾

খেয়াল করো — **একই বিড়াল, কিন্তু কে কী করতে পারবে তা নির্ভর করছে মানুষটা কে তার ওপর।** Linux-এ প্রতিটা ফাইল আর directory-র গায়ে ঠিক এই নিয়মটাই লেখা থাকে। আর এটা কোনো খুঁটিনাটি ফিচার না — এই permission ব্যবস্থাই হলো সেই কারণ, যার জন্য এক server-এ শত মানুষ কাজ করেও একজন আরেকজনের ফাইল ঘাঁটতে পারে না।

নিয়মটা পড়তে শেখা

`ls -l` তো কতবার চালিয়েছো — এবার output-এর **প্রথম রহস্যময় অংশটা** পড়তে শিখবো:

```
touch niyom.txt
ls -l niyom.txt
```

Output:

Licensed to sakilorb@gmail.com · Do not redistribute

```
-rw-r--r-- 1 rahim rahim 0 Jul 12 10:30 niyom.txt
```

ওই শুরুর `-rw-r--r--` — এটাই ফাইলের গায়ে লেখা নিয়মকানুন। ভেঙে পড়ি। মোট ১০টা ঘর:

```
-    rw-    r--    r--
↑     ↑      ↑      ↑
ধরন মালিক দল  বাকিরা
```

প্রথম ঘর — ধরন: `-` মানে সাধারণ ফাইল, `d` মানে directory।

বাকি ৯ ঘর — তিনজনের জন্য তিনটা করে অনুমতি। কারা এই তিনজন?

- **মালিক (owner/user)** — যে ফাইলটা বানিয়েছে। তুমি।
- **দল (group)** — মালিকের দলের লোকজন। পরিবারের সদস্যরা।
- **বাকিরা (others)** — সিস্টেমের বাকি সবাই। বাইরের মানুষ।

আর প্রত্যেকের জন্য তিন রকম অনুমতি:

- `r` **(read)** — পড়া। লিনুকে দেখা।
- `w` **(write)** — বদলানো/মোছা। লিনুকে খাওয়ানো, ওষুধ দেওয়া।
- `x` **(execute)** — চালানো (ফাইলটা যদি প্রোগ্রাম হয়)। লিনুকে দিয়ে খেলা দেখানো।

অনুমতি না থাকলে সেই ঘরে `-` বসে। তাহলে `-rw-r--r--` মানে দাঁড়ালো:

Licensed to sakilorb@gmail.com · Do not redistribute

কে	অনুমতি	মানে
মালিক	`rw-`	পড়তে + বদলাতে পারবে
দল	`r--`	শুধু পড়তে পারবে
বাকিরা	`r--`	শুধু পড়তে পারবে

ব্যস! যে লাইনটা এতদিন হিজিবিজি লাগতো, এখন তুমি অনর্গল পড়তে পারো। `ls -l` চালিয়ে আশেপাশের ফাইলগুলোর নিয়ম পড়ে দেখো তো!

নিয়ম বদলানো — `chmod`

মালিক হিসেবে তুমি নিয়ম বদলাতে পারো — কমান্ডের নাম `chmod` (**ch**ange **mod**e)। ব্যবহারের সহজ কায়দাটা হলো **কাকে (u/g/o/a) + দেবে না কাড়বে (+/-) + কী (r/w/x)**:

```
chmod g+w niyom.txt      # দলকে (g) লেখার অনুমতি দাও (+w)
chmod o-r niyom.txt      # বাকিদের (o) থেকে পড়ার অনুমতি কাড়ো (-r)
chmod a+r niyom.txt      # সবাইকে (a = all) পড়ার অনুমতি
```

প্রতিটার পর `ls -l niyom.txt` চালিয়ে দেখো নিয়মের লাইনটা কেমন বদলাচ্ছে — নিজের চোখে দেখলে গেঁথে যাবে।

সবচেয়ে বাস্তব ব্যবহারটা হলো **script চালানোর অনুমতি**। পরের পর্বে যখন নিজের প্রথম script লিখবে, সেটা চালানোর আগে লাগবে:

```
chmod +x amar-script.sh
```

`+x` = execute অনুমতি। নতুন বানানো ফাইল by default চালানো যায় না — এটা নিরাপত্তা: কোনো ফাইল **প্রোগ্রাম হিসেবে চলবে** কিনা সেটা সচেতনভাবে তোমাকে বলে দিতে হয়।

Licensed to sakilorb@gmail.com · Do not redistribute

> 💡 ইন্টারনেটে দেখবে অনেকে সংখ্যা দিয়ে লেখে — `chmod 755`, `chmod 644`। এটা একই জিনিসের সংখ্যায় লেখা রূপ: r=4, w=2, x=1 যোগ করে প্রতি দলের জন্য এক অঙ্ক। যেমন 7=4+2+1=rwx, 5=4+1=r-x, তাই 755 = `rwxr-xr-x`। এখনই মুখস্থ করতে হবে না — শুধু চিনে রাখো, যাতে দেখলে ভয় না লাগে।

> ⚠️ আর কোথাও `chmod 777` (সবাইকে সব অনুমতি) করার পরামর্শ দেখলে সাবধান — এটা "সমস্যা সমাধানের" নামে বাড়ির সব দরজা-জানালা খুলে রাখা। কাজ হয়তো চলবে, কিন্তু নিরাপত্তা শূন্য। প্রায় সবসময় এর চেয়ে ভালো সমাধান আছে।

মালিকানা বদল — `chown`

নিয়ম বদলানো আর **মালিক** বদলানো আলাদা জিনিস। মালিকানা বদলায় `chown` (**ch**ange **own**er):

```
sudo chown karim niyom.txt          # মালিক এখন karim
sudo chown karim:karim niyom.txt    # মালিক ও দল দুটোই
```

খেয়াল করো `sudo` লাগলো — কারণ মালিকানা হস্তান্তর মালিকের চেয়েও বড় সিদ্ধান্ত, এটা শুধু **বাড়িওয়ালা** পারে। কে এই বাড়িওয়ালা? এবার তার গল্প।

`sudo` — মালিকের ক্ষমতা, এবং তার দায়

> 🐾 **দৃশ্য ২০: সিন্দুকের চাবি**

Licensed to sakilorb@gmail.com · Do not redistribute

তোমার বাসায় একটা সিন্দুক আছে — ভেতরে দলিল, গয়না, জরুরি কাগজ। চাবিটা সারাদিন পকেটে নিয়ে ঘোরো? না — দরকারের সময় বের করো, কাজ সেরে তুলে রাখো। কারণ চাবি সবসময় হাতে থাকলে ভুলের সম্ভাবনাও সবসময় থাকে।

Linux-এ প্রতিটা সিস্টেমের একজন **সর্বময় কর্তা** আছে — নাম **root** (অধ্যায় ৪-এর `/root` ঘরটা মনে আছে? এই সেই কর্তার ঘর)। root-এর কাছে **কোনো permission-ই বাধা না** — যেকোনো ফাইল পড়া, বদলানো, মোছা — সব পারে। এমনকি ভুল করে পুরো সিস্টেম মুছে ফেলাও। 😨

তাহলে root হিসেবেই সব কাজ করলে হয় না? **না — আর এটাই Linux-এর জ্ঞানের জায়গা।** তুমি সারাদিন কাজ করো সাধারণ ব্যবহারকারী হিসেবে (চাবি সিন্দুকে), আর যখন সত্যিই বড় কাজ দরকার — সফটওয়্যার ইনস্টল, সিস্টেম ফাইল সম্পাদনা — তখন **এক কমান্ডের জন্য** চাবি বের করো:

```
sudo apt install htop
```

`sudo` = **s**uper**u**ser **do** — "কর্তার ক্ষমতায় এই কাজটা করো।" কাজ শেষ, ক্ষমতাও শেষ। পরের কমান্ড আবার সাধারণ তুমি।

এই নকশার সৌন্দর্যটা দেখো: তোমার প্রতিটা সাধারণ মুহূর্তে ভুল করার ক্ষমতাও সাধারণ। `rm -rf` দিয়ে ভুল করলেও নিজের ঘরের বাইরে ক্ষতি করতে পারবে না — সিস্টেম বেঁচে যাবে। ক্ষমতা যত কম সময় হাতে, দুর্ঘটনা তত কম। (নিরাপত্তার জগতে এর গালভরা নামও আছে: principle of least privilege।)

> ⚠️ তাই উল্টো অভ্যাসটাও গেঁথে নাও: **কোনো কমান্ড কাজ না করলেই সামনে `sudo` বসিয়ে দেওয়া — না।** আগে ভাবো: এই কাজে কর্তার ক্ষমতা লাগার কথা কি? `Permission denied` অনেক সময় সিস্টেমের ভুল না — সিস্টেম তোমাকে **রক্ষা** করছে। আর ইন্টারনেট

Licensed to sakilorb@gmail.com · Do not redistribute

> থেকে কপি করা কমান্ডে `sudo` থাকলে চালানোর আগে অন্তত বুঝে নাও কমান্ডটা কী করে — চাবিটা তো তুমিই দিচ্ছো।

User আর Group — এক নজরে

শেষ টুকরা। তুমি ছাড়াও সিস্টেমে আরো user আছে — দেখো:

```
cat /etc/passwd | cut -d ':' -f 1
```

(দেখলে? গত অধ্যায়ের `cut` এখানেই কাজে লেগে গেলো! `/etc/passwd` ফাইলে প্রতি লাইনে এক user, তথ্যগুলো `:` দিয়ে ভাগ করা — আমরা শুধু প্রথম টুকরা অর্থাৎ নামটা কাটলাম।)

অনেক অচেনা নাম দেখবে — এরা বেশিরভাগই **system user**: বিভিন্ন সফটওয়্যারের নিজস্ব "কর্মচারী", মানুষ না। আর তুমি কোন কোন দলে আছো:

```
groups
```

Group জিনিসটার আসল ক্ষমতা টের পাবে team-এ কাজ করার সময় — "developers দলের সবাই এই ফোল্ডারে লিখতে পারবে" — এক নিয়মে দশজনের ব্যবস্থা। আপাতত ধারণাটুকু থাকলেই চলবে।

🛠️ হাতেকলমে

1. `gopon.txt` নামে ফাইল বানাও, তাতে একটা গোপন কথা লেখো (`echo` আর `>` তো জানোই)
2. `ls -l gopon.txt` — নিয়মের লাইনটা জোরে জোরে পড়ো: কে কী পারে?

Licensed to sakilorb@gmail.com · Do not redistribute

3. `chmod o-r gopon.txt` দিয়ে বাকিদের পড়া বন্ধ করো — `ls -l`-এ পার্থক্য দেখো
4. নিজের থেকেই পড়ার অনুমতি কেড়ে নাও: `chmod u-r gopon.txt` — এবার `cat gopon.txt` চালাও। কী হলো? 😄 (নিজের বানানো তালা নিজের দরজায়!) `chmod u+r` দিয়ে ফেরত নাও
5. `sudo ls /root` চালাও — বাড়িওয়ালার ঘরে উঁকি। এবার `sudo` ছাড়া চালিয়ে দেখো পার্থক্যটা
6. ভাবনার প্রশ্ন (উত্তর লিখতে হবে না): `/etc/passwd` সবাই পড়তে পারে (`ls -l /etc/passwd` দেখো) — কিন্তু বদলাতে পারে শুধু root। কেন এই নকশা ঠিকঠাক?

📜 হুকুমনামা

কমান্ড	কাজ
`ls -l`	নিয়মকানুন দেখা (`rwx` পড়তে জানো এখন!)
`chmod u/g/o/a +/- r/w/x ফাইল`	অনুমতি দেওয়া-কাড়া
`chmod +x ফাইল`	চালানোর অনুমতি (script-এর জন্য)
`sudo chown user:group ফাইল`	মালিকানা বদল
`sudo কমান্ড`	কর্তার ক্ষমতায় এক কমান্ড
`groups`	আমি কোন কোন দলে

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১১: লিনুর স্বাস্থ্য পরীক্ষা

> 🐾 **দৃশ্য ২১: লিনু আজ ঝিমাচ্ছে**

আজ লিনুর কী যেন হয়েছে। খেলছে না, ডাকলে আসছে না, কোণায় বসে ঝিমাচ্ছে। তুমি চিন্তায় পড়লে — জ্বর? পেট খারাপ? নাকি রাতে ঘুম হয়নি?

কম্পিউটারও ঝিমায়। "ল্যাপটপ হ্যাং হয়ে গেছে", "সব স্লো চলছে", "ফ্যান বিমানের মতো ঘুরছে" — চেনা অভিযোগ, তাই না? পার্থক্য হলো: লিনুকে ডাক্তারের কাছে নিতে হয়, আর কম্পিউটারের ডাক্তার **তুমি নিজেই হতে পারো।** আজ শিখবো লিনুর full health checkup।

Process — লিনুর শরীরের ভেতরে যা চলছে

প্রথমে একটা ধারণা। তোমার কম্পিউটারে এই মুহূর্তে **শত শত প্রোগ্রাম একসাথে চলছে** — ব্রাউজার, টার্মিনাল, ঘড়ি, নেটওয়ার্কের দেখভাল করা প্রোগ্রাম, আরো কত কী। প্রতিটা **চলন্ত প্রোগ্রামকে** বলে একটা **process**।

ফাইল আর process-এর পার্থক্যটা ধরো এভাবে: ফাইল হলো ঘুমন্ত অবস্থা, process হলো জেগে ওঠা অবস্থা। `/usr/bin/htop` একটা ফাইল — ডিস্কে শুয়ে আছে। তুমি `htop` চালালে সেটা জেগে উঠে process হয়ে যায়। একই প্রোগ্রাম দুইবার চালালে দুটো আলাদা process।

আর প্রতিটা process-এর একটা পরিচয়পত্র নম্বর আছে — **PID** (Process ID)। এটা লাগবে একটু পরেই।

Licensed to sakilorb@gmail.com · Do not redistribute

`ps` — এই মুহূর্তের ছবি

```
ps aux
```

সব process-এর তালিকা — কে চালিয়েছে (USER), পরিচয় নম্বর (PID), কত CPU আর মেমোরি খাচ্ছে (%CPU, %MEM), আর কমান্ডটা কী (COMMAND)। তালিকা বিশাল — কিন্তু তুমি এখন জানো বিশাল output কীভাবে সামলাতে হয়:

```
ps aux | less            # পাতা উল্টে দেখো
ps aux | grep firefox    # নির্দিষ্ট কাউকে খোঁজো
ps aux | wc -l           # মোট কয়টা process চলছে
```

দেখলে? গত পর্বের pipe এখন তোমার প্রতিটা কাজে আপনাআপনি ঢুকে যাচ্ছে। এটাই দক্ষতা তৈরি হওয়ার লক্ষণ। 😊

`top` আর `htop` — চলন্ত ড্যাশবোর্ড

`ps` হলো স্থিরচিত্র — এক মুহূর্তের ছবি। আর **live দেখতে:**

```
top
```

প্রতি কয়েক সেকেন্ডে নিজে নিজে হালনাগাদ হয় — কোন process এখন কত খাচ্ছে, CPU-র চাপ কত। `q` দিয়ে বের হও।

কিন্তু `top`-এর চেহারাটা একটু রুক্ষ। মনে আছে অধ্যায় ৭-এ `htop` ইনস্টল করেছিলে? এবার তার আসল ব্যবহার:

```
htop
```


Licensed to sakilorb@gmail.com · Do not redistribute

রঙিন বার, সাজানো তালিকা, মাউসও কাজ করে! যা যা দেখবে:

- **ওপরের বারগুলো** — প্রতিটা CPU core-এর ব্যস্ততা আর মেমোরির দখল, live
- **তালিকা** — সবচেয়ে খিদে-পাওয়া process গুলো ওপরে
- **F6** চেপে সাজানোর নিয়ম বদলাও (CPU নাকি মেমোরি অনুযায়ী), **F4** দিয়ে ছাঁকো, **F9** দিয়ে… সেটা পরের অংশে 😈

> 💡 "কম্পিউটার স্লো লাগছে" — এখন থেকে তোমার প্রথম কাজ: `htop` খোলো, দেখো ওপরে কে বসে আছে। ৯০% ক্ষেত্রে অপরাধী এক নজরেই ধরা পড়বে — হয় ব্রাউজারের কোনো ট্যাব মেমোরি গিলছে, নয়তো কোনো process CPU আঁকড়ে বসে আছে।

`kill` — বেয়াড়া process থামানো

> 🐾 **দৃশ্য ২২: পাগলা দৌড়**

মাঝে মাঝে লিনুর "পাগলা মুহূর্ত" আসে — কারণ ছাড়া ঘরময় ঝড়ের বেগে দৌড়ায়, পর্দায় ঝোলে, কিছুতেই থামে না। বিড়ালওয়ালারা মাত্রই চেনে। 😹

Process-দেরও এটা হয় — কোনো প্রোগ্রাম আটকে যায় (hang), বা পাগলের মতো CPU খেতে থাকে, বন্ধের বোতামেও কাজ হয় না। তখন দরকার `kill`।

নামটা যত ভয়ংকর, ব্যাপারটা তত না — `kill` আসলে process-কে **বার্তা (signal) পাঠায়:**

Licensed to sakilorb@gmail.com · Do not redistribute

```
kill 12345
# PID 12345-কে ভদ্র অনুরোধ: "গুছিয়ে বন্ধ হও" (signal 15, TERM)
kill -9 12345       # শেষ অস্ত্র: "এক্ষুনি, বিনা প্রশ্নে, শেষ" (signal 9, KILL)
```

নিয়মটা ভদ্রতার: **আগে সাধারণ** `kill` — process নিজের কাজ গুছিয়ে, ফাইল সেভ করে বন্ধ হওয়ার সুযোগ পায়। সে **তাতেও কান না দিলে** তবেই `-9` — এটা process-এর কাছে পৌঁছায়ই না, সরাসরি kernel তাকে তুলে নেয়। কোনো সেভ, কোনো গোছানো — কিছুই হয় না। তাই `-9` হলো শেষ ভরসা, প্রথম পছন্দ না।

PID খুঁজে বের করে মারার পুরো চেইনটা:

```
ps aux | grep firefox     # PID দেখো (ধরো 4521)
kill 4521
```

নাম জানলে শর্টকাটও আছে:

```
killall firefox           # এই নামের সব process-কে
```

আর `htop`-এর ভেতরে তো আরো আরাম — তীর চিহ্নে process বাছো, **F9** চাপো, signal বাছো, Enter। এজন্যই `htop` সবার এত প্রিয়।

`df` আর `du` — পেট কতটা ভরা

Process গেলো — এবার **জায়গার** হিসাব। দুটো কমান্ড, প্রায় একই নাম, কাজ আলাদা দিক থেকে:

`df` **(disk free)** — **পুরো ডিস্কের** হিসাব, বড় ছবি:

```
df -h
```


Licensed to sakilorb@gmail.com · Do not redistribute

`-h` মানে **h**uman-readable — সংখ্যাগুলো GB/MB-তে, মানুষের ভাষায় (নইলে দেখাবে block সংখ্যায় — রোবটের ভাষায় 🤖)। Output-এ দেখো `/`-এর লাইনটা — Size, Used, Avail, Use% — তোমার মূল ডিস্ক কতটা ভরলো।

`du` **(disk usage) — নির্দিষ্ট জায়গা** কতটা খাচ্ছে, ছোট ছবি:

```
du -sh ~/Downloads        # এই ফোল্ডারটা মোট কত বড় (-s = সারাংশ)
du -h ~ | sort -rh | head -n 10
```

দ্বিতীয়টা পড়ে দেখো — চেনা চেনা লাগছে না? `du` সব ফোল্ডারের মাপ বললো, `sort -rh` বড় থেকে ছোটে সাজালো (`-h` এখানে human-readable সংখ্যা বুঝে সাজায়), `head` সেরা দশ দেখালো। **"ডিস্ক ভরে গেছে, কে খেলো?"-র এক লাইনের উত্তর।** আবারও সেই লেগো দর্শন। 😄

`free` **— মেমোরির (RAM) হিসাব:**

```
free -h
```

> 💡 এখানে একটা জিনিস নতুনদের প্রায়ই ঘাবড়ে দেয়: `free`-তে দেখবে available মেমোরি কম, অনেকখানি "buff/cache"-এ। **এটা সমস্যা না** — Linux খালি RAM-কে অপচয় মনে করে, তাই ফাঁকা জায়গায় ঘনঘন-লাগা ফাইল আগে থেকেই তুলে রাখে (cache), যাতে সব দ্রুত চলে। কোনো প্রোগ্রামের দরকার হলে সে জায়গা সাথে সাথেই ছেড়ে দেয়। বিড়াল যেমন সব ফাঁকা বাক্স দখল করে বসে থাকে, কিন্তু দরকারে জায়গা ছাড়ে — Linux-ও তাই। 📦🐱

Licensed to sakilorb@gmail.com · Do not redistribute

পেছনের কাজ — `&`, `jobs`, `fg`

শেষ কৌশল। কিছু কমান্ড অনেকক্ষণ চলে — বড় ফাইল কপি, ডাউনলোড। এতক্ষণ তুমি বসে থাকতে টার্মিনাল আটকে রেখে। আর না — কমান্ডের শেষে `&` বসাও:

```
sleep 60 &
```

(`sleep 60` = "৬০ সেকেন্ড শুধু বসে থাকো" — মহড়ার জন্য perfect কমান্ড।) Output:

```
[1] 8734
```

মানে: কাজ নম্বর `[1]`, PID `8734` — **পেছনে (background) চলছে**, আর তোমার prompt ফেরত! তুমি এখন অন্য কাজ করো, ও পেছনে চলুক।

```
jobs          # পেছনে কী কী চলছে
fg            # সর্বশেষটাকে সামনে (foreground) ফেরত আনো
```

আরেকটা চাল: চলন্ত কমান্ডে `Ctrl+Z` চাপলে সেটা **থেমে** পেছনে যায় — `bg` লিখলে পেছনেই আবার চলতে শুরু করে, `fg` লিখলে সামনে ফেরে।

আর `nohup` (**no h**ang**up**) — টার্মিনাল বন্ধ করলে সাধারণত তার পেছনের কাজগুলোও মরে যায়; `nohup কমান্ড &` দিয়ে চালালে টার্মিনাল বন্ধ হলেও কাজ চলতে থাকে। এখন শুধু নামটা চিনে রাখো — পর্ব ৬-এ server-এ কাজ চালিয়ে রাখার সময় এটা ফিরে আসবে।

🛠️ হাতেকলমে: লিনুর ডাক্তার তুমি

1. `htop` খোলো — তোমার সিস্টেমে এখন সবচেয়ে বেশি মেমোরি কে খাচ্ছে? CPU কে?

Licensed to sakilorb@gmail.com · Do not redistribute

2. আরেকটা টার্মিনালে `sleep 300 &` চালাও, তারপর `htop`-এ **F4** চেপে "sleep" লিখে খুঁজে বের করো — এরপর **F9** Enter দিয়ে ওকে বিদায় করো। অভিনন্দন, প্রথম শিকার! 😈
3. `df -h` — তোমার মূল ডিস্ক কত % ভরা?
4. এক লাইনে বের করো তোমার home-এর সবচেয়ে মোটা ৫টা ফোল্ডার (এই অধ্যায়েই চেইনটা আছে — না দেখে লেখার চেষ্টা করো)
5. `sleep 100 &` চালিয়ে `jobs` দেখো, `fg` দিয়ে সামনে আনো, `Ctrl+C` দিয়ে থামাও — পুরো জীবনচক্র এক process-এর!
6. `free -h` চালাও — buff/cache-এ কত আছে? (আর মনে করো: ওটা বাক্স-দখল-করা বিড়াল, সমস্যা না 📦)

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড	কাজ
`ps aux`	সব process-এর স্থিরচিত্র
`top` / `htop`	live ড্যাশবোর্ড(`htop` -ই ভালো)
`kill PID`	ভদ্র অনুরোধে বন্ধ
`kill -9 PID`	শেষ অস্ত্র — বিনা প্রশ্নে শেষ
`killall নাম`	নাম ধরে সবাইকে
`df -h`	পুরো ডিস্কের হিসাব
`du -sh জায়গা`	এই জায়গাটা কত বড়
`free -h`	RAM-এর হিসাব
`কমান্ড &`	পেছনে চালাও
`jobs` / `fg` / `bg`	পেছনের কাজ দেখা/সামনে আনা/আবার চালানো

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১২: Environment ও Shell Customization

> 🐾 **দৃশ্য ২৩: লিনুর অভ্যাস**

ছয় মাস হয়ে গেলো লিনু তোমার সাথে আছে। আর মজার ব্যাপারটা খেয়াল করেছো? **ও এখন তোমার মতো হয়ে গেছে।** তুমি যখন ঘুমাও, ও ঘুমায়। তোমার চেয়ারের পাশেই ওর প্রিয় জায়গা। এমনকি তোমার কি-বোর্ডের শব্দ শুনলেই ও বোঝে — কাজ চলছে, বিরক্ত করা যাবে না।

বিড়াল ধীরে ধীরে মালিকের **অভ্যাসে** ঢুকে যায়। আজকের অধ্যায়ে আমরা উল্টোটা করবো — **টার্মিনালকে তোমার অভ্যাসে ঢোকাবো।** এতদিন তুমি Linux-এর নিয়মে চলেছো; আজ থেকে Linux চলবে তোমার নিয়মে।

Environment Variable — লিনুর মনে রাখা তথ্য

লিনু কিছু জিনিস সবসময় মনে রাখে — খাবারের বাটি কোথায়, তুমি কে, বাড়ি কোনটা। Shell-ও ঠিক তাই — কিছু তথ্য সবসময় মনে রাখে, নাম **environment variable**। দেখো:

```
echo $HOME
echo $USER
echo $SHELL
```

Output: তোমার home-এর ঠিকানা, তোমার নাম, তোমার দোভাষী কে (মনে আছে অধ্যায় ৩? সম্ভবত `/bin/bash`)।

Licensed to sakilorb@gmail.com · Do not redistribute

আর অধ্যায় ২-এর হাতেকলমে-র সেই রহস্য — `echo $USER` চালিয়ে "লিনু কি তোমাকে চিনে ফেলেছে?" — এতদিনে উত্তর পেলে। 😄 নিয়মটা হলো: `$` **চিহ্ন মানে "এই নামের variable-এর ভেতরের মানটা দাও।"** সব variable একসাথে দেখতে: `env`।

নিজের variable-ও বানানো যায়:

```
PRIYO_RONG="teal"
echo $PRIYO_RONG
```

(`=`-এর দুপাশে স্পেস **দেওয়া যাবে না** — bash-এর জেদি নিয়ম।) এটা কাজে লাগে configuration-এ — যেমন প্রোগ্রামকে গোপন তথ্য (API key) দেওয়ার প্রচলিত উপায়ই হলো environment variable। পরে যখন কোনো প্রজেক্টের README-তে দেখবে `export API_KEY=...` — চিনে ফেলবে।

`$PATH` রহস্য — লিনু কমান্ড খুঁজে পায় কীভাবে?

সব variable-এর সেরা রহস্যটা এবার। কখনো ভেবেছো — তুমি `ls` লেখো, আর shell **কীভাবে জানে** `ls` জিনিসটা কোথায় আছে? (অধ্যায় ৬-এ `which ls` দিয়ে দেখেছিলে — `/usr/bin/ls`-এ থাকে। কিন্তু shell জানলো কী করে যে ওখানে খুঁজতে হবে?)

উত্তর:

```
echo $PATH
```

Output অনেকটা এরকম:

```
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:...
```


Licensed to sakilorb@gmail.com · Do not redistribute

এটা হলো shell-এর **খোঁজার তালিকা** — `:` দিয়ে আলাদা করা কতগুলো directory। তুমি যেকোনো কমান্ড লেখা মাত্র shell এই তালিকা ধরে **বাঁ থেকে ডানে** প্রতিটা ঘরে খোঁজে: "এই নামের প্রোগ্রাম আছে? আছে? আছে?" — প্রথম যেখানে পায়, সেটাই চালায়। কোথাও না পেলে? সেই চেনা `command not found`।

এখন অনেক পুরনো ধাঁধার উত্তর মিলবে — কখনো কোনো টুল ইনস্টল করার পরেও `command not found` পেলে ৯০% সম্ভাবনা: **প্রোগ্রামটা এমন জায়গায় বসেছে যেটা `$PATH`-এর তালিকায় নেই।** সমাধান হয় জায়গাটা তালিকায় যোগ করা:

```
export PATH="$PATH:/notun/jayga"
```

(পড়ো: PATH-এর নতুন মান = পুরনো PATH + `:` + নতুন জায়গা। `export` মানে — শুধু এই shell না, এর থেকে জন্ম নেওয়া process-রাও যেন জানে।)

কিন্তু একটা সমস্যা আছে… এভাবে বানানো variable, alias — সব **টার্মিনাল বন্ধ করলেই হাওয়া।** স্থায়ী করার জায়গাটা কোথায়? এবার এই অধ্যায়ের — আসলে এই পর্বের — সেরা জিনিস।

`.bashrc` — লিনুর অভ্যাসের খাতা

মনে আছে পর্ব ২-এ সোফার নিচে লুকানো ফাইলদের? তাদের মধ্যে একজনের নাম ছিলো `.bashrc` — আর আমি বলেছিলাম, "ওটা দিয়ে লিনুর অভ্যাস বদলানো যায়।" সময় এসেছে।

`.bashrc` **হলো এমন একটা script, যেটা প্রতিবার নতুন টার্মিনাল খোলার সময় bash নিজে নিজে চালায়।** মানে — এই ফাইলে তুমি যা লিখে রাখবে, প্রতিটা নতুন টার্মিনালে সেটা **আপনাআপনি ঘটবে।** এটাই অভ্যাসের খাতা: একবার লেখো, চিরকাল চলে।

খুলে দেখো (ভয় নেই, শুধু দেখা):

Licensed to sakilorb@gmail.com · Do not redistribute

```
nano ~/.bashrc
```

আগে থেকেই অনেক কিছু লেখা — Ubuntu-র দেওয়া default অভ্যাস। **কিছু মুছো না** — আমরা শুধু একদম শেষে নিজেরটা যোগ করবো।

Alias — নিজের বানানো হুকুম

`.bashrc`-তে সবচেয়ে বেশি যেটা লেখা হয়: **alias** — লম্বা কমান্ডের নিজের-দেওয়া ডাকনাম।

ফাইলের একদম শেষে যোগ করো:

```
# ===== amar nijer hukum =====
alias ll='ls -la'
alias ..='cd ..'
alias taja='sudo apt update && sudo apt upgrade'
alias mukhosto='history | grep'
```

দেখো কী বানালাম:

- `ll` → `ls -la` — দুই অক্ষরে বিস্তারিত তালিকা (এত প্রচলিত যে অনেক সিস্টেমে আগেই বানানো থাকে)
- `..` → `cd ..` — এক ধাপ ওপরে, চার অক্ষর বাঁচলো
- `taja` → পুরো আপডেট-জোড়া এক শব্দে! অধ্যায় ৭-এর "সপ্তাহে একবার" অভ্যাসটা এখন `taja` লেখা যত সহজ
- `mukhosto` → `mukhosto apt` লিখলেই ইতিহাস থেকে apt-ওয়ালা সব কমান্ড হাজির

Licensed to sakilorb@gmail.com · Do not redistribute

সেভ করে বের হও (`Ctrl+O` , `Ctrl+X` — nano তো এখন পুরনো বন্ধু)। কিন্তু চালিয়ে দেখো — `ll` **কাজ করছে না!** কেন?

কারণ `.bashrc` পড়া হয় **টার্মিনাল খোলার সময়** — আর এই টার্মিনাল তো খোলা হয়েছিলো তোমার লেখার আগে। দুটো সমাধান: নতুন টার্মিনাল খোলো, অথবা এই টার্মিনালকেই খাতাটা আবার পড়াও:

```
source ~/.bashrc
```

এবার `ll` চালাও — কাজ করছে! 🎉 আর এখন থেকে প্রতিটা নতুন টার্মিনালে তোমার হুকুমগুলো জন্ম থেকেই তৈরি থাকবে।

> 💡 এই মুহূর্তটার আলাদা গুরুত্ব আছে: **এতদিন তুমি Linux-এর দেওয়া কমান্ড শিখছিলে — আজ প্রথম নিজের কমান্ড বানালে।** ব্যবহারকারী থেকে নির্মাতা হওয়ার প্রথম পা। আর পরের পর্বে এই পা-টাই দৌড় হয়ে যাবে — আস্ত script লিখবে।

History আর গতির কৌশল

শেষে কয়েকটা কৌশল, যেগুলো জানা আর না-জানায় টার্মিনালে গতির আকাশ-পাতাল তফাত হয়:

`history` — তোমার সব চালানো কমান্ডের খাতা (অধ্যায় ৯-এর হাতেকলমে-তে দেখা হয়েছেই)।

⬆️ **ওপরের তীর** — আগের কমান্ডগুলো একে একে ফেরত আসে। ছোট্ট জিনিস, দিনে একশোবার লাগে।

Licensed to sakilorb@gmail.com · Do not redistribute

`Ctrl+R` — ইতিহাসে **উল্টো খোঁজ**। চাপো, তারপর লিখতে শুরু করো (যেমন `apt`) — সাথে সাথে ইতিহাস থেকে মিলে যাওয়া সর্বশেষ কমান্ডটা হাজির। আবার `Ctrl+R` চাপলে তার আগেরটা। Enter-এ চালাও, `Esc` -এ শুধু নিয়ে এসো। **এটা শিখলে টার্মিনাল-জীবন বদলে যায়** — লম্বা কমান্ড দ্বিতীয়বার টাইপ করা মানুষ আর তুমি — দুই প্রজাতি। 😄

`!!` — ঠিক আগের কমান্ডটা আবার। এর সেরা ব্যবহার — `sudo` দিতে ভুলে গেছো? পুরোটা আবার না লিখে:

```
apt install tree        # Permission denied... ধুর, sudo ভুলে গেছি
sudo !!                 # মানে: sudo দিয়ে আগের কমান্ডটাই — চলে গেলো!
```

আর **Tab** — সে তো অধ্যায় ৪ থেকেই তোমার সবচেয়ে ঘনিষ্ঠ বন্ধু, নাকি? না হয়ে থাকলে আজ থেকে জোর করে অভ্যাস করো: **পুরো নাম টাইপ করা নিষেধ।** 😄

🛠️ হাতেকলমে: টার্মিনালটা নিজের করে নাও

1. `echo $PATH` চালাও — তালিকায় কয়টা directory আছে গুনে বলো (hint: পর্ব ৪-এর ছাত্র তুমি — `echo $PATH | tr ':' '\n' | wc -l` — নতুন অতিথি `tr` কে অনুমান করো তো কী করলো!)
2. ওপরের চারটা alias `.bashrc`-তে যোগ করো, `source` করো, প্রতিটা চালিয়ে দেখো
3. নিজের একটা alias বানাও — নিজের দরকারে, নিজের নামে। (আইডিয়া: `alias porbo='cd ~/projects'` ?)
4. `Ctrl+R` চেপে `htop` খোঁজো — ইতিহাস থেকে তুলে এনে চালাও, একটা অক্ষরও নতুন করে টাইপ না করে

Licensed to sakilorb@gmail.com · Do not redistribute

5. ইচ্ছা করে `sudo` ছাড়া `apt install cowsay` চালাও, তারপর `sudo !!` — জাদুটা নিজের হাতে
6. Bonus: `.bashrc`-র একদম শেষে যোগ করো `echo "miyaw! 🐾"` — সেভ করে **নতুন টার্মিনাল** খোলো। লিনু এখন প্রতিবার তোমাকে সালাম দেবে 🐱

📜 হুকুমনামা

কমান্ড	কাজ
`echo $VARIABLE`	variable-এর মান দেখো
`env`	সব environment variable
`export NAME="মান"`	variable বানাও (child process-সহ)
`$PATH`	কমান্ড খোঁজার তালিকা
`alias নাম='কমান্ড'`	নিজের হুকুম বানাও
`~/.bashrc`	অভ্যাসের খাতা — প্রতি টার্মিনালে চলে
`source ~/.bashrc`	খাতাটা এখনই আবার পড়ো
`Ctrl+R`	ইতিহাসে উল্টো খোঁজ
`sudo !!`	আগের কমান্ড, এবার sudo-সহ

🐾 **পর্ব ৪ শেষ।** থামো একটু — পেছনে তাকাও। এই পর্বে তুমি যা যা করেছো: কমান্ড জুড়ে data analysis (pipe), ফাইলের গায়ের নিয়ম পড়া আর বদলানো (permission), সিস্টেমের ডাক্তারি (process), আর টার্মিনালকে নিজের অভ্যাসে গড়ে নেওয়া (`.bashrc`)। বই শুরুর সেই কালো-স্ক্রিন-দেখে-ঘাবড়ানো মানুষটার সাথে আজকের তোমার তফাতটা মাপো তো।

তুমি এখন আর সাধারণ ব্যবহারকারী নও — তুমি **power user**।

Licensed to sakilorb@gmail.com · Do not redistribute

কিন্তু একটা জিনিস এখনো বাকি। এতদিন প্রতিটা হুকুম তুমি **নিজ হাতে** দিয়েছো। পরের পর্বে লিনুকে শেখাবো **নিজে নিজে কাজ করা** — আস্ত script লিখবো, আর cron দিয়ে ঠিক করে দেবো কখন কোন কাজ আপনাআপনি হবে। রাত ৩টায় তুমি ঘুমাবে, লিনু জেগে backup নেবে। 😴🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পর্ব ৫: লিনুকে প্রশিক্ষণ (Scripting & Automation)

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১৩: প্রথম Bash Script

> 🐾 **দৃশ্য ২৪: নতুন খেলা শেখানো**

লিনুকে তুমি এখন একটা নতুন জিনিস শেখাচ্ছো। আগে যেমন ছিলো — "বসো" বললে বসে, "বল আন" বললে আনে। এক হুকুমে এক কাজ। কিন্তু আজ তুমি শেখাচ্ছো একটা **আস্ত খেলা**: "লিনু, সকাল-রুটিন!" — আর ও একে একে করে ফেলে: বল আনে □ ঝুড়িতে রাখে □ খাবারের বাটির পাশে বসে □ লেজ নাড়ে।

চারটা কাজ, একটা নামে। একবার শিখিয়ে দিলে যতবার খুশি বলো — প্রতিবার নিখুঁতভাবে করবে, একটা ধাপও ভুলবে না।

এটাই **script**: কতগুলো কমান্ড এক ফাইলে সাজিয়ে রাখা, যাতে এক ডাকে পুরোটা চলে। আর সত্যি বলতে — তুমি script-এর সাথে আগেই দেখা করেছো, টেরও পাওনি। `.bashrc` মনে আছে? আমি বলেছিলাম, "এটা এমন একটা script, যেটা প্রতিবার টার্মিনাল খোলার সময় bash নিজে চালায়।" আজ আমরা নিজেরাই ওরকম জিনিস বানাবো।

প্রথম script — তিন লাইনে

কাজের জায়গা বানিয়ে শুরু:

```
mkdir -p ~/scripts
cd ~/scripts
nano prothom.sh
```

ভেতরে লেখো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
#!/bin/bash
echo "miyaw! ami linu"
echo "ajke amar prothom script cholche"
```

সেভ করে বের হও (`Ctrl+O` , `Ctrl+X`)। এবার দুটো জিনিস বুঝি:

প্রথম লাইন `#!/bin/bash` — এটার নাম **shebang** (শি-ব্যাং! নামটাই মজার 😄)। এই লাইনটা সিস্টেমকে বলে: "এই ফাইলটা কোন দোভাষী দিয়ে পড়াতে হবে" — আমাদের ক্ষেত্রে `/bin/bash` । প্রতিটা bash script-এর **প্রথম লাইনে** এটা থাকবে — এটা নিয়ম না, রীতি — কিন্তু এমন রীতি যেটা না মানলে পরে ভুগতে হয়।

`.sh` **extension** — বোঝায় এটা shell script। Linux-এ extension আসলে বাধ্যতামূলক না (Windows-এর মতো না), কিন্তু মানুষ আর এডিটরদের চেনার সুবিধার জন্য দেওয়া রীতি।

এবার চালাই:

```
./prothom.sh
```

Output:

```
bash: ./prothom.sh: Permission denied
```

থামো — **ঘাবড়িয়ো না।** এক সেকেন্ড ভাবো... এটা তো তোমার **চেনা** সমস্যা! অধ্যায় ১০-এ শিখেছিলে: "নতুন বানানো ফাইল by default চালানো যায় না — কোনো ফাইল প্রোগ্রাম হিসেবে চলবে কিনা সেটা সচেতনভাবে তোমাকে বলে দিতে হয়।" আর ওষুধটাও শেখা আছে:

```
chmod +x prothom.sh
./prothom.sh
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
miyaw! ami linu
ajke amar prothom script cholche
```

🎉 **তোমার প্রথম প্রোগ্রাম চললো!** হ্যাঁ, প্রোগ্রামই — যে ফাইল কম্পিউটারকে দিয়ে কাজ করায়, সেটাই প্রোগ্রাম। তুমি এখন যেই দলে, সেই দলের নাম প্রোগ্রামার।

> 💡 শুরুর `./` -টা কেন লাগলো? অধ্যায় ১২-এর `$PATH` মনে করো — কমান্ড লিখলে shell শুধু PATH-এর তালিকাভুক্ত ঘরগুলোতে খোঁজে। `~/scripts` তো সেই তালিকায় নেই! `./prothom.sh` মানে — "তালিকা-টালিকা বাদ, **এই ঘরের** এই ফাইলটা চালাও।" (চাইলে `~/scripts` -কে PATH-এ যোগও করতে পারো — কীভাবে, সেটাও অধ্যায় ১২-তেই শেখা আছে। দেখেছো, সব জোড়া লেগে যাচ্ছে? 😄)

Variable — script-এর মনে রাখার ক্ষমতা

অধ্যায় ১২-তে টার্মিনালে variable বানিয়েছিলে — script-এও হুবহু একই নিয়ম (কারণ script মানেই তো টার্মিনালের কমান্ডগুলো ফাইলে রাখা!):

```
nano biral.sh
```

```
#!/bin/bash
NAM="linu"
BOYOSH=2

echo "amar biral er nam $NAM"
echo "$NAM er boyosh $BOYOSH bochor"
```


Licensed to sakilorb@gmail.com · Do not redistribute

`chmod +x biral.sh` করে চালাও। নিয়ম দুটো আবার মনে করিয়ে দিই:

- `=`-এর দুপাশে **স্পেস নেই** (`NAM = "linu"` লিখলেই error — bash ভাববে `NAM` একটা কমান্ড!)
- ব্যবহারের সময় সামনে `$`

Input নেওয়া — `$1`, `$2`: হুকুমের সাথে জিনিস ধরিয়ে দেওয়া

🐾 লিনুকে তুমি শুধু "আন!" বলো না — বলো "আন, **বলটা**!" বা "আন, **দড়িটা**!" খেলা এক, জিনিস বদলায়।

Script-ও চালানোর সময় সাথে জিনিস ধরিয়ে দেওয়া যায় — এদের বলে **argument** (অধ্যায় ৩-এর "হুকুম-ভঙ্গি-বস্তু" মনে আছে? এই সেই **বস্তু**)। Script-এর ভেতরে প্রথম জিনিসটার নাম `$1`, দ্বিতীয়টা `$2`, এভাবে:

```
nano salam.sh
```

```
#!/bin/bash
echo "salam, $1!"
echo "tumi $2 theke esecho, tai na?"
```

```
chmod +x salam.sh
./salam.sh Rahim Dhaka
```

```
salam, Rahim!
tumi Dhaka theke esecho, tai na?
```


Licensed to sakilorb@gmail.com · Do not redistribute

`Rahim` গেলো `$1`-এ, `Dhaka` গেলো `$2`-তে। এই ছোট্ট জিনিসটাই script-কে **যন্ত্র** বানায় — এক script, হাজার রকম ব্যবহার।

`if-else` — লিনুর সিদ্ধান্ত নেওয়া

🐾 লিনু দরজার শব্দ পেলে দুই রকম আচরণ করে: তুমি হলে দৌড়ে আসে, অচেনা কেউ হলে খাটের নিচে। **শর্ত দেখে সিদ্ধান্ত** — script-ও পারে:

```
nano dorja.sh
```

```
#!/bin/bash
if [ "$1" = "malik" ]; then
    echo "miyaw miyaw! douraiya ashlam! 🐱"
else
    echo "...khater niche lukailam 😱"
fi
```

চালাও দুইভাবে:

```
chmod +x dorja.sh
./dorja.sh malik
./dorja.sh chor
```

গঠনটা খুঁটিয়ে দেখো — bash-এর `if` একটু খুঁতখুঁতে, এই জায়গাগুলোতে সবাই প্রথমে হোঁচট খায়:

- শর্তটা থাকে `[ ]`-এর ভেতরে, আর **ব্র্যাকেটের সাথে স্পেস বাধ্যতামূলক** — `[ "$1" = "malik" ]` ঠিক, `["$1"="malik"]` ভুল
- `then` আলাদা লাইনে বা `;`-এর পরে

Licensed to sakilorb@gmail.com · Do not redistribute

- শেষ হয় `fi` দিয়ে — হ্যাঁ, `if` উল্টো করে! bash-এর রসবোধ 😄
- `"$1"`-এর কোটেশনগুলো অভ্যাস করো — argument ফাঁকা থাকলে বা তাতে স্পেস থাকলে কোটেশন-ছাড়া script অদ্ভুতভাবে ভাঙে

সংখ্যার তুলনায় চিহ্ন আলাদা: `-eq` (সমান), `-gt` (বড়), `-lt` (ছোট)। আর খুব কাজের দুটো: `-f` (ফাইলটা আছে?) আর `-d` (directory-টা আছে?):

```
if [ -d "$1" ]; then
    echo "$1 folder ta ache"
else
    echo "$1 bole kichu nai"
fi
```

Loop — একই কাজ বারবার

🐾 লিনুর সামনে পাঁচটা বল রাখো — ও কী করে? প্রতিটার কাছে যায়, শোঁকে, থাবা দেয়। **প্রতিটার জন্য একই কাজ।** এটাই loop:

```
nano khela.sh
```

```
#!/bin/bash
for BOL in lal nil sobuj holud kalo; do
    echo "linu $BOL bol ta niye khelche..."
done
```

চালালে পাঁচ লাইন — প্রতি বলের জন্য এক। `for ... in ... do ... done` — তালিকার প্রতিটা জিনিসের জন্য মাঝের কাজটুকু।

Licensed to sakilorb@gmail.com · Do not redistribute

আসল শক্তিটা দেখো — তালিকার জায়গায় **wildcard** বসাও (পর্ব ২-এর জাল!):

```
for FILE in *.txt; do
    echo "paisi: $FILE"
done
```

বর্তমান ফোল্ডারের **প্রতিটা** `.txt` ফাইলের জন্য কাজ। এখন কল্পনা করো `echo`-র জায়গায় `cp` (প্রতিটার backup), বা `mv` (প্রতিটার নাম বদল) — **শত ফাইলের কাজ, চার লাইনের script!** মাউস দিয়ে একটা একটা করে? সে জীবন তোমার পেছনে পড়ে গেছে। 😎

সংখ্যা ধরে ঘুরতে:

```
for i in {1..5}; do
    echo "chokkor $i"
done
```

(`{1..5}` — চেনা লাগছে? পর্ব ২-এর হাতেকলমে-র সেই brace expansion — `ghor{1,2,3}`! বোনাস হিসেবে শিখিয়েছিলাম, আজ কাজে লেগে গেলো।)

🛠️ Mini Project: এক হুকুমে প্রজেক্ট-ঘর তৈরি

এবার সব জোড়া লাগাই — variable, argument, if, আর কাজের কমান্ড — একটা **সত্যিকারের দরকারি** script-এ।

সমস্যাটা বাস্তব: নতুন প্রজেক্ট শুরু করা মানেই প্রতিবার একই একঘেয়ে কাজ — ফোল্ডার বানাও, ভেতরে সাব-ফোল্ডার বানাও, README ফাইল খোলো... আজ থেকে ওই পুরো কাজ হবে **এক কমান্ডে।**

Licensed to sakilorb@gmail.com · Do not redistribute

```
nano notun-project.sh
```

লেখো — প্রতিটা লাইনের পাশে মন্তব্য দিলাম (`#` দিয়ে শুরু লাইন/অংশ bash পড়ে না — এগুলো মানুষের জন্য নোট):

```
#!/bin/bash

# jodi nam na dey, thamiye dao
if [ -z "$1" ]; then
    echo "bhul! project er nam dao: ./notun-project.sh nam"
    exit 1
fi

PROJECT="$1"

# ei name age thekei kichu thakle, thamo
if [ -d "$PROJECT" ]; then
    echo "'$PROJECT' name folder age thekei ache!"
    exit 1
fi

# ghor banai
mkdir -p "$PROJECT"/{src,docs,tests}

# README banai, bhitore duita line
echo "# $PROJECT" > "$PROJECT/README.md"
echo "shuru: $(date)" >> "$PROJECT/README.md"

# report dei
echo "✅ '$PROJECT' toiri!"
tree "$PROJECT"
```


Licensed to sakilorb@gmail.com · Do not redistribute

নতুন যা যা দেখলে:

- `-z "$1"` — "জিনিসটা কি ফাঁকা?" — নাম না দিয়ে চালালে ভদ্রভাবে ধমক
- `exit 1` — script-কে এখানেই থামিয়ে দাও; `1` মানে "সমস্যা হয়েছিলো" (ঠিকঠাক শেষ হলে script নিজেই `0` বলে — এই সংখ্যার আসল ক্ষমতা `&&`-এর সাথে: মনে আছে "প্রথমটা সফল হলে দ্বিতীয়টা"? bash সফলতা মাপে এই সংখ্যা দিয়েই!)
- `$(date)` — `$( )`-এর ভেতরের কমান্ড চলে তার **output** ওই জায়গায় বসে যায়। মানে README-তে লেখা হবে বানানোর তারিখ-সময়। এই চালটার নাম command substitution — script-জগতের অন্যতম কাজের অস্ত্র
- আর `mkdir`-এর `{src,docs,tests}`, README-র `>` `>>`, শেষের `tree` — সব তোমার পুরনো বন্ধু

এবার:

```
chmod +x notun-project.sh
./notun-project.sh
./notun-project.sh amar-app
./notun-project.sh amar-app
```

প্রথমবার — ধমক (নাম দাওনি)। দ্বিতীয়বার — ✅ আর সুন্দর গাছ। তৃতীয়বার — "আগে থেকেই আছে!" **তিনটা পথই কাজ করছে — এটাই ভালো script-এর লক্ষণ: শুধু ঠিক কাজটা করে না, ভুল ব্যবহারেও ভদ্র থাকে।**

Licensed to sakilorb@gmail.com · Do not redistribute

🛠️ হাতেকলমে

1. এমন script লেখো যেটা `$1`-এ একটা ফাইলের নাম নেবে — ফাইলটা **থাকলে** তার লাইন সংখ্যা বলবে (`wc -l` তো জানোই), **না থাকলে** "nai to!" বলবে
2. Loop দিয়ে ১ থেকে ১০ — কিন্তু twist: `if` জুড়ে ৭-এর বেলায় বলবে "lucky 7!" আর বাকিদের বেলায় শুধু সংখ্যা (hint: সংখ্যার তুলনা মানে `-eq`)
3. `notun-project.sh`-কে নিজের মতো বদলাও — তোমার আসল কাজে যে ফোল্ডার-কাঠামো লাগে, সেটা বানাক
4. চ্যালেঞ্জ: এমন script যেটা বর্তমান ফোল্ডারের সব `.txt` ফাইলের নামের শেষে `.backup` জুড়ে কপি বানায় (hint: loop + `cp` + `"$FILE.backup"`)
5. ভেবে বলো: `salam.sh`-তে `./salam.sh "Rahim Uddin" Dhaka` চালালে কী হবে? আগে অনুমান, তারপর চালিয়ে মিলাও — কোটেশন কী করলো?

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

জিনিস	কাজ
`#!/bin/bash`	shebang — প্রথম লাইন, দোভাষী নির্ধারণ
`chmod +x`	script-কে চালানোর অনুমতি
`./script.sh`	এই ফোল্ডারের script চালাও
`$1`, `$2`	প্রথম, দ্বিতীয় argument
`if [ শর্ত ]; then ... else ... fi`	সিদ্ধান্ত
`-z` / `-f` / `-d` / `-eq`	ফাঁকা? / ফাইল আছে? / ফোল্ডার আছে? / সংখ্যা সমান?
`for X in তালিকা; do ... done`	প্রতিটার জন্য ঘোরো
`$(কমান্ড)`	কমান্ডের output বসাও
`exit 1`	থামো, "সমস্যা" জানিয়ে
`# মন্তব্য`	মানুষের জন্য নোট

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১৪: সময় ধরে কাজ — Cron

> 🐾 **দৃশ্য ২৫: ভোর পাঁচটার অ্যালার্ম**

বিড়ালওয়ালাদের একটা গোপন কষ্ট আছে, যেটা বাইরের মানুষ জানে না: **বিড়ালের পেটে ঘড়ি আছে।** ভোর পাঁচটা মানে ভোর পাঁচটা — লিনু তোমার বুকের ওপর উঠে বসবে, মুখে থাবা দেবে, প্রয়োজনে কানের কাছে গান গাইবে — যতক্ষণ না খাবার পড়ে। ছুটির দিন? লিনুর অভিধানে ওই শব্দ নেই। 😹

আজ আমরা এই ক্ষমতাটাই **উল্টো দিকে** ব্যবহার করবো। লিনুর পেটের ঘড়িটা এবার তোমার কাজে লাগবে: **তুমি সময় ঠিক করে দেবে, লিনু সেই সময়ে নিজে নিজে কাজটা করবে।** তুমি ঘুমাবে — কাজ হবে।

গত অধ্যায়ে লিনু শিখেছে খেলা (script)। এই অধ্যায়ে শিখবে **রুটিন** (cron)। দুটো মেলালে — automation। এই একটা শব্দই তোমাকে যারা টার্মিনাল জানে না তাদের থেকে চিরতরে আলাদা করে দেবে।

Cron আর Crontab

Linux-এ সবসময় একটা process চুপচাপ চলতে থাকে — নাম **cron** — যার একটাই কাজ: **ঘড়ি দেখা।** প্রতি মিনিটে সে একটা খাতা খুলে দেখে — "এই মিনিটে কারো কিছু করার কথা আছে?" — থাকলে করে দেয়, না থাকলে পরের মিনিটের অপেক্ষা।

সেই খাতার নাম **crontab** (cron table) — লিনুর রুটিনের খাতা। প্রত্যেক user-এর নিজের খাতা আছে। তোমারটা খোলো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
crontab -e
```

প্রথমবার জিজ্ঞেস করবে কোন এডিটরে খুলবে — একটা তালিকা দেখাবে, যেখানে nano-র পাশে লেখা থাকবে **easiest**। জীবনে প্রথমবার কোনো সফটওয়্যার নিজে থেকে সহজটা দেখিয়ে দিচ্ছে — সুযোগ নাও, nano-র নম্বরটা চাপো। 😄 (ভুলোর ঘরে ঢুকে পড়লে? মন্ত্র জানোই: `Esc` `:q!` — বেরিয়ে এসে `select-editor` চালিয়ে nano বেছে নাও।)

খাতা খুললো — ভেতরে `#` দিয়ে শুরু কিছু ব্যাখ্যা লেখা (মন্তব্য — গত অধ্যায়েই চিনেছো)। আমরা লিখবো একদম শেষে।

রুটিনের ভাষা — পাঁচটা তারা

Crontab-এর প্রতিটা লাইন একটা রুটিন, আর তার গঠন:

```
মিনিট ঘণ্টা মাসের-দিন মাস সপ্তাহের-দিন  কমান্ড
```

পাঁচটা ঘর সময়ের জন্য, তারপর কাজটা। যে ঘরে `*` (তারা) — মানে "**প্রতিটা**"। কয়েকটা পড়ে দেখি, অভ্যাস হয়ে যাবে:

Licensed to sakilorb@gmail.com · Do not redistribute

```
30 5 * * *     → মিনিট=৩০, ঘণ্টা=৫, বাকি সব তারা
               → প্রতিদিন ভোর ৫:৩০-এ (লিনুর খাবারের অ্যালার্ম! ⏰)

0 * * * *      → মিনিট=০, বাকি সব তারা
               → প্রতি ঘণ্টায়, কাঁটায় কাঁটায়

0 2 * * 0      → রবিবার (সপ্তাহের-দিন: ০=রবি) রাত ২:০০-এ, সপ্তাহে একবার

*/10 * * * *   → প্রতি ১০ মিনিটে (*/10 মানে "১০ ভাগে ভাগ করে")

0 9 1 * *      → প্রতি মাসের ১ তারিখ সকাল ৯টায়
```

⚠️ সবচেয়ে কমন ভুলটা আগেই বলে দিই: ঘরের **ক্রম** গুলিয়ে ফেলা — প্রথম ঘর মিনিট, **তারপর** ঘণ্টা। `5 30 * * *` লিখলে ওটা ভোর ৫:৩০ না — ওটা "প্রতিদিন ৩০টা বাজে ৫ মিনিটে"... আর ৩০টা তো বাজে না, তাই কাজটা **কোনোদিনই চলবে না।** মনে রাখার সূত্র: **ছোট থেকে বড় — মিনিট, ঘণ্টা, দিন, মাস।**

💡 আর একটা দারুণ সাইট মুখস্থ করে রাখো: **crontab.guru** — যেকোনো cron লাইন লিখলে সাদা বাংলায় (আচ্ছা, সাদা ইংরেজিতে 😄) বলে দেয় ওটা কখন চলবে। পাকা লোকেরাও জটিল রুটিন লেখার আগে ওখানে মিলিয়ে নেয়।

প্রথম রুটিন — লিনুর হাজিরা খাতা

তত্ত্ব অনেক হলো — এবার সত্যি সত্যি একটা রুটিন বসাই, আর **নিজের চোখে চলতে দেখি**।

`crontab -e`-তে একদম শেষে যোগ করো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
* * * * * echo "miyaw! ekhon $(date)" >> /home/rahim/linu-hajira.txt
```

(পাঁচটাই তারা = **প্রতি মিনিটে** — শুধু পরীক্ষার জন্য, পরে খুলে ফেলবো। আর `rahim`-এর জায়গায় নিজের username — মনে না থাকলে `whoami`, সেই অধ্যায় ৩-এর বন্ধু!)

> ⚠️ এখানে path-টা পুরো লিখলাম কেন, `~/linu-hajira.txt` না? **Cron-এর জগৎটা তোমার টার্মিনালের চেয়ে অনেক ফাঁকা** — তোমার `.bashrc` সেখানে চলে না, alias নেই, এমনকি `$PATH`-ও ছোট। তাই cron-এ নিয়ম: **সব path পুরো লেখো** — ফাইলের, script-এর, সবকিছুর। Cron-এ কাজ "চলছে না"-র ৯০% রহস্যের সমাধান এই এক লাইনে।

সেভ করে বের হও — দেখাবে `crontab: installing new crontab`। রুটিন চালু! এবার এক-দুই মিনিট অপেক্ষা... (চা বানিয়ে আনো ☕)... তারপর:

```
cat ~/linu-hajira.txt
tail -f ~/linu-hajira.txt
```

প্রতি মিনিটে এক লাইন করে জমছে! আর `tail -f` দিয়ে বসে থাকলে **চোখের সামনে** নতুন লাইন হাজির হবে — অধ্যায় ৮-এর সেই live জাদু, এবার লেখক তুমি না, লেখক **cron**। তোমার কম্পিউটার এইমাত্র প্রথমবার সম্পূর্ণ নিজে নিজে তোমার জন্য কাজ করলো। অনুভূতিটা টের পাচ্ছো? 🤖

পরীক্ষা শেষ — `crontab -e` খুলে লাইনটা মুছে দাও (নইলে লিনু প্রতি মিনিটে হাজিরা দিতেই থাকবে, ফাইল বাড়তেই থাকবে!)। খাতায় কী আছে দেখতে: `crontab -l`।

Licensed to sakilorb@gmail.com · Do not redistribute

🛠️ Mini Project: রাতের পাহারাদার — Automatic Daily Backup

এবার আসল জিনিস — এই পর্বের, এবং হয়তো পুরো বইয়ের সবচেয়ে **দরকারি** প্রজেক্ট। গল্পটা সবার জীবনে একবার ঘটে: মাসের পর মাসের কাজ এক ফোল্ডারে… একদিন ভুল `rm`, বা ডিস্ক নষ্ট, বা নিজেরই ভুল সিদ্ধান্ত — **সব শেষ।** যাদের backup ছিলো, তারা সেদিন শুধু কাঁধ ঝাঁকিয়ে ফাইল ফিরিয়ে আনে। বাকিরা গল্পটা সারাজীবন বলে বেড়ায়। 😅

আমরা প্রথম দলে থাকবো — আর কষ্টটাও করবে লিনু, প্রতি রাতে, নিজে নিজে।

ধাপ ১ — Backup script! গত অধ্যায়ের সব বিদ্যা একসাথে:

```
nano ~/scripts/backup.sh
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
#!/bin/bash

# ki backup hobe, kothay jabe
UTSHO="/home/rahim/projects"
GONTOBBO="/home/rahim/backups"

# gontobbo na thakle banai
mkdir -p "$GONTOBBO"

# ajker tarikh diye file er nam: backup-2026-07-14.tar.gz
TARIKH=$(date +%F)
FILE="$GONTOBBO/backup-$TARIKH.tar.gz"

# purota ekta compressed file e bandhi
tar -czf "$FILE" "$UTSHO" 2>> "$GONTOBBO/backup-error.log"

# hajira likhi
echo "$(date): backup hoyeche → $FILE" >> "$GONTOBBO/backup-
log.txt"

# 7 din er purono backup gulo felte thaki
find "$GONTOBBO" -name "backup-*.tar.gz" -mtime +7 -delete
```

(`rahim` আর `projects` — নিজেরটা বসাও। যে ফোল্ডারের কাজ হারালে কাঁদবে, সেটাই `UTSHO` ।)

নতুন মুখ দুটোর পরিচয়:

- **`tar -czf ফাইল.tar.gz ফোল্ডার`** — পুরো ফোল্ডারকে **এক ফাইলে বেঁধে চেপে** ফেলে (`c` reate, g `z` ip-চাপ, `f` ile-এ)। Linux জগতের সুটকেস-গোছানো। ফেরত খুলতে: `tar -xzf ফাইল` (e `x` tract)

Licensed to sakilorb@gmail.com · Do not redistribute

- `-mtime +7 -delete` — `find`-এর (পর্ব ২-এর গোয়েন্দা!) দুই নতুন কায়দা: "৭ দিনের বেশি পুরনো" + "পেলে মুছে দাও"। নইলে backup জমতে জমতে ডিস্কই শেষ — অধ্যায় ১১-এর `df -h` মনে আছে? 😄
- আর `$(date +%F)` — গত অধ্যায়ের command substitution; `+%F` মানে `2026-07-14` ফরম্যাট — ফাইলের নামে তারিখ, এক নজরে চেনা যায়

ধাপ ২ — নিজে হাতে একবার পরীক্ষা। Cron-এ দেওয়ার আগে script **সবসময়** হাতে চালিয়ে দেখবে — এটা পাকা নিয়ম:

```
chmod +x ~/scripts/backup.sh
~/scripts/backup.sh
ls -lh ~/backups
cat ~/backups/backup-log.txt
```

`.tar.gz` ফাইল দেখা যাচ্ছে? Log-এ লাইন পড়েছে? তাহলে —

ধাপ ৩ — লিনুর রুটিনে তোলা। `crontab -e`, শেষে:

```
0 3 * * * /home/rahim/scripts/backup.sh
```

পড়ে দেখো নিজেই: মিনিট=০, ঘণ্টা=৩, বাকি তারা — **প্রতিদিন রাত ৩টায়।** পুরো path, যেমন নিয়ম।

ব্যস। আজ থেকে প্রতি রাত ৩টায় — তুমি ঘুমে, ল্যাপটপ খোলা থাকলে লিনু জেগে: প্রজেক্ট বাঁধবে, চাপবে, তারিখ লিখে তুলে রাখবে, পুরনোগুলো ফেলবে, খাতায় হাজিরা দেবে। **একটা আঙুলও তোমাকে নাড়াতে হবে না।**

Licensed to sakilorb@gmail.com · Do not redistribute

কাল সকালে উঠে শুধু একবার দেখো: `ls -lh ~/backups` — রাতের পাহারাদার কাজ করেছে কিনা। 🌙🐾

> 💡 WSL ব্যবহারকারীদের জন্য সৎ স্বীকারোক্তি: WSL-এ cron তখনই চলে যখন WSL নিজে চালু থাকে — কম্পিউটার বন্ধ তো লিনুও ঘুমে। মন খারাপ কোরো না — **আসল মঞ্চ সামনেই।** পর্ব ৬-এ যখন নিজের server পাবে (যেটা ২৪ ঘণ্টা জেগে থাকে), এই একই script, এই একই crontab লাইন সেখানে বসিয়ে দেবে — আর সেখানে রুটিন চলবে বছরের প্রতিটা রাত, কোনো ছুটি ছাড়া। ঠিক লিনুর পেটের ঘড়ির মতো।

🛠️ হাতেকলমে

1. crontab.guru-তে গিয়ে ভাঙো: `15 14 * * 1-5` — কখন চলবে? (`1-5` কায়দাটা নতুন — অনুমান করো, সাইট মিলিয়ে দেবে)
2. এমন রুটিন লেখো (খাতায়, বসাতে হবে না): প্রতি শুক্রবার রাত ৯টায়
3. হাজিরা-খাতার পরীক্ষাটা নিজে করো — বসাও, দুই মিনিট `tail -f`-এ দেখো, তারপর মুছে দাও
4. Backup ব্যবস্থাটা পুরো দাঁড় করাও — নিজের আসল কাজের ফোল্ডার দিয়ে। কাল সকালে `ls -lh ~/backups` — ফল দেখো
5. চ্যালেঞ্জ: backup.sh-তে জোড়ো — backup শেষে ফোল্ডারে ৩টার বেশি `.tar.gz` থাকলে log-এ লিখবে "onek backup jomeche!" (hint: `ls ... | wc -l` + `if` + `-gt` — তিন পর্বের তিন অস্ত্র, এক লাইনে)
6. ভাবনার প্রশ্ন: রুটিনটা রাত ৩টায় কেন দিলাম — সকাল ১০টায় না কেন? (ভাবো: ওই সময় কম্পিউটার কী করছে, আর backup চলার সময় বড় ফোল্ডার হলে কী হয়)

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড	কাজ
`crontab -e`	রুটিনের খাতা সম্পাদনা
`crontab -l`	খাতায় কী আছে
`মিনিট ঘণ্টা দিন মাস বার কমান্ড`	রুটিনের গঠন (ছোট → বড়)
`*` / `*/10` / `1-5`	প্রতিটা / প্রতি ১০ / ১ থেকে ৫
`tar -czf নাম.tar.gz ফোল্ডার`	বেঁধে চাপো (backup!)
`tar -xzf নাম.tar.gz`	খোলো
`find ... -mtime +7 -delete`	৭ দিনের পুরনোগুলো মোছো
crontab.guru	রুটিনের অনুবাদক — বুকমার্ক করো

🐾 **পর্ব ৫ শেষ।** ভেবে দেখো ব্যাপারটা: বইয়ের শুরুতে লিনু তোমার একটা হুকুমও শুনতো না। তারপর এক হুকুমে এক কাজ শিখলো। তারপর হুকুমের চেইন। আর আজ? **তুমি না থাকলেও লিনু কাজ করে।** রাত ৩টায়, তুমি ঘুমে, আর তোমার শেখানো খেলাটা ও নিখুঁতভাবে খেলে চলেছে — প্রতিদিন।

এটাই automation — আর এটাই সেই দক্ষতা যেটা একজন engineer-কে আলাদা করে: যে কাজ দুইবারের বেশি করতে হয়, সেটা script-এ ঢালো; যে script নিয়মিত চালাতে হয়, সেটা cron-এ তোলো। এই এক বাক্য মেনে চলা মানুষদের ক্যারিয়ার অন্যরকম হয়।

বাকি রইলো শেষ পর্ব — আর সেটা সবচেয়ে রোমাঞ্চকর। এতদিন লিনু খেলেছে তোমার বাসার ভেতরে। পর্ব ৬-এ ও পা রাখবে **বাইরের জগতে** — শিখবে দূরের কম্পিউটারের সাথে কথা বলা (SSH), নেটওয়ার্কের খোঁজখবর, আর সবশেষে সেই মুহূর্ত যার জন্য পুরো বইটা লেখা: **নিজের server-এ,**

Licensed to sakilorb@gmail.com · Do not redistribute

নিজের হাতে গড়া, ইন্টারনেটে জ্যান্ত একটা website — যেখানে এই বইয়ের প্রতিটা অধ্যায়ের বিদ্যা এক জায়গায় কাজে লাগবে।

লিনু জানালার ধারে বসে বাইরে তাকিয়ে আছে। ওর চোখে বড় জগতের স্বপ্ন। 🌍🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পর্ব ৬: লিনু বাইরের জগতে (Networking & Server)

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১৫: লিনুর সাথে দূর থেকে কথা — SSH

> 🐾 **দৃশ্য ২৬: গ্রামের বাড়ির বিড়াল**

ধরো তোমার গ্রামের বাড়িতেও একটা বিড়াল আছে — লিনুর ভাই। তুমি ঢাকায়, ও গ্রামে। এখন যদি এমন একটা জাদুর ফোন থাকতো — যেটা দিয়ে তুমি ঢাকায় বসেই গ্রামের বিড়ালটাকে হুকুম দিতে পারতে, আর ও সেখানে বসে কাজ করে দিতো — কেমন হতো?

Linux জগতে এই জাদুর ফোনটা **সত্যিই আছে।** নাম **SSH** (**S**ecure **Sh**ell)। এটা দিয়ে তুমি নিজের টার্মিনালে বসে পৃথিবীর যেকোনো প্রান্তের Linux কম্পিউটারে ঢুকতে পারো — আর ঢোকার পর মনে হবে তুমি **ওই কম্পিউটারেই** বসে আছো: সেই চেনা prompt, সেই `ls`, সেই `cd` — সব একই, শুধু মেশিনটা হাজার মাইল দূরে।

আর এইটাই হলো সেই দক্ষতা যার কথা বইয়ের শুরুতে বলেছিলাম — "SSH করে সার্ভারে ঢুকে log দেখতে পারবেন?" — এই অধ্যায়ের পর উত্তরটা হবে: **"অবশ্যই।"**

SSH কীভাবে কাজ করে — দুই মিনিটের ছবি

ছবিটা সরল:

- **দূরের কম্পিউটারে** একটা process সবসময় কান পেতে বসে থাকে — নাম `sshd` (ssh daemon; daemon মানে পেছনে চুপচাপ চলা process — অধ্যায় ১১-এর background job-দের স্থায়ী সংস্করণ ভাবো)
- **তোমার কম্পিউটার থেকে** তুমি `ssh` কমান্ড দিয়ে ডাক দাও

Licensed to sakilorb@gmail.com · Do not redistribute

- দুজনের মাঝে একটা **encrypted সুড়ঙ্গ** তৈরি হয় — মাঝপথে কেউ আড়ি পাতলেও শুনবে শুধু হিজিবিজি। (নামের **S**ecure অংশটা এখানেই — আর এজন্যই গোটা পৃথিবীর সব কোম্পানি নিশ্চিন্তে SSH ব্যবহার করে)

ডাক দেওয়ার গঠন:

```
ssh ব্যবহারকারী@ঠিকানা
```

যেমন: `ssh rahim@203.0.113.10` — মানে "ওই ঠিকানার কম্পিউটারে, rahim হিসেবে ঢুকতে চাই।" ঠিকানাটা হলো **IP address** — ইন্টারনেটে প্রতিটা কম্পিউটারের বাড়ির নম্বর (পরের অধ্যায়ে এ নিয়ে আরো কথা হবে)।

এখনো তোমার দূরের কোনো কম্পিউটার নেই — সেটা আসছে অধ্যায় ১৭-তে। তাই এই অধ্যায়ে আমরা **সরঞ্জামগুলো তৈরি করে** রাখবো, যাতে সময় এলে এক মিনিটে কাজে নামা যায়।

Password বনাম চাবি — SSH Key

SSH-এ ঢোকার দুটো উপায়:

উপায় ১: Password। ঢোকার সময় password চাইবে, দিলে ঢুকতে দেবে। সহজ, কিন্তু দুর্বল — কারণ ইন্টারনেটে বসানো একটা server-এ **সারাদিন, সারারাত** পৃথিবীর নানা প্রান্ত থেকে robot-রা এলোপাথাড়ি password অনুমান করে ঢোকার চেষ্টা চালায়। (মজা করছি না — অধ্যায় ১৭-তে নিজের server-এর log-এ নিজের চোখে দেখবে!) Password যদি দুর্বল হয়, কোনো একদিন লেগে যাবে।

Licensed to sakilorb@gmail.com · Do not redistribute

উপায় ২: SSH Key। একটা **চাবি-তালা জোড়া**:

- **Private key (গোপন চাবি)** — থাকবে শুধু তোমার কম্পিউটারে। কাউকে দেবে না, কোথাও পাঠাবে না — **কখনোই।**
- **Public key (তালা)** — এটা তুমি দূরের server-এ বসিয়ে দেবে। এটা গোপন না — তালা তো সবাই দেখতেই পারে, খুলতে পারে শুধু চাবিওয়ালা।

ঢোকার সময় server তালা দেখাবে, তোমার `ssh` চাবি মেলাবে — মিললে ঢুকবে। অনুমান করার কিছুই নেই — robot-রা বেকার। 🔐

চাবি-তালা বানাও (তোমার নিজের কম্পিউটারে):

```
ssh-keygen -t ed25519
```

(`ed25519` হলো চাবি বানানোর আধুনিক, দ্রুত আর নিরাপদ পদ্ধতির নাম — এটাই এখনকার প্রচলিত পছন্দ।) তিনটা প্রশ্ন করবে:

- **কোথায় রাখবো?** — Enter (default জায়গাই ঠিক: `~/.ssh/`)
- **Passphrase?** — চাবির নিজেরও একটা password রাখা যায়, বাড়তি নিরাপত্তা। শেখার সময় Enter চেপে ফাঁকা রাখতে পারো; আসল কাজে দেওয়া ভালো
- আরেকবার confirm — Enter

দেখো কী তৈরি হলো:

```
ls -l ~/.ssh
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
id_ed25519        ← গোপন চাবি (কাউকে না!)
id_ed25519.pub    ← তালা (.pub = public — এটাই বিলি করার জিনিস)
```

তালাটা দেখতে কেমন? `cat ~/.ssh/id_ed25519.pub` — এক লাইনের হিজিবিজি। এই লাইনটাই পরে server-এর একটা বিশেষ ফাইলে (`~/.ssh/authorized_keys` — "অনুমোদিত তালাদের তালিকা") বসিয়ে দিলে কাজ শেষ। কীভাবে — অধ্যায় ১৭-তে হাতেকলমে।

> 💡 GitHub ব্যবহার করো বা করবে? সুখবর: GitHub-এ কোড push করার নিরাপদ উপায়ও **এই একই চাবি-তালা!** Settings → SSH Keys-এ ওই `.pub` লাইনটা বসিয়ে দিলেই হয়। এক শেখায় দুই জগৎ।

ফাইল পাঠানো-আনা — `scp` আর `rsync`

দূরের কম্পিউটারে ঢুকতে পারা তো হলো — কিন্তু **ফাইল** আনা-নেওয়া করবে কীভাবে? Pendrive নিয়ে তো আর হাজার মাইল যাওয়া যায় না। 😄

`scp` **(secure copy)** — চেনা `cp`-রই দূরপাল্লার ভাই। গঠনও এক: `scp উৎস গন্তব্য` — শুধু দূরের ঠিকানা লেখা হয় `ব্যবহারকারী@ঠিকানা:পথ` কায়দায়:

```
scp boi.pdf rahim@203.0.113.10:/home/rahim/      # আমার → দূরে
scp rahim@203.0.113.10:/home/rahim/log.txt .
# দূরের → আমার (. মানে "এখানে")
scp -r amar-folder rahim@203.0.113.10:~/         # পুরো ফোল্ডার — সেই চেনা -r!
```

`rsync` — `scp`-র বুদ্ধিমান বড় ভাই। পার্থক্যটা একটা দৃশ্যেই পরিষ্কার হবে:

Licensed to sakilorb@gmail.com · Do not redistribute

🐾 ধরো লিনুর ১০০টা খেলনা গ্রামের বাড়িতে পাঠাতে হবে। `scp` প্রতিবার **সবগুলো** নতুন করে পাঠায় — আগে ৯৮টা পাঠানো থাকলেও। আর `rsync` আগে দুই বাড়ির তালিকা মিলিয়ে দেখে, তারপর **শুধু নতুন বা বদলানো ২টা** পাঠায়। বড় প্রজেক্টে এই পার্থক্য মানে ঘণ্টা বনাম সেকেন্ড।

```
rsync -avz amar-project/ rahim@203.0.113.10:~/amar-project/
```

(`-a` = সব সমেত আর্কাইভ-কায়দায়, `-v` = কী করছো বলতে বলতে করো, `-z` = পথে চেপে পাঠাও।) দ্বিতীয়বার একই কমান্ড চালিয়ে দেখো — নিমেষে শেষ, কারণ পাঠানোর মতো নতুন কিছু নেই। এজন্যই deploy-এর জগতে `rsync` এত প্রিয়।

> ⚠️ `rsync` -এ পথের শেষের `/` -টার আলাদা মানে আছে: `amar-project/` (slash-সহ) মানে "ফোল্ডারের **ভেতরের জিনিসগুলো**", আর `amar-project` (slash-ছাড়া) মানে "ফোল্ডারটা **নিজেই**"। গুলিয়ে গেলে গন্তব্যে ফোল্ডারের ভেতরে ফোল্ডার হয়ে বসে থাকে — সবার একবার হয়, তোমারও হবে, তখন এই লাইনটা মনে কোরো। 😄

🛠️ হাতেকলমে

1. `ssh-keygen -t ed25519` দিয়ে নিজের চাবি-তালা বানাও — অধ্যায় ১৭-র জন্য তৈরি থাকলো
2. `ls -l ~/.ssh` চালাও — দুটো ফাইলের **permission** পড়ো (অধ্যায় ১০-এর বিদ্যা!)। গোপন চাবিটার নিয়ম কী? কেন এত কড়া, ভাবো
3. `cat ~/.ssh/id_ed25519.pub` — নিজের তালাটা চোখে দেখো (এটা দেখানো নিরাপদ; ভুলেও `.pub` -ছাড়াটা কোথাও পেস্ট কোরো না!)

Licensed to sakilorb@gmail.com · Do not redistribute

4. GitHub account থাকলে: Settings → SSH and GPG keys → New SSH key → `.pub`-এর লাইনটা বসাও — আজ থেকে password ছাড়া push!
5. ভাবনার প্রশ্ন: বাসার দরজার চাবি আর তালার ছবি ফেসবুকে দিলে কোনটায় বিপদ? SSH-এর সাথে মিলাও 😄

📜 হুকুমনামা

কমান্ড	কাজ
`ssh user@ঠিকানা`	দূরের কম্পিউটারে ঢোকো
`ssh-keygen -t ed25519`	চাবি-তালা বানাও
`~/.ssh/id_ed25519`	গোপন চাবি — কাউকে না
`~/.ssh/id_ed25519.pub`	তালা — server/GitHub-এ বসাও
`scp ফাইল user@ঠিকানা:পথ`	দূরে ফাইল পাঠাও (`-r` = ফোল্ডার)
`rsync -avz উৎস/ গন্তব্য`	বুদ্ধিমান sync — শুধু বদলটুকু

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১৬: Network-এর খোঁজখবর

> 🐾 **দৃশ্য ২৭: জানালার ধারে**

লিনু আজকাল জানালার ধারেই বসে থাকে। বাইরের জগৎটা ও বুঝতে চায় — কোন বাড়ি কোথায়, কে কাকে ডাকছে, কোন পথে কী আসে-যায়। দূরের জগতে পা দেওয়ার আগে পাড়াটা তো চিনতে হবে, নাকি?

Server-এ কাজ করতে গেলে network-এর গোটা তত্ত্ব লাগে না — লাগে **কয়েকটা ধারণা আর গোটা পাঁচেক কমান্ড।** এই অধ্যায়ে ঠিক সেটুকুই — যেটুকু অধ্যায় ১৭-র যুদ্ধে লাগবে।

IP Address — বাড়ির নম্বর

আগের অধ্যায়েই দেখেছো: ইন্টারনেটে প্রতিটা কম্পিউটারের একটা ঠিকানা থাকে — **IP address**। যেমন `203.0.113.10` — চারটা সংখ্যা, ডট দিয়ে আলাদা। নিজেরটা দেখো:

```
ip a
```

অনেক কিছু দেখাবে — খোঁজো `inet` লেখা লাইনগুলো। দুটো জিনিস চোখে পড়বে:

- `127.0.0.1` — এটা প্রতিটা কম্পিউটারে থাকে, মানে "**আমি নিজে**"। ডাকনাম `localhost`। নিজের কম্পিউটারে চলা জিনিস নিজেই দেখতে এই ঠিকানা (একটু পরেই ব্যবহার করবে!)

Licensed to sakilorb@gmail.com · Do not redistribute

- `192.168.x.x` **ধাঁচের একটা** — তোমার **ঘরের ভেতরের** ঠিকানা: রাউটার তার wifi-এর সবাইকে এমন নম্বর দেয়। এগুলো পাড়ার ঠিকানা — ইন্টারনেটের বাইরে থেকে সরাসরি ডাকা যায় না

আর server-এর থাকে **public IP** — পৃথিবীর যেকোনো জায়গা থেকে ডাকা যায় এমন ঠিকানা। অধ্যায় ১৭-তে তোমার server-এর সাথে ওরকম একটা ঠিকানাও পাবে — ইন্টারনেটে তোমার নিজের বাড়ির নম্বর। 🏠

`ping` — বেঁচে আছো?

Network জগতের সবচেয়ে সরল আর সবচেয়ে ব্যবহৃত প্রশ্ন — "ওই ঠিকানায় কেউ আছো? সাড়া দাও!"

```
ping google.com
```

```
64 bytes from ...: icmp_seq=1 ttl=115 time=48.2 ms
64 bytes from ...: icmp_seq=2 ttl=115 time=47.8 ms
```

প্রতি লাইন মানে একটা ডাক আর তার সাড়া — `time` হলো যাওয়া-আসার সময় (ঢাকা থেকে Google প্রায় ৪০-৬০ ms — আলোর গতিতেও সিঙ্গাপুর ঘুরে আসতে এটুকু লাগে!)। `Ctrl+C` দিয়ে থামাও (পুরনো বন্ধু), শেষে সারাংশ দেখাবে — কয়টা ডাকে কয়টা সাড়া।

> 💡 "নেট চলছে না!" — এখন থেকে তোমার প্রথম পরীক্ষা: `ping google.com`। সাড়া আসছে = নেট আছে, সমস্যা অন্য কোথাও। `Name or service not known` জাতীয়

Licensed to sakilorb@gmail.com · Do not redistribute

> error = আসলেই সংযোগে গোলমাল। আর অধ্যায় ১৭-তে server কেনার পর প্রথম যে কাজটা করবে — `ping server-এর-ঠিকানা` — "বেঁচে আছো তো?" 😄

Port — বাড়ির দরজাগুলো

এবার এই অধ্যায়ের সবচেয়ে জরুরি ধারণা। একটা ধাঁধা দিয়ে শুরু:

এক server-এ একইসাথে website চলছে, SSH চলছে, database চলছে। বাইরে থেকে সবার ঠিকানা তো **একটাই IP** — তাহলে তোমার ব্রাউজারের ডাক website-এর কাছে যায় আর তোমার `ssh`-এর ডাক sshd-র কাছে যায় **কীভাবে?** ডাক গুলিয়ে যায় না কেন?

উত্তর: **এক বাড়ি, অনেক দরজা।** প্রতিটা IP-র সাথে ৬৫,৫৩৫টা নম্বরওয়ালা দরজা আছে — এদের নাম **port**। প্রতিটা service নিজের একটা দরজায় কান পেতে বসে থাকে, আর ডাক দেওয়ার সময় ঠিকানার সাথে দরজার নম্বরও বলা হয়। কিছু দরজার নম্বর যুগ যুগ ধরে ঠিক করা:

Port	কে বসে থাকে
22	SSH — গত অধ্যায়ের `ssh` আসলে এই দরজায় টোকা দেয়
80	HTTP — সাধারণ website
443	HTTPS — তালা-মারা website (ব্রাউজারের ঠিকানায় যে 🔒 দেখো)

মানে `ssh rahim@203.0.113.10` আসলে বলছে — "ওই বাড়ির **২২ নম্বর দরজায়** টোকা দাও।" আর ব্রাউজারে কোনো সাইট খোলা মানে সেই বাড়ির ৮০ বা ৪৪৩ নম্বর দরজায় ডাক।

`curl` আর `wget` — টার্মিনাল থেকে ইন্টারনেট

ব্রাউজার ছাড়াই ইন্টারনেটে ডাকাডাকির দুই অস্ত্র:

Licensed to sakilorb@gmail.com · Do not redistribute

`curl` — কোনো ঠিকানায় ডাক দিয়ে **জবাবটা টার্মিনালে** দেখায়:

```
curl ifconfig.me
```

এই ছোট্ট কমান্ডটা চালাও — একটা IP ফেরত আসবে। ওটা তোমার **public IP** — পৃথিবী তোমাকে যে ঠিকানায় চেনে! (`ip a`-র 192.168 ঠিকানার সাথে মিলবে না — ওটা পাড়ার, এটা পৃথিবীর।)

`curl`-এর আসল কদর বুঝবে API নিয়ে কাজ করার দিন — কোনো service ঠিক জবাব দিচ্ছে কিনা এক লাইনে পরীক্ষা। অধ্যায় ১৭-তে নিজের website দাঁড় করিয়েই প্রথম পরীক্ষাটা `curl` দিয়েই করবো।

`wget` — জবাব দেখায় না, **ফাইল হিসেবে নামায়:**

```
wget https://example.com/boi.pdf
```

সোজা হিসাব: **দেখতে চাইলে `curl`, নামাতে চাইলে `wget`।**

`ufw` — দারোয়ান বসানো

> 🐾 **দৃশ্য ২৮: কে ঢুকবে, কে ঢুকবে না**

৬৫ হাজার দরজাওয়ালা বাড়ি — শুনতেই ছমছমে, তাই না? সব দরজা খোলা রাখা মানে চোরের নিমন্ত্রণ। সমাধান সেই পুরনো আমলের: **দারোয়ান** — যে জানে কোন দরজা খোলা থাকবে, বাকি সব বন্ধ।

Licensed to sakilorb@gmail.com · Do not redistribute

Linux-এর দারোয়ানের নাম **firewall**, আর Ubuntu-তে তাকে হুকুম দেওয়ার সহজ কমান্ড `ufw` (**U**ncomplicated **F**ire**w**all — নামেই বলছে "জটিলতা-ছাড়া")। হুকুমগুলো এতই সোজা যে পড়লেই বোঝা যায়:

```
sudo ufw allow 22       # ২২ নম্বর (SSH) খোলা
sudo ufw allow 80       # ৮০ (website) খোলা
sudo ufw allow 443      # ৪৪৩ (তালা-মারা website) খোলা
sudo ufw enable         # দারোয়ান, ডিউটিতে যাও!
sudo ufw status         # হাজিরা দেখা
```

ব্যস — এই তিন দরজা ছাড়া বাকি ৬৫ হাজার দরজায় বাইরে থেকে টোকা দিলেও সাড়া নেই।

> ⚠️ **জীবন বাঁচানো সতর্কতা, এখনই মুখস্থ করো:** দূরের server-এ `ufw enable` চালানোর **আগে** অবশ্যই `allow 22` দিতে হবে। ক্রমটা উল্টে ফেললে কী হবে ভাবো — দারোয়ান ডিউটিতে গেলো, SSH-এর দরজাও বন্ধ করলো... আর তুমি? **নিজের বাড়ির বাইরে, চাবি ভেতরে।** 🔒😱 Server-জগতে নিজেকে-তালা-মারা (lockout) নতুনদের ক্লাসিক দুর্ঘটনা — তুমি জেনে গেলে, তোমার হবে না।

🛠️ হাতেকলমে

1. `ip a` চালিয়ে নিজের ভেতরের ঠিকানা বের করো, তারপর `curl ifconfig.me` দিয়ে বাইরের — দুটো আলাদা কেন, নিজের ভাষায় বলো
2. `ping google.com` ১০ সেকেন্ড চালিয়ে থামাও — গড় `time` কত ms?
3. `ping 192.0.2.1` চালাও (এটা ইচ্ছা করে অচল ঠিকানা) — সাড়া না-আসা দেখতে কেমন, চিনে রাখো

Licensed to sakilorb@gmail.com · Do not redistribute

4. `curl wttr.in/Dhaka` — মজার উপহার: টার্মিনালে আবহাওয়া! ☔ (একটা server আছে যে curl-এর ডাকে ছবি-আঁকা আবহাওয়া পাঠায়)
5. ভাবনার প্রশ্ন: বইয়ের শুরু থেকে WSL/VM-এ কাজ করছো, firewall ছাড়া — ভয়ের কিছু ছিলো না কেন? (Hint: তোমার ঠিকানাটা পাড়ার না পৃথিবীর?) আর server-এ প্রথম দিনেই দারোয়ান লাগবে কেন?

📜 হুকুমনামা

কমান্ড	কাজ
`ip a`	আমার ঠিকানাগুলো
`ping ঠিকানা`	বেঁচে আছো? (`Ctrl+C` থামাও)
`curl ঠিকানা`	ডেকে জবাব দেখো
`curl ifconfig.me`	আমার public IP
`wget ঠিকানা`	ফাইল নামাও
Port 22 / 80 / 443	SSH / HTTP / HTTPS-এর দরজা
`sudo ufw allow নম্বর`	দরজা খোলা রাখো
`sudo ufw enable` / `status`	দারোয়ান চালু / হাজিরা

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১৭: 🏆 Final Project — লিনুকে চাকরিতে পাঠানো

> 🐾 **দৃশ্য ২৯: বিদায়... না, শুরু**

আজ বিশেষ দিন। লিনু আর ঘরের বিড়াল থাকছে না — **চাকরিতে যাচ্ছে।** ওর নতুন কর্মস্থল: একটা server, যেটা দিন-রাত ২৪ ঘণ্টা জেগে থেকে পৃথিবীর যেকোনো প্রান্তের মানুষকে তোমার website দেখাবে।

আর সত্যিটা হলো — চাকরিটা আসলে লিনুর না, **তোমার।** এই অধ্যায়ে তুমি যা যা করবে, সেগুলোই একজন junior DevOps/backend engineer-এর চাকরির প্রথম সপ্তাহের কাজ: server নেওয়া, নিরাপদ করা, web server বসানো, site deploy করা। পুরো বইয়ের **প্রতিটা** পর্বের বিদ্যা আজ এক মঞ্চে নামবে — মিলিয়ে মিলিয়ে দেখো।

চলো। 🚀

ধাপ ০: একটা VPS নেওয়া

VPS (Virtual Private Server) = ভাড়ার কম্পিউটার, যেটা কোনো datacenter-এ ২৪ ঘণ্টা চলে, আর পুরোটা **তোমার** — root-সহ। (অধ্যায় ২-এর VirtualBox মনে আছে? VPS ব্যাপারটা ওরকমই একটা virtual machine — শুধু চলছে অন্য কারো শক্তিশালী কম্পিউটারে, public IP-সহ।)

কোথা থেকে নেবে: DigitalOcean, Vultr, Linode, Hetzner — সবারই শুরুর দামের server আছে, মাসে মোটামুটি $৪-৬ (৫০০-৭০০ টাকার মতো)। অনেকে নতুনদের ফ্রি credit-ও দেয় —

Licensed to sakilorb@gmail.com · Do not redistribute

খুঁজে দেখো। শেখার জন্য সবচেয়ে ছোটটাই যথেষ্ট। লাগবে একটা international payment-এর ব্যবস্থা (দেশি অনেক ব্যাংকের dual currency card-এ হয়)।

নেওয়ার সময় তিনটা পছন্দ করতে বলবে:

- **OS:** Ubuntu LTS (কোনটা নেবো ভাবাই লাগছে না, তাই না? 😄)
- **অঞ্চল (region):** Singapore — বাংলাদেশ থেকে সবচেয়ে কাছের, ping সবচেয়ে কম
- **SSH key:** ফর্মে public key চাইলে **এখনই** তোমার `~/.ssh/id_ed25519.pub`-এর লাইনটা বসিয়ে দাও (`cat` করে কপি — অধ্যায় ১৫-তে বানানোই আছে!) — তাহলে জন্ম থেকেই server চাবি-তালায় চলবে

মিনিটখানেকের মধ্যে server তৈরি — আর স্ক্রিনে দেখাবে ওর **public IP**। ওটাই এখন তোমার সবচেয়ে দামি সংখ্যা — বইয়ের বাকি অংশে যেখানে `SERVER_IP` লিখবো, ওখানে ওটা বসাবে।

প্রথম কাজ — অধ্যায় ১৬-র বিদ্যা: **বেঁচে আছো?**

```
ping SERVER_IP
```

সাড়া আসছে? চলো ভেতরে।

ধাপ ১: প্রথম প্রবেশ

```
ssh root@SERVER_IP
```

প্রথমবার একটা প্রশ্ন করবে — "এই ঠিকানার এই আঙুলের ছাপ (fingerprint) চেনো না, বিশ্বাস করবে?" — নতুন কারো সাথে প্রথম দেখায় স্বাভাবিক প্রশ্ন। `yes` লিখে Enter।

Licensed to sakilorb@gmail.com · Do not redistribute

আর তারপর... prompt বদলে গেলো:

```
root@amar-server:~#
```

থামো। এক মিনিট শুধু তাকিয়ে থাকো। 🎉 তুমি এই মুহূর্তে সিঙ্গাপুরের কোনো এক ঠান্ডা datacenter-এর একটা কম্পিউটারের ভেতরে বসে আছো — ঢাকার চেয়ার থেকে। `ls` চালাও, `pwd` চালাও — সব চেনা, সব তোমার। বইয়ের প্রথম অধ্যায়ের সেই ভয়-পাওয়া মানুষটা আজ **আন্তর্জাতিক।** 😄

আর prompt-টা খুঁটিয়ে দেখো — অধ্যায় ৩-এর পড়া কাজে লাগাও: `root@` মানে তুমি স্বয়ং কর্তা, আর শেষের `$`-এর জায়গায় `#` — মনে আছে বলেছিলাম, "এটা যদি `#` হয়, তুমি root"? সেই দিন আজ এলো।

কর্তা হিসেবে প্রথম হুকুম — নতুন সংসারকে টাটকা করা (অধ্যায় ৭!):

```
apt update && apt upgrade -y
```

(root-এর `sudo` লাগে না — সে নিজেই sudo-র উৎস। আর `-y` মানে "সব প্রশ্নের উত্তর হ্যাঁ ধরে এগোও"।)

ধাপ ২: নিজের ঘর বানানো — root ছাড়ো

কিন্তু অধ্যায় ১০-এ কী শিখেছিলে? **"সারাদিন কর্তা সেজে থাকা বিপজ্জনক — চাবি থাকবে সিন্দুকে, দরকারে বের হবে।"** root হিসেবে দৈনন্দিন কাজ মানে প্রতিটা typo-র হাতে পুরো server-এর জীবন। তাই পাকা নিয়ম: প্রথম দিনেই একটা সাধারণ user বানাও, root-এ ঢোকা বন্ধ করো।

```
adduser rahim
```


Licensed to sakilorb@gmail.com · Do not redistribute

(password দাও — শক্ত একটা; বাকি প্রশ্নগুলো Enter চেপে পার হও।) এবার rahim-কে "দরকারে-চাবি-তোলার" অধিকার দাও — মানে `sudo` দলে ঢোকাও:

```
usermod -aG sudo rahim
```

(পড়ো: user-কে modify করো, যোগ করো Group-এ, দলের নাম sudo — অধ্যায় ১০-এর **group** ধারণাটা এই যে কাজে লেগে গেলো!)

এবার rahim-এর ঘরে তোমার **তালা** বসাই, যাতে চাবি দিয়ে সরাসরি rahim হয়েই ঢোকা যায়। Server ছেড়ে বেরোও (`exit`), তারপর নিজের কম্পিউটার থেকে এই একটা কমান্ড:

```
ssh-copy-id rahim@SERVER_IP
```

এই ছোট্ট যন্ত্রটা তোমার `.pub` তালাটা নিয়ে rahim-এর `~/.ssh/authorized_keys`-এ (অধ্যায় ১৫-র সেই "অনুমোদিত তালাদের তালিকা") ঠিকঠাক বসিয়ে দেয় — permission-সহ। (rahim-এর password চাইবে — এই শেষবারের মতো।) এবার পরীক্ষা:

```
ssh rahim@SERVER_IP
```

Password ছাড়াই ঢুকে গেলে? 🔐 চাবি-তালা চলছে। Prompt-ও দেখো — `rahim@amar-server:~$` — আবার ভদ্র `$`।

ধাপ ৩: দরজা মজবুত করা — SSH hardening

এখন server-টা নিরাপদ করি। মনে আছে অধ্যায় ১৫-তে বলেছিলাম robot-দের কথা? কথাটা কতটা সত্যি, নিজের চোখে দেখো — server-এ ঢুকে:

Licensed to sakilorb@gmail.com · Do not redistribute

```
sudo grep "Failed password" /var/log/auth.log | tail -n 20
```

Server-এর বয়স এক ঘণ্টাও হয়নি — অথচ সম্ভবত এর মধ্যেই তালিকায় অচেনা IP-দের ব্যর্থ চেষ্টা জমতে শুরু করেছে! (আর দেখো কমান্ডটা — `grep`, pipe, `tail`, `/var/log` — পর্ব ২, ৪ আর অধ্যায় ৪ এক লাইনে। তুমি এখন এসব **এমনিই** পড়ে ফেলো, খেয়াল করেছো?)

> 💡 `No such file or directory` পেলে ঘাবড়িয়ো না — Ubuntu-র নতুন সংস্করণগুলোয় এই খাতাটা default-এ থাকে না, ডায়েরি লেখা হয় অন্য ব্যবস্থায় (journal)। সেখান থেকে পড়ার কমান্ড:
>
> `sudo journalctl -u ssh | grep "Failed" | tail -n 20` — pipe-এর অংশটুকু হুবহু এক, শুধু ডায়েরির উৎস বদলালো।

এদের রাস্তা পুরো বন্ধ করবো — **password-ঢোকা নিষিদ্ধ, শুধু চাবি।** SSH-এর নিয়মের খাতাটা খোলো — সেটা থাকে, কোথায় আবার — `/etc/`-তে (অধ্যায় ৪: "নিয়মকানুনের খাতার ঘর"!):

```
sudo nano /etc/ssh/sshd_config
```

খুঁজে বের করে (nano-তে খোঁজা: `Ctrl+W`) এই দুটো লাইন এমন করো — সামনে `#` থাকলে সেটাও সরাও:

```
PermitRootLogin no
PasswordAuthentication no
```

মানে: root-এর সরাসরি ঢোকা বন্ধ, আর password দিয়ে ঢোকা বন্ধ — রইলো শুধু চাবি।

Licensed to sakilorb@gmail.com · Do not redistribute

> ⚠️ **সেভ করার আগে পড়ো:** এই দুই লাইন চালু করার পর চাবি-ছাড়া কেউ ঢুকতে পারবে না — **তুমিও না।** তাই ধাপ ২-এর পরীক্ষাটা (চাবি দিয়ে rahim হয়ে ঢোকা) সফল না হয়ে থাকলে **এখানে থেমে** আগে ওটা মেলাও। আর সোনালি নিয়ম: **পুরনো SSH-জানালাটা খোলা রেখেই** নতুন টার্মিনালে ঢোকার পরীক্ষা করো — নতুনটা ব্যর্থ হলে পুরনোটা দিয়ে ভুল শোধরানো যাবে। (অধ্যায় ১৬-র lockout-গল্পের শিক্ষা — সাবধানের মার নেই!)

সেভ করে নিয়ম কার্যকর করো:

```
sudo systemctl restart ssh
```

(`systemctl` — server-এর service-দের [পেছনে সারাক্ষণ চলা process — daemon-দের] চালু/বন্ধ/restart করার কমান্ড। এখন এটুকু জানাই যথেষ্ট, একটু পরে আরেকবার দেখা হবে।)

শেষে — দারোয়ান। অধ্যায় ১৬ থেকে হুবহু, **ক্রমটাসহ:**

```
sudo ufw allow 22
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable
sudo ufw status
```

তোমার server এখন একটা দুর্গ: ঢোকার পথ একটাই (চাবি), দরজা খোলা তিনটা, root অদৃশ্য, আর robot-রা দেয়ালে মাথা ঠুকছে। 🏰

Licensed to sakilorb@gmail.com · Do not redistribute

ধাপ ৪: Nginx — website-এর দোকান খোলা

এবার আসল মাল — **web server**: এমন একটা program যে ৮০/৪৪৩ দরজায় কান পেতে বসে থাকে, আর ব্রাউজার ডাকলে HTML পাঠিয়ে দেয়। জগতের সবচেয়ে জনপ্রিয়টার নাম **Nginx** (উচ্চারণ: এনজিন-এক্স)। বসানো — অধ্যায় ৭-এর দুই লাইন:

```
sudo apt install nginx
```

ব্যস, চলছেও — Ubuntu-তে ইনস্টলের সাথে সাথেই service চালু হয়ে যায়। বিশ্বাস হয় না? দুইভাবে প্রমাণ দেখো। এক — server নিজেকে নিজে ডাকুক (অধ্যায় ১৬-র `localhost` আর `curl`!):

```
curl localhost
```

HTML-এর ঢল নামলো — "Welcome to nginx!" লেখা কোথাও দেখা যাচ্ছে? আর দুই — **আসল মুহূর্তটা:** নিজের ফোন বা কম্পিউটারের ব্রাউজার খোলো, ঠিকানায় লেখো:

```
http://SERVER_IP
```

"Welcome to nginx!" — পৃথিবীর যেকোনো প্রান্ত থেকে, যে কেউ, এই মুহূর্তে এই পাতাটা দেখতে পাচ্ছে। বন্ধুকে ঠিকানাটা পাঠিয়ে দেখো! 🌍

ধাপ ৫: নিজের site তোলা

Nginx-এর welcome পাতা তো Nginx-এর — এবার **তোমারটা।** Nginx by default পাতা দেখায় `/var/www/html` ফোল্ডার থেকে (ওই যে `/var` — অধ্যায় ৪-এ চিনেছিলে)। ওখানে নিজের index.html বসালেই কাজ শেষ।

নিজের কম্পিউটারে (server-এ না!) একটা site বানাও:

Licensed to sakilorb@gmail.com · Do not redistribute

```
mkdir ~/amar-site
nano ~/amar-site/index.html
```

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Linu-r Prothom Website</title>
  <style>
    body { background: #0D1117; color: #eee; font-family: sans-
serif;
           display: flex; height: 100vh; align-items: center;
justify-content: center; }
    .card { text-align: center; }
    h1 { color: #2DD4BF; }
  </style>
</head>
<body>
  <div class="card">
    <h1>miyaw! 🐾</h1>
    <p>ei website ta cholche amar nijer server e —</p>
    <p>ja ami nije shajiyechi, "Linuke Posh Manano" boi pore.</p>
  </div>
</body>
</html>
```

(HTML না জানলেও হুবহু টুকে নাও — এ বই Linux-এর, HTML-এর গল্প অন্য বইয়ে। 😄)

এবার পাঠাও — কী দিয়ে পাঠাবে, বলো তো? **অধ্যায় ১৫!**

```
scp ~/amar-site/index.html rahim@SERVER_IP:/tmp/
```


Licensed to sakilorb@gmail.com · Do not redistribute

(সরাসরি `/var/www/html`-এ পাঠানো যাবে না — কেন? **অধ্যায় ১০:** ওই ফোল্ডারের মালিক root, rahim-এর সেখানে `w` নেই! তাই কৌশল: আগে `/tmp`-এ [অধ্যায় ৪: "আবর্জনার ঝুড়ি" — অস্থায়ী জিনিসের জায়গা], তারপর server-এ ঢুকে sudo দিয়ে সরানো।)

```
ssh rahim@SERVER_IP
sudo mv /tmp/index.html /var/www/html/index.html
```

ব্রাউজারে ফিরে যাও — `http://SERVER_IP` — **refresh!**

miyaw! 🐾

ওইটা তোমার পাতা। তোমার server-এ। তোমার সাজানো দুর্গের ভেতর থেকে, তোমার বসানো দোকানদার (Nginx) পৃথিবীকে দেখাচ্ছে। ঢাকার এক টেবিল থেকে তুমি সিঙ্গাপুরের এক যন্ত্রকে পোষ মানিয়ে পৃথিবীর সামনে দাঁড় করিয়ে দিয়েছো।

এই মুহূর্তটার একটা screenshot নাও। সত্যি বলছি — বছর দুয়েক পরে, যখন তুমি চোখ বুজে server সামলাবে, এই ছবিটা দেখে হাসবে আর মনে পড়বে — শুরুটা হয়েছিলো একটা কালো বিড়ালের গল্প দিয়ে।

ধাপ ৬ (শেষ পালিশ): লিনুর পুরনো বিদ্যাগুলো এখানে

Server তো দাঁড়ালো — এবার দেখো আগের পর্বগুলো এখানে কেমন জীবন্ত হয়ে ওঠে:

পর্ব ৪ (স্বাস্থ্য): `htop` চালাও server-এ — কতটুকু RAM খাচ্ছে nginx? `df -h` — ডিস্ক কত ফাঁকা? এই দুটো কমান্ড server-ওয়ালাদের রোজ সকালের চা।

অধ্যায় ৮ (tail -f): এইটা কোরো — ব্রাউজারে নিজের site খুলে রেখে server-এ:

Licensed to sakilorb@gmail.com · Do not redistribute

```
sudo tail -f /var/log/nginx/access.log
```

এবার ব্রাউজারে refresh দাও — log-এ **লাইন পড়লো, তোমার চোখের সামনে, live!** ওই লাইনটা তুমি। বন্ধুকে link দাও — তার ঢোকাও দেখবে। মনে আছে অধ্যায় ৮-এ বলেছিলাম, "সিনেমার হ্যাকারদের স্ক্রিনের ঝরঝরে লেখা আসলে tail -f"? আজ ওই স্ক্রিনটা তোমার। 😎

পর্ব ৫ (রাতের পাহারাদার): সেই backup.sh মনে আছে — আর WSL-এ cron-এর সীমাবদ্ধতার দুঃখ? **এই সেই মঞ্চ।** Script-টা server-এ তোলো (scp!), `UTSHO` বদলে দাও `/var/www/html`, crontab-এ বসাও — ব্যস: এমন এক যন্ত্রে রুটিন চলছে যেটা **কোনোদিন ঘুমায় না।** প্রতি রাত ৩টায়, বছরের প্রতিটা রাত।

একটা নতুন উপহার — service সামলানো: Nginx-এর মতো service-দের নিয়ন্ত্রণের চারটা হুকুম, `systemctl` দিয়ে:

```
sudo systemctl status nginx     # কেমন আছো? (সবুজ active দেখো!)
sudo systemctl restart nginx    # আবার চালু হও (settings বদলালে লাগে)
sudo systemctl stop nginx       # থামো (চালিয়ে দেখো — site অফ! তারপর start 😄)
sudo systemctl enable nginx     # server restart হলেও নিজে থেকে উঠো
```

এরপর নিজে নিজে (guided ইঙ্গিত, hand-holding ছাড়া)

বইয়ের শিক্ষা শেষ — এবার engineer-এর মতো নিজে explore করো। তিনটা অভিযান, ক্রমশ কঠিন:

1. **Site-টা সুন্দর করো** — HTML বদলাও, ছবি যোগ করো (`scp` দিয়ে তোলো), নিজের portfolio বানিয়ে ফেলো

Licensed to sakilorb@gmail.com · Do not redistribute

2. **একটা domain জোড়ো** — একটা সস্তা domain কিনে তার DNS-এ "A record"-এ server-এর IP বসাও — তারপর `http://tomarnam.com`! (DNS নতুন জগৎ — "domain A record কীভাবে যোগ করে" খুঁজলেই পথ পাবে)
3. **HTTPS-এর তালা লাগাও** — মনে আছে ৪৪৩ নম্বর দরজা খুলে রেখেছি কিন্তু ব্যবহার করিনি? **certbot** নামের একটা চমৎকার (আর ফ্রি!) যন্ত্র আছে যেটা তোমার site-এ 🔒 বসিয়ে দেয়। খোঁজো: "certbot nginx ubuntu" — দুটো কমান্ডের কাজ, কিন্তু খুঁজে বের করাটাই অনুশীলন

আটকে যাবে — **সেটাই পরিকল্পনা।** Error message পড়া, খোঁজা, log দেখা (`/var/log`!), আবার চেষ্টা — এই চক্রটাই আসল পেশা। বইয়ের ১৭ অধ্যায় তোমাকে হুকুম শিখিয়েছে; এই আটকে-যাওয়াগুলো তোমাকে engineer বানাবে।

📜 হুকুমনামা (server-জীবনের প্রথম সপ্তাহ)

কমান্ড	কাজ
`ssh root@IP` → `adduser` → `usermod -aG sudo`	কর্তা হয়ে ঢুকে সাধারণ user গড়া
`ssh-copy-id user@IP`	তালা বসানো — password-জীবনের শেষ দিন
`/etc/ssh/sshd_config`	SSH-এর নিয়মের খাতা (root no, password no)
`sudo systemctl status/restart/stop/enable নাম`	service সামলানো
`sudo apt install nginx`	দোকান খোলা
`/var/www/html/`	site-এর ফাইল যেখানে থাকে
`sudo tail -f /var/log/nginx/access.log`	অতিথি দেখা, live

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১৮: লিনুর নতুন নতুন কর্মস্থল — Cloud জুড়ে Deployment

> 🐾 **দৃশ্য ৩০: চাকরির অফারের চিঠি**

লিনুর প্রথম চাকরির খবর ছড়িয়ে পড়েছে। আর কী কাণ্ড — দরজার সামনে এখন **চিঠির স্তূপ!** সব চাকরির অফার। কোনোটা ছোট্ট ছিমছাম অফিস থেকে, কোনোটা আকাশছোঁয়া কাচের টাওয়ার থেকে, কোনোটায় আবার লেখা — "আপনাকে কিছুই সামলাতে হবে না, শুধু কাজটা দিন — বাকি সব আমরা!"

লিনু চিঠিগুলোর ওপর বসে তোমার দিকে তাকালো — "কোনটায় যাবো?"

অধ্যায় ১৭-তে তুমি লিনুকে একটা VPS-এ চাকরি দিয়েছো। কিন্তু বাস্তব জগতে server রাখার জায়গা একটা না — **অনেক**, আর ইন্টারভিউ থেকে চাকরি — সব জায়গায় এদের নাম শুনবে: AWS, Google Cloud, Azure, DigitalOcean, Railway... এই অধ্যায়ে আমরা এই পুরো মানচিত্রটা চিনবো, আর সবচেয়ে বড় সত্যটা নিজের হাতে প্রমাণ করবো: **ভেতরে ঢুকলে সবগুলোই সেই একই লিনু।** সাথে বোনাস — static পাতা ছাড়িয়ে এবার **আসল application** deploy করা শিখবো, professional কায়দায়।

কর্মস্থলের তিন জাত

চিঠিগুলো আসলে তিন জাতের অফিস থেকে এসেছে:

জাত ১: VPS — ছিমছাম অফিস। DigitalOcean, Hetzner, Vultr, Linode — অধ্যায় ১৭-তে যেটা করলে। তুমি পাও একটা ফাঁকা Linux মেশিন + public IP — ব্যস। সাজানো থেকে সামলানো,

Licensed to sakilorb@gmail.com · Do not redistribute

সব তোমার। **সুবিধা:** সস্তা, সরল, পুরো নিয়ন্ত্রণ, শেখার জন্য স্বর্গ। **দায়:** security থেকে backup — সব দায়িত্বও তোমার।

জাত ২: বড় Cloud — কাচের টাওয়ার। AWS (Amazon), GCP (Google), Azure (Microsoft)। এখানেও server ভাড়া পাওয়া যায় (AWS-এ নাম **EC2**, GCP-তে **Compute Engine**) — কিন্তু সাথে আরো শত শত সেবা: managed database, ফাইল storage, load balancer, AI সেবা... **সুবিধা:** যত বড় হও, সঙ্গে বাড়ে; চাকরির বাজারে এদের নাম সোনার হরফে। **দায়:** জটিলতা বেশি, আর দাম হিসাব না রাখলে মাস শেষে বিল দেখে মূর্ছা। 😅

জাত ৩: PaaS — "সব আমরা সামলাবো" অফিস। Railway, Render, Fly.io, Heroku। এখানে server-ই দেখতে পাবে না — শুধু কোড দাও (সাধারণত GitHub থেকে), ওরা নিজেরা build করে, চালায়, HTTPS-ও লাগিয়ে দেয়। **সুবিধা:** deploy পাঁচ মিনিটে, সামলানোর ঝামেলা শূন্য। **দায়:** নিয়ন্ত্রণ কম, খরচ বাড়ে দ্রুত, আর ভেতরে কী হচ্ছে না বুঝলে আটকে গেলে অসহায়।

> 💡 তাহলে এই বই এত কষ্ট করে VPS শেখালো কেন, PaaS-ই তো সোজা? কারণ ক্রমটা: **PaaS ব্যবহার করা সহজ, কিন্তু PaaS-এর ভেতরে কী ঘটছে সেটা VPS-শেখা মানুষই বোঝে।** যে Nginx-systemd-log নিজ হাতে চালিয়েছে, তার কাছে PaaS হলো আরাম; যে চালায়নি, তার কাছে PaaS হলো জাদুর কালো বাক্স — আর জাদু ভাঙলেই বিপদ। তুমি এখন প্রথম দলে।

কাচের টাওয়ারে এক দিন — AWS EC2

সবচেয়ে বড় নামটা দিয়েই শুরু করি। AWS-এ ঢুকে (aws.amazon.com — free tier আছে, নতুনদের জন্য ছোট server একদম ফ্রি বা নামমাত্র) **EC2** সেবায় গিয়ে "Launch instance" চাপলে যে ফর্মটা আসবে — সেটা পড়ে তুমি হাসবে, কারণ **প্রতিটা জিনিস তোমার চেনা**, শুধু পোশাক নতুন:

Licensed to sakilorb@gmail.com · Do not redistribute

AWS যা বলে	তুমি যেটা চেনো
AMI: Ubuntu Server LTS	কোন distro — অধ্যায় ১ 😄
Instance type (t3.micro…)	কত RAM/CPU-র মেশিন
Key pair	তোমার সেই **চাবি-তালা!** — অধ্যায় ১৫
Security group	দারোয়ানের খাতা — cloud-স্তরের **ufw**, অধ্যায় ১৬
Storage (gp3, 20 GiB)	ডিস্ক — `df -h` যেটা মাপে

Key pair-এ নিজের public key তুলে দাও (বা ওদের বানানোটা নামিয়ে নাও), Security group-এ ২২ আর ৮০ খোলো — লঞ্চ। মিনিটখানেক পর public IP হাজির, আর তারপর:

```
ssh ubuntu@EC2_IP
```

(AWS-এর Ubuntu image-এ default user-এর নাম `ubuntu` — root-ঢোকা আগেই বন্ধ করা, ভালো অভ্যাসটা ওরা নিজেরাই করে রেখেছে।) ঢুকে চালাও `htop`, `df -h`, `apt update` — **সব হুবহু এক।** এটাই এই অধ্যায়ের সবচেয়ে বড় উপহার: Amazon-এর টাওয়ার হোক বা Hetzner-এর ছিমছাম ঘর — **ভেতরে সেই একই Ubuntu, সেই একই লিনু, তোমার সেই একই ১৭ অধ্যায়ের বিদ্যা।** Cloud provider বদলানো মানে শুধু নতুন দরজা চেনা; ঘরের ভেতরটা তোমার মুখস্থ।

GCP-তেও গল্প এক — সেবার নাম Compute Engine, security group-এর নাম "firewall rules", default user তোমার Google নামে। Oracle Cloud-এর free tier-টাও জেনে রাখো — বাজারের সবচেয়ে উদার ফ্রি server অনেকদিন ধরে ওদেরই, শেখার মেশিনের জন্য দারুণ।

আসল Application Deploy — Static পাতার পরের ধাপ

অধ্যায় ১৭-তে Nginx সরাসরি একটা HTML ফাইল দেখিয়েছে। কিন্তু আসল জগতে তোমার থাকবে **application** — Python/Node/যেকোনো ভাষার একটা **চলন্ত process**, যে হিসাব করে,

Licensed to sakilorb@gmail.com · Do not redistribute

database ঘাঁটে, তারপর জবাব বানায়। তাকে ইন্টারনেটে তুলতে লাগে তিনটা টুকরার একটা চেইন — আর চেইন জিনিসটা তো তোমার রক্তে ঢুকে গেছে (পর্ব ৪!):

```
পৃথিবী → Nginx (৮০/৪৪৩ দরজায়) → তোমার app (ভেতরের একটা port-এ, যেমন ৮০০০)
```

Nginx এখানে **reverse proxy** — সদর দরজার অভ্যর্থনাকারী: অতিথি সামলায়, ভিড় সামলায়, তারপর ভেতরের ঘরে (তোমার app-এ) কথা পৌঁছে দেয়। App কখনো সরাসরি পৃথিবীর মুখোমুখি হয় না — এটাই professional নিয়ম।

চলো পুরোটা দাঁড় করাই — একটা ছোট্ট Python app দিয়ে (Python না জানলেও টুকে নাও — গঠনটাই শেখার বিষয়, ভাষা না)। Server-এ:

```
sudo apt install python3-pip python3-venv
mkdir ~/miyaw-app && cd ~/miyaw-app
python3 -m venv venv && source venv/bin/activate
pip install flask gunicorn
nano app.py
```

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>miyaw! ami ekta ashol application 🐾</h1>"
```

Gunicorn হলো Python app-এর production চালক (development-এর খেলনা server দিয়ে আসল জগতে নামতে নেই)। চালিয়ে দেখো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
gunicorn --bind 127.0.0.1:8000 app:app
```

আরেকটা টার্মিনালে (বা `Ctrl+Z` + `bg` — অধ্যায় ১১!) পরীক্ষা:

```
curl 127.0.0.1:8000
```

`miyaw!` ফেরত এলো? App বেঁচে আছে — কিন্তু শুধু **ভেতরে** (`127.0.0.1` মানে কী, অধ্যায় ১৬ বলেছে)। বাইরের পৃথিবী এখনো ওকে পায় না — এবং পাবেও না সরাসরি; পাবে Nginx-এর মাধ্যমে।

App-কে চাকরি দেওয়া — systemd Service

`Ctrl+C`-তে gunicorn মরে যায়, টার্মিনাল বন্ধেও মরে — চাকরি এমন হলে চলে? App-কে বানাতে হবে **service** — nginx যেমন: নিজে চলবে, ক্র্যাশ করলে নিজে উঠবে, server restart-এও নিজে উঠবে। অধ্যায় ১৭-তে `systemctl` দিয়ে service **সামলিয়েছো** — আজ নিজে একটা **বানাবে।** Service-দের নিয়োগপত্র থাকে `/etc/systemd/system/`-এ (আবারও `/etc` — নিয়মকানুনের ঘর!):

```
sudo nano /etc/systemd/system/miyaw.service
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
[Unit]
Description=Miyaw App
After=network.target

[Service]
User=rahim
WorkingDirectory=/home/rahim/miyaw-app
Environment="GOPON_KOTHA=linu-is-best"
ExecStart=/home/rahim/miyaw-app/venv/bin/gunicorn --bind
127.0.0.1:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

পড়ে দেখো — সবই চেনা ধারণা: কে চালাবে (`User` — অধ্যায় ১০!), কোথায় দাঁড়িয়ে (`WorkingDirectory`), কী চালাবে (`ExecStart` — **পুরো path**, cron-এর সেই শিক্ষা!), মরলে কী (`Restart=always`), আর `Environment` — গোপন কথা app-কে দেওয়ার সেই কায়দা (অধ্যায় ১২-র environment variable — বলেছিলাম না ফিরে আসবে!)। এবার নিয়োগ:

```
sudo systemctl daemon-reload        # নতুন নিয়োগপত্র পড়ো
sudo systemctl start miyaw          # কাজ শুরু
sudo systemctl enable miyaw         # restart-এও নিজে উঠবে
sudo systemctl status miyaw         # সবুজ active? 🎉
```

Nginx-কে অভ্যর্থনায় বসানো

শেষ টুকরা — Nginx-কে বলা: "৮০-তে যে-ই আসুক, ভেতরের ৮০০০-এ পৌঁছে দাও।" Nginx-এর সাইটের নিয়ম থাকে `/etc/nginx/sites-available/` -এ:

Licensed to sakilorb@gmail.com · Do not redistribute

```
sudo nano /etc/nginx/sites-available/miyaw
```

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

চালু করা মানে `sites-enabled` ফোল্ডারে এর একটা link বসানো, পুরনো default-টা সরিয়ে:

```
sudo ln -s /etc/nginx/sites-available/miyaw /etc/nginx/sites-
enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t                          # নিয়মে ভুল নেই তো? (সবসময় আগে এটা!)
sudo systemctl restart nginx
```

(`ln -s` = shortcut/link বানানো — নতুন ছোট্ট কমান্ড, উপহার। আর `nginx -t`-র `t` মানে test — config ভুল থাকলে restart-এর **আগেই** ধরা পড়বে, সাইট ডাউন হবে না। Professional-দের পাকা অভ্যাস।)

এবার ব্রাউজারে `http://SERVER_IP` —

miyaw! ami ekta ashol application 🐾

Licensed to sakilorb@gmail.com · Do not redistribute

পুরো চেইনটা দাঁড়িয়ে গেলো: পৃথিবী Nginx Gunicorn তোমার Python কোড। এটাই — **হুবহু এটাই** — আজকের লাখো production সাইটের কাঠামো। Django হোক, Node হোক, ভাষা যা-ই হোক — গঠন এই একটাই: app ভেতরের port-এ + systemd-র নিয়োগপত্র + Nginx-এর অভ্যর্থনা।

Deploy-এর সর্বজনীন Checklist

যেখানেই deploy করো — VPS, EC2, যেকোনো মেশিন — ক্রমটা এক। দেয়ালে টাঙানোর মতো করে দিলাম:

ধাপ	কাজ	বিদ্যা কোথাকার
১	Server + চাবি-তালায় ঢোকা, সাধারণ user	অধ্যায় ১৫, ১৭
২	`apt update && upgrade`, দরজা (ufw/security group)	অধ্যায় ৭, ১৬
৩	কোড তোলা (`rsync` / `scp` /git)	অধ্যায় ১৫
৪	App চালিয়ে ভেতরে পরীক্ষা (`curl 127.0.0.1:port`)	অধ্যায় ১৬
৫	systemd service — নিয়োগপত্র	এই অধ্যায়
৬	Nginx reverse proxy + `nginx -t`	এই অধ্যায়
৭	HTTPS (certbot)	অধ্যায় ১৭-র চ্যালেঞ্জ
৮	Backup script + cron	পর্ব ৫!
৯	Log-এ চোখ (`tail -f`, `journalctl -u miyaw`)	অধ্যায় ৮, ১৭

দেখেছো ব্যাপারটা? ন-টা ধাপের **আট-টাই** এই বইয়ের পুরনো অধ্যায় — deployment আসলে নতুন কোনো বিদ্যা না, তোমার সব বিদ্যার **সাজানো প্রয়োগ।**

Licensed to sakilorb@gmail.com · Do not redistribute

🛠️ হাতেকলমে

1. miyaw-app পুরোটা নিজের server-এ দাঁড় করাও — চেইনের চার স্তরের প্রতিটা আলাদা পরীক্ষা করো: gunicorn (`curl 127.0.0.1:8000`) → systemd (`status`) → nginx (`nginx -t`) → পৃথিবী (ব্রাউজার)
2. `sudo systemctl stop miyaw` করে ব্রাউজারে দেখো কী error আসে (502 Bad Gateway — চিনে রাখো: মানে "অভ্যর্থনাকারী আছে, ভেতরের ঘর ফাঁকা")। আবার start দাও
3. App-এ একটা লাইন বদলাও → `sudo systemctl restart miyaw` → refresh। অভিনন্দন — এটাই deploy-এর দৈনন্দিন চক্র!
4. `sudo journalctl -u miyaw | tail -n 20` — নিজের service-এর ডায়েরি পড়ো
5. একটা PaaS-ও চেখে দেখো: Railway বা Render-এ GitHub থেকে এই app-টাই তুলে দাও — পাঁচ মিনিটে হবে, আর প্রতিটা ধাপে তুমি **বুঝবে** ওরা ভেতরে কী করছে (build, port, env var...) — সেটাই পার্থক্য
6. ভাবনার প্রশ্ন: gunicorn `0.0.0.0:8000`-এ bind করে ufw-তে ৮০০০ খুলে দিলেও তো সরাসরি চলতো — Nginx মাঝে রাখলাম কেন? (ভাবো: HTTPS কে সামলাবে, একই server-এ দুটো app হলে কী হবে, static ফাইল কে দেবে)

Licensed to sakilorb@gmail.com · Do not redistribute

📜 হুকুমনামা

কমান্ড/জিনিস	কাজ
EC2 key pair / security group	চাবি-তালা / cloud-স্তরের ufw
`python3 -m venv venv` + `source venv/bin/activate`	Python-এর নিজস্ব ঘর
`gunicorn --bind 127.0.0.1:8000 app:app`	app-এর production চালক
`/etc/systemd/system/নাম.service`	নিজের service-এর নিয়োগপত্র
`sudo systemctl daemon-reload`	নতুন নিয়োগপত্র পড়াও
`/etc/nginx/sites-available/` + `ln -s`	সাইটের নিয়ম লেখা ও চালু
`sudo nginx -t`	restart-এর **আগে** config পরীক্ষা
`sudo journalctl -u নাম`	service-এর ডায়েরি
502 Bad Gateway	Nginx আছে, পেছনের app নেই

🐾 পর্ব ৬ — এবং মূল বই — শেষ।

দৃশ্য ১ মনে আছে? বৃষ্টির সন্ধ্যা, গেটের সামনে ভেজা এক বিড়ালছানা, আর তোমার সংশয় — "এ আবার কেমন প্রাণী, কথাই তো শোনে না।"

আজ দেখো — সেই প্রাণী তোমার হুকুমে ফাইল গোছায়, নিজের স্বাস্থ্যের খবর দেয়, রাত ৩টায় একা জেগে backup নেয়, আর এখন তো সিঙ্গাপুরের এক datacenter-এ বসে **চাকরিই করছে** — পৃথিবীকে তোমার বানানো পাতা দেখাচ্ছে। বদলটা কিন্তু লিনুর হয়নি — Linux সেই ১৯৯১ থেকে এমনই আছে। **বদলেছো তুমি।** কালো স্ক্রিনটা এখন আর দেয়াল না — দরজা।

লিনু এখন তোমার। আর তুমি জানো ওকে কীভাবে ডাকতে হয়।

Licensed to sakilorb@gmail.com · Do not redistribute

মিঁয়াও। 🐾

Licensed to sakilorb@gmail.com · Do not redistribute



পরিশিষ্ট

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট ক: লিনুর হুকুমনামা 📜 — জরুরি Command Cheat Sheet

(ছাপিয়ে দেয়ালে লাগানোর জন্য — সিরিয়াসলি, লাগিয়ো 😄)

পথঘাট: `pwd` আমি কোথায় · `ls -la` সব দেখাও · `cd জায়গা` যাও · `cd` বাসায় · `cd -` আগের জায়গায় · **Tab** = নাম লিখে দাও

জিনিসপত্র: `mkdir -p` ঘর বানাও · `touch` খালি ফাইল · `cp -r` কপি · `mv` সরাও/নাম বদলাও · `rm -r` মোছো ⚠️ ফেরে না

খোঁজা: `find জায়গা -name "নাম"` ফাইল খোঁজো · `grep -rn "লেখা" জায়গা` লেখার ভেতরে খোঁজো · `which` কমান্ড কোথায়

পড়া-লেখা: `cat` পুরোটা · `less` পাতা উল্টে (`q` বের) · `head` / `tail` মাথা/লেজ · `tail -f` live · `nano` লেখো (`^O` সেভ `^X` বের) · vim-মুক্তি: `Esc :q!`

খাওয়ানো: `sudo apt update` মেনু · `sudo apt upgrade` খাওয়াও · `sudo apt install/remove নাম` · `apt search` খোঁজো

চেইন: `|` জোড়ো · `>` ফাইলে (মুছে!) · `>>` ফাইলে (জুড়ে) · `wc -l` গোনো · `sort` সাজাও · `uniq -c` জোড়া গুনো · `cut -d' ' -f1` টুকরা কাটো

অনুমতি: `ls -l` নিয়ম পড়ো · `chmod +x` চালানোর অনুমতি · `chmod u/g/o±rwx` · `sudo chown user:group` মালিক বদল · `sudo` এক হুকুমের কর্তৃত্ব

Licensed to sakilorb@gmail.com · Do not redistribute

স্বাস্থ্য: `htop` ড্যাশবোর্ড· `ps aux | grep নাম` খোঁজো · `kill PID` থামাও (`-9` শেষ অস্ত্র) · `df -h` ডিস্ক · `du -sh` এটা কত বড় · `free -h` RAM · `&` পেছনে · `jobs` / `fg` দেখা/আনা

নিজের মতো: `echo $VAR` · `export` · `$PATH` খোঁজার তালিকা · `alias নাম='কমান্ড'` · `~/.bashrc` অভ্যাসের খাতা · `source ~/.bashrc` · `Ctrl+R` ইতিহাসে খোঁজো · `sudo !!` আগেরটা sudo-সহ

Script: `#!/bin/bash` প্রথম লাইন · `$1 $2` argument · `if [ ]; then fi` · `for X in তালিকা; do done` · `$(কমান্ড)` output বসাও · `exit 1`

রুটিন: `crontab -e` খাতা খোলো · `মিনিট ঘণ্টা দিন মাস বার কমান্ড` (ছোট□বড়!) · সব path পুরো লেখো · crontab.guru মিলিয়ে নাও

বাইরের জগৎ: `ssh user@IP` ঢোকো · `ssh-keygen -t ed25519` চাবি-তালা · `ssh-copy-id` তালা বসাও · `scp` / `rsync -avz` ফাইল পাঠাও · `ping` বেঁচে আছো · `curl` ডাকো · `sudo ufw allow 22` আগে, `enable` পরে! · `sudo systemctl status/restart নাম`

সর্বজনীন: `Ctrl+C` থামো · `q` বের হও · `man কমান্ড` অভিধান · error পড়ো □ Google করো □ এটাই পেশা 😄

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট খ: লিনু কথা না শুনলে — সাধারণ Error ও সমাধান

`command not found` লিনু এই ডাক চেনে না। তিনটা সম্ভাবনা, এই ক্রমে দেখো: (১) বানান — case-sensitive, মনে আছে? (২) জিনিসটা ইনস্টল আছে তো? (`sudo apt install ...`) (৩) ইনস্টল আছে কিন্তু `$PATH` -এ নেই (অধ্যায় ১২)।

`Permission denied` লিনু বলছে "তোমার অনুমতি নেই।" আগে ভাবো — **থাকার কথা কি?** নিজের script হলে: `chmod +x` । সিস্টেমের জিনিস হলে: বুঝে তারপর `sudo` । SSH-এ পেলে: চাবি-তালার গোলমাল — ঠিক user-এ ঢুকছো তো? চাবিটা `~/.ssh/` -এ আছে তো?

`No such file or directory` পথ ভুল। `pwd` দিয়ে দেখো কোথায় দাঁড়িয়ে, `ls` দিয়ে দেখো আশপাশে আসলে কী আছে — নামের বানান/বড়হাত মিলছে তো? আর Tab দিয়ে লিখলে এ ভুল হওয়ারই কথা না। 😄

`apt` **-এ** `Could not get lock` **/** `is locked` দোকানে একসাথে দুজন ঢুকেছে — অন্য কোনো apt চলছে (হয়তো সিস্টেমের নিজের আপডেট)। এক-দুই মিনিট অপেক্ষা করে আবার। বারবার হলে: `ps aux | grep apt` — কে আটকে আছে দেখো।

Script চালালে `syntax error near unexpected token` ৯০% ক্ষেত্রে: `[ ]` -এর ভেতরের স্পেস বাদ পড়েছে, বা `then` / `do` / `fi` / `done` -এর কোনোটা হারিয়েছে। অধ্যায় ১৩-র গঠনের সাথে লাইন ধরে মেলাও।

Cron-এ কাজ "চলছে না" চারটা প্রশ্ন, ক্রমে: (১) `crontab -l` -এ লাইনটা আছে তো? (২) ক্রম ঠিক তো — **মিনিট আগে, ঘণ্টা পরে**? (৩) **সব path পুরো লেখা** তো — script-এর, ফাইলের? (৪) script-টা হাতে চালালে চলে তো? (`chmod +x` ছিলো?)

Licensed to sakilorb@gmail.com · Do not redistribute

SSH: `Connection refused` / `Connection timed out` refused = বাড়ি আছে, দরজা বন্ধ (sshd চলছে না, বা ভুল port)। timed out = ডাকই পৌঁছাচ্ছে না (ভুল IP, নেট নেই, বা firewall-এ ২২ বন্ধ — lockout হয়ে গেলে provider-এর website-এর "console" দিয়ে ঢুকে `ufw allow 22`)।

টার্মিনাল "hang" — কিছু লিখলে কিছু হয় না সম্ভবত কোনো কমান্ড চলছে বা কোনো program-এর ভেতরে ঢুকে গেছো। অস্ত্র, ক্রমে: `q` (pager?) → `Ctrl+C` (চলন্ত কমান্ড?) → `Esc :q!` (ভুলোর ঘর?) → `Ctrl+D` (কোনো shell/program-এর ভেতরে?)। কিছুতেই না হলে জানালা বন্ধ করে নতুন খোলো — লজ্জার কিছু নেই, সবাই করে। 😄

সব স্লো / জমে যাচ্ছে `htop` — ওপরে কে? মেমোরি শেষ হলে দোষী সাধারণত ব্রাউজার; CPU ১০০% হলে দেখো কোন process। `kill` জানোই।

সর্বজনীন পদ্ধতি: error-টা **পুরো পড়ো** (অর্ধেক পড়ে অনুমান না) → বুঝলে ঠিক করো → না বুঝলে **হুবহু** কপি করে খোঁজো — লাখো মানুষ এই পথ আগে হেঁটেছে, উত্তর প্রায় সবসময় প্রথম পাতাতেই।

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট গ: এরপর কী — লিনুর সাথে সামনের রাস্তা

বইটা শেষ, কিন্তু রাস্তাটা সবে শুরু। তোমার এখনকার ভিত থেকে স্বাভাবিক পরের ধাপগুলো — **ক্রম মেনে এগোলে** সবচেয়ে মসৃণ লাগবে:

১. Git ও GitHub (এখনই, যদি না জানো) — কোডের ইতিহাস রাখার যন্ত্র, প্রতিটা developer-এর দৈনন্দিন সঙ্গী। টার্মিনাল জানো বলে Git শেখা তোমার জন্য এখন সহজ — আর SSH key তো বানানোই আছে! 😄

২. নিজের ভাষার server-জগৎ — Python জানলে (Journey by Python-এর যাত্রী?) এবার Django/Flask app বানিয়ে **এই বইয়ের server-এই** deploy করো। Nginx তখন static পাতার বদলে তোমার app-এর সামনে দাঁড়াবে (খোঁজার শব্দ: "reverse proxy")। এই মুহূর্তটা — নিজের লেখা app নিজের server-এ — ক্যারিয়ারের অন্যতম মধুর মাইলফলক।

৩. Docker — পুরো পরিবেশটাকে (app + dependencies + settings) একটা বাক্সে ভরে যেকোনো মেশিনে চালানো। "আমার কম্পিউটারে তো চলছিলো!" সমস্যার স্থায়ী সমাধান। Linux-এর ভিত ছাড়া Docker ধোঁয়াশা লাগে — তোমার লাগবে না।

৪. CI/CD — GitHub-এ push করা মাত্র নিজে নিজে পরীক্ষা নিজে নিজে server-এ deploy। পর্ব ৫-এর automation দর্শনেরই বড় সংস্করণ (খোঁজো: GitHub Actions)।

৫. Cloud (AWS/GCP/Azure) — তোমার VPS-অভিজ্ঞতারই শিল্প-মাপের রূপ: শত server, database-সেবা, load balancer। VPS নিজ হাতে সাজিয়েছো বলে cloud-এর ধারণাগুলো তোমার কাছে চেনা লাগবে — অধিকাংশ শিক্ষার্থীর যেটা লাগে না।

Licensed to sakilorb@gmail.com · Do not redistribute

এই পাঁচ ধাপের রাস্তাটার একটা নাম আছে — **DevOps**। আর লক্ষ্য সেদিকে না হলেও ক্ষতি নেই: backend, data, security, ML — যেদিকেই যাও, এই বইয়ের ভিত সঙ্গে যাবে। Linux হলো সেই বিদ্যা যেটা কোনো দিকে গেলেই নষ্ট হয় না।

শেষ উপদেশ: **প্রতিদিন টার্মিনাল খোলো।** দরকার না থাকলেও। ফাইল গোছাও টার্মিনালে, খোঁজো `grep`-এ, ছোট কাজে ছোট script। দক্ষতা মুখস্থে আসে না — আঙুলে আসে। লিনুকে রোজ আদর না করলে ও অভিমান করে। 🐾

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট ঘ: Resources ও Community

হাতের কাছে সবসময়: - `man` আর `--help` — সেরা reference তোমার টার্মিনালেই - **explainshell.com** — যেকোনো জটিল কমান্ড পেস্ট করো, টুকরো টুকরো করে বুঝিয়ে দেবে (ইন্টারনেট থেকে পাওয়া কমান্ড চালানোর আগে এখানে দেখা — চমৎকার অভ্যাস) - **crontab.guru** — cron-এর অনুবাদক (অধ্যায় ১৪ থেকেই চেনা)

গভীরে যেতে: - **linuxjourney.com** — ধাপে ধাপে, ফ্রি, চমৎকার সাজানো - **OverTheWire: Bandit** — খেলার ছলে টার্মিনাল-দক্ষতা: ধাঁধা ভাঙতে ভাঙতে level পার হবে। এই বই শেষ করা কারো জন্য প্রথম কয়েক level পানিভাত — তারপর জমবে খেলা 😄 - Ubuntu-র official documentation (help.ubuntu.com) — server-এর কাজে নির্ভরযোগ্য

আটকে গেলে: - Error message হুবহু খোঁজো — Stack Overflow, Ask Ubuntu (askubuntu.com) — পৃথিবীর তাবৎ আটকে-যাওয়া মানুষের প্রশ্নোত্তর জমা আছে - AI সহকারীদেরও জিজ্ঞেস করতে পারো — তবে একটা নিয়ম মেনো: **উত্তর কপি করার আগে বুঝে নাও কমান্ডটা কী করছে** (explainshell মনে আছে?)। বুঝে চালানো আর না-বুঝে চালানোর মধ্যেই engineer আর দুর্ঘটনার পার্থক্য — বিশেষত `sudo` আর `rm` থাকলে

দেশি community: - Facebook-এ বাংলাদেশের developer group-গুলোতে যুক্ত হও — প্রশ্ন করো, আর কিছুদিন পর **উত্তরও দিও**: শেখার সবচেয়ে দ্রুত রাস্তা কাউকে শেখানো - নিজের শেখার গল্প লেখো — আজকের তোমার সমস্যাটা ছয় মাস আগের তোমার মতো কারো জন্য সোনার খনি

আর সবশেষে — এই বই পড়ে কী বানালে, কোথায় আটকালে, কোন অধ্যায়টা সবচেয়ে কাজে লাগলো — জানাতে ভুলো না। লেখকের জন্য পাঠকের "আমি পেরেছি!" বার্তার চেয়ে বড় সম্মানী হয় না।

Licensed to sakilorb@gmail.com · Do not redistribute

মিঁয়াও — শেষবারের মতো না, প্রথমবারের মতো। 🐾

Licensed to sakilorb@gmail.com · Do not redistribute



— সমাপ্ত —

মিঁয়াও — শেষবারের মতো না, প্রথমবারের মতো।

Licensed to sakilorb@gmail.com · Do not redistribute